



รายงาน Mini Project

วิชา 240-319 Embedded System Developer Module

หุ่นยนต์เสิร์ฟอาหารอัจฉริยะ Smart Food Delivery Robot

จัดทำโดย

นายธีรภัทร	ภักดี	รหัสนักศึกษา	6710110199	Section 01
นายพัทธรินทร์	กิตติภูมิพงศ์	รหัสนักศึกษา	6710110282	Section 01
นายสรวิศ	สุขการณ์	รหัสนักศึกษา	6710110428	Section 01

เสนอ

รศ.ดร.	ปัญญาศ	ไชยกาพ
ผศ.ดร.	วชรินทร์	แก้วอภิชัย
รศ.ดร.	ทวิศักดิ์	เรืองพีระกุล

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา

240-319 ชุมติวิชานักพัฒนาระบบฝังตัว (EMBEDDED SYSTEM DEVELOPER MODULE)

มหาวิทยาลัยสงขลานครินทร์ วิทยาเขตหาดใหญ่

ภาคเรียนที่ 1 ปี การศึกษา 2569

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา 240-319 ชุติวิชานักพัฒนาระบบฝังตัว (Embedded System Developer Module) จัดทำขึ้นเพื่อศึกษา ออกแบบ และพัฒนาระบบสมองกลฝังตัวที่สามารถนำไปประยุกต์ใช้งานได้จริง การจัดทำโครงการในครั้งนี้ช่วยให้ผู้จัดทำได้ฝึกทักษะการปฏิบัติ พร้อมทั้งเสริมสร้างความรู้และความเข้าใจในเนื้อหาของรายวิชาให้ชัดเจนมากยิ่งขึ้นทั้งในด้านทฤษฎีและการนำไปประยุกต์ใช้จริง

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์แก่ผู้อ่านหรือนักศึกษาที่กำลังศึกษาข้อมูลเกี่ยวกับการพัฒนาระบบ Embedded System หากมีข้อเสนอแนะหรือข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับคำแนะนำและขออภัยมา ณ ที่นี้ด้วย

คณะผู้จัดทำ

สารบัญ

คำนำ	ก
สารบัญ	ข
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	2
1.3 ขอบเขตของโครงการ	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
1.5 นิยามศัพท์เฉพาะ	2
บทที่ 2 แนวคิด ทฤษฎี และเอกสารที่เกี่ยวข้อง	4
2.1 ระบบฝังตัวและสถาปัตยกรรมของหุ่นยนต์	4
2.1.1 แนวคิดของระบบฝังตัว	4
2.1.2 การจัดการงานของระบบหุ่นยนต์	5
2.2 อินพุต เอาต์พุต และการควบคุมมอเตอร์	6
2.2.1 GPIO, interrupt และ PWM	6
2.2.2 มอเตอร์กระแสตรงและวงจร H-bridge	7
2.2.3 Encoder และการวัดความเร็ว	7
2.2.4 การควบคุมความเร็วแบบป้อนกลับ	8
2.3 การเคลื่อนที่แบบ DIFFERENTIAL DRIVE และ ODOMETRY	10
2.4 เซนเซอร์และการตรวจจับสิ่งกีดขวาง	12
2.4.1 Keypad 4x4 และตัวขยายพอร์ต PCF8574	12
2.4.2 LiDAR และเขตหยุดเพื่อความปลอดภัย	13
2.5 การสื่อสารและการจัดการสถานะ	15
2.5.1 Serial communication และ ROS 2	15
2.5.2 Finite State Machine และการทำงานตามเวลา	17
2.6 ระบบจ่ายไฟและวงจรขับเคลื่อน	17
2.7 สรุป	18

บทที่ 3 วิธีการดำเนินการ	20
3.1 การทำงานโดยภาพรวม	20
3.1.1 ลำดับสถานะการส่งอาหาร	20
3.2 การทำงานของฮาร์ดแวร์	22
3.2.1 ผังการเชื่อมต่อและระบบจ่ายไฟ	22
3.2.2 Raspberry Pi จอสัมผัส และ LiDAR	23
3.2.3 Arduino Uno ระบบขับเคลื่อน และ Encoder	24
3.2.4 Keypad และจอ MAX7219 Dot Matrix	25
3.2.5 การกำหนดการใช้งานของ Arduino Uno	26
3.2.6 อุปกรณ์หลักที่ใช้ในระบบ	28
3.3 การพัฒนาโปรแกรมและหลักการทำงาน	29
3.3.1 โปรแกรมควบคุมบน Arduino Uno	29
3.3.2 โปรแกรมควบคุมบน Raspberry Pi	34
3.3.3 รูปแบบข้อมูลที่ใช้สื่อสารระหว่าง Raspberry Pi กับ Arduino	48
3.4 ขั้นตอนการประกอบและตรวจสอบระบบ	51
บทที่ 4 ผลการดำเนินงาน	53
4.1 ผลการประกอบหุ่นยนต์และการเชื่อมต่อระบบ	53
4.1.1 ภาพหุ่นยนต์ต้นแบบหลังประกอบ	53
4.1.2 การติดตั้ง Raspberry Pi, Arduino, LiDAR, มอเตอร์ และแบตเตอรี่	54
4.2 ผลการทำงานของจอสัมผัส POS/KIOSK และ KEYPAD	57
4.2.1 การเลือกชั้นวางอาหาร	57
4.2.2 การเลือกโต๊ะปลายทาง	57
4.2.3 การยืนยันอาหารและเริ่มภารกิจ	59
4.2.4 การยืนยันรับอาหารผ่านจอสัมผัสหรือ Keypad	60
4.3 ผลการทดสอบการเคลื่อนที่ของหุ่นยนต์	61
4.3.1 การเดินหน้า ถอยหลัง และหยุด	61
4.3.2 การหมุนอยู่กับที่	61
4.3.3 ผลจาก Encoder และ Odometry	63

4.3.4. การเดินทางตาม Waypoint	63
4.3.5. การแสดงลูกศรบนจอ LED MATRIX	64
4.4 ผลการทำงานของระบบตรวจจับสิ่งกีดขวางและความปลอดภัย	65
4.4.1. การหยุดเมื่อ LiDAR ตรวจพบสิ่งกีดขวาง	65
4.4.2. การเดินทางต่อเมื่อเส้นทางปลอดภัย	67
4.5 ผลการทดสอบภารกิจส่งอาหาร	68
4.5.1. การส่งอาหารไปยังโต๊ะเดียว	68
4.5.2. การส่งอาหารไปยังหลายโต๊ะ	69
4.5.3. การร่อยรับอาหาร	71
4.5.4. การกลับมายังจุดเริ่มต้น	71
4.5.5. การยกเลิกหรือรีเซ็ตภารกิจ	73
4.6 สรุปผลการทดสอบระบบโดยรวม	74
4.7 ปัญหาและข้อจำกัดที่พบ	75
บทที่ 5 สรุป และอภิปรายผล	76
5.1 สรุปผลการดำเนินงาน	76
5.2 อภิปรายผล	77
5.3 ข้อเสนอแนะ	78
5.3.1 แนวทางการนำไปใช้	78
5.3.2 ข้อจำกัดของโครงการ	79
5.3.3 แนวทางการปรับปรุงและพัฒนาต่อ	79
ภาคผนวก	81
ภาคผนวก ก การวิเคราะห์ต้นทุน	81
ภาคผนวก ข ภาพการทำงาน	86

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

ธุรกิจอาหารและเครื่องดื่มเป็นภาคธุรกิจที่มีความสำคัญต่อเศรษฐกิจไทย โดยข้อมูลธุรกิจอาหารและเครื่องดื่มที่จดทะเบียนกับกรมพัฒนาธุรกิจการค้าในปี 2567 มีรายได้รวมประมาณ 337.2 พันล้านบาท และธุรกิจร้านอาหารมีรายได้ประมาณ 308.8 พันล้านบาท หรือคิดเป็นร้อยละ 91.6 ของรายได้ธุรกิจอาหารและเครื่องดื่มที่จดทะเบียนทั้งหมด ขณะเดียวกัน สำนักงานนโยบายและยุทธศาสตร์การค้ารายงานว่า ตลาดหุ่นยนต์บริการในประเทศไทยเติบโตกว่าร้อยละ 147.8 ในปี 2565 มีหุ่นยนต์บริการที่ใช้งานจริงมากกว่า 1,660 ตัว และคาดการณ์ว่าจะเพิ่มเป็น 2,270 ตัวในปี 2567 ตัวเลขเหล่านี้สะท้อนให้เห็นถึงขนาดของอุตสาหกรรมอาหารและแนวโน้มการประยุกต์ใช้เทคโนโลยีเพื่อสนับสนุนการให้บริการ อย่างไรก็ตามจากการศึกษายังพบปัญหาการบริการในร้านอาหารขนาดเล็กได้ เช่น การนำอาหารจากครัวไปส่งยังโต๊ะลูกค้ายังคงเป็นงานที่ต้องทำซ้ำและอาจเกิดความล่าช้าหรือความผิดพลาดได้ โดยเฉพาะในช่วงที่มีผู้ใช้บริการจำนวนมาก จึงมีความสนใจในการนำระบบอัตโนมัติมาช่วยสนับสนุนการขนส่งอาหารภายในร้าน

ระบบสมองกลฝังตัว (Embedded System) และระบบอัตโนมัติ (Automation) มีบทบาทสำคัญต่อการดำเนินชีวิตและการพัฒนาเศรษฐกิจในสังคมปัจจุบัน ระบบสมองกลฝังตัวช่วยให้อุปกรณ์ต่าง ๆ สามารถรับข้อมูลจากสภาพแวดล้อม ประมวลผลข้อมูล และตอบสนองตามเงื่อนไขที่กำหนดได้อย่างเหมาะสม ขณะที่ระบบอัตโนมัติช่วยลดขั้นตอนการทำงานที่ซ้ำซ้อน เพิ่มความสม่ำเสมอในการปฏิบัติงาน และสนับสนุนการทำงานที่ต้องการความรวดเร็วหรือความต่อเนื่อง เทคโนโลยีทั้งสองจึงถูกนำไปประยุกต์ใช้ในหลายด้าน เช่น อุตสาหกรรม การแพทย์ การคมนาคม เกษตรกรรม บ้านอัจฉริยะ และธุรกิจบริการ นอกจากนี้ยังเพิ่มประสิทธิภาพของกระบวนการทำงานแล้ว ยังช่วยให้มนุษย์สามารถมุ่งเน้นงานที่ต้องใช้การตัดสินใจ การดูแลผู้รับบริการ และความคิดสร้างสรรค์ได้มากขึ้น การออกแบบระบบดังกล่าวจึงควรคำนึงถึงความถูกต้อง ความปลอดภัย ความน่าเชื่อถือ ต้นทุน และความเหมาะสมกับบริบทการใช้งาน

จากความสำคัญของปัญหาและบทบาทของเทคโนโลยีดังกล่าว คณะผู้จัดทำจึงสนใจพัฒนาต้นแบบหุ่นยนต์ส่งอาหารอัตโนมัติสำหรับช่วยลำเลียงอาหารภายในพื้นที่จำลอง โดยมุ่งประยุกต์ใช้ความรู้ ในขอบเขตของวิชา Embedded System Developer Module ได้แก่ ระบบสมองกลฝังตัว การควบคุมอุปกรณ์ การสื่อสารระหว่างระบบ การตรวจจับสภาพแวดล้อม และการนำทางอัตโนมัติ โครงการนี้จึงเป็นแนวทางสำหรับศึกษาและบูรณาการเทคโนโลยีที่เกี่ยวข้องกับการพัฒนาระบบอัตโนมัติในงานบริการ

1.2 วัตถุประสงค์

เพื่อศึกษาการประยุกต์ใช้ความรู้จากวิชา 240-319 Embedded System Developer Module มาสร้างต้นแบบหุ่นยนต์ส่งอาหาร (Food Delivery Robot) สำหรับร้านอาหารขนาดเล็ก

1.3 ขอบเขตของโครงการ

โครงการนี้ครอบคลุมการออกแบบและพัฒนาหุ่นยนต์ส่งอาหารต้นแบบ -ขับเคลื่อนแบบ Differential Drive เหมาะสมสำหรับร้านอาหารขนาดเล็ก โดยทดสอบในพื้นที่จำลองภายในอาคารและใช้เส้นทางกับตำแหน่งที่กำหนดไว้ล่วงหน้า ระบบรองรับการเลือกชั้นวางและโต๊ะปลายทางผ่านระบบ POS หรือ Keypad รวมถึงการ ควบคุมการเคลื่อนที่ การสื่อสารระหว่างระบบ การตรวจจับสิ่งกีดขวาง การสื่อสารระหว่างอุปกรณ์ และการเคลื่อนที่ในพื้นที่จำลอง

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ได้ต้นแบบหุ่นยนต์ส่งอาหารอัตโนมัติสำหรับทดสอบการทำงานในพื้นที่จำลอง เพื่อแก้ปัญหาการบริการของพนักงานเสิร์ฟอาหาร
2. ได้ศึกษาแนวทางการประยุกต์ใช้ระบบสมองกลฝังตัวและระบบอัตโนมัติกับงานบริการ การการควบคุม การสื่อสาร การตรวจจับสภาพแวดล้อม และการนำทางอัตโนมัติ
3. ได้แนวทางสำหรับพัฒนาระบบช่วยขนส่งอาหารในร้านอาหารขนาดเล็ก
4. ได้พื้นฐานสำหรับการพัฒนาหุ่นยนต์บริการที่มีความสามารถสูงขึ้นในอนาคต

1.5 นิยามศัพท์เฉพาะ

ระบบสมองกลฝังตัว (Embedded System) หมายถึง ระบบคอมพิวเตอร์ที่ออกแบบให้ทำงานเฉพาะด้าน โดยสามารถรับข้อมูล ประมวลผล และควบคุมอุปกรณ์ตามวัตถุประสงค์ที่กำหนด

ระบบอัตโนมัติ (Automation) หมายถึง การใช้ระบบควบคุมและเทคโนโลยีเพื่อให้กระบวนการทำงานดำเนินไปตามเงื่อนไขที่กำหนด โดยลดการควบคุมจากมนุษย์ในขั้นตอนที่ทำซ้ำ

หุ่นยนต์ส่งอาหาร (Food Delivery Robot) หมายถึง

หุ่นยนต์บริการที่ใช้ช่วยลำเลียงอาหารจากจุดเตรียมอาหารไปยังจุดหมายภายในพื้นที่ที่กำหนด

Differential Drive หมายถึง รูปแบบการขับเคลื่อนที่ควบคุมความเร็วของล้อซ้ายและล้อขวาแยกจากกัน เพื่อกำหนดทิศทางและการหมุนของหุ่นยนต์

Waypoint หมายถึง จุดตำแหน่งหรือจุดอ้างอิงที่กำหนดไว้บนเส้นทาง เพื่อใช้ควบคุมลำดับการเดินทางของหุ่นยนต์

Encoder หมายถึง อุปกรณ์ที่ตรวจวัดการหมุนหรือการเคลื่อนที่ของมอเตอร์และแปลงเป็นสัญญาณสำหรับใช้ประเมินระยะทางหรือความเร็ว

LiDAR หมายถึง เซนเซอร์ที่ใช้แสงเลเซอร์วัดระยะห่างจากวัตถุ เพื่อรับรู้สภาพแวดล้อมและตรวจจับสิ่งกีดขวาง

ROS 2 หมายถึง เฟรมเวิร์กสำหรับพัฒนาซอฟต์แวร์หุ่นยนต์และแลกเปลี่ยนข้อมูลระหว่างส่วนต่าง ๆ ของระบบ

SLAM หมายถึง กระบวนการที่หุ่นยนต์สร้างแผนที่ของพื้นที่พร้อมประมาณตำแหน่งของตนเองขณะเคลื่อนที่

POS/Kiosk หมายถึง ส่วนติดต่อผู้ใช้สำหรับรับรายการอาหาร เลือกจุดหมาย และจัดการข้อมูลภารกิจของหุ่นยนต์

บทที่ 2

แนวคิด ทฤษฎี และเอกสารที่เกี่ยวข้อง

บทนี้นำเสนอแนวคิดและหลักการทางวิศวกรรมที่ใช้เป็นพื้นฐานในการออกแบบหุ่นยนต์ส่งอาหาร ครอบคลุมสถาปัตยกรรมระบบฝังตัว การรับสัญญาณ การควบคุมมอเตอร์ การประมาณตำแหน่ง การตรวจสอบสิ่งกีดขวาง การสื่อสาร และการจัดการภารกิจ แต่ละหัวข้ออธิบายหลักการพร้อมเชื่อมโยงกับระบบของโครงการ เพื่อให้เห็นหน้าที่และข้อจำกัดของแนวทางที่เลือก

2.1 ระบบฝังตัวและสถาปัตยกรรมของหุ่นยนต์

2.1.1 แนวคิดของระบบฝังตัว

ระบบฝังตัว (embedded system) คือระบบคอมพิวเตอร์ที่ออกแบบให้ทำงานเฉพาะด้านภายใต้ข้อจำกัดของอุปกรณ์และสภาพแวดล้อม ตัวอย่างงาน ได้แก่ การอ่านสัญญาณจากเซนเซอร์ การประมวลผลตามเงื่อนไข และการควบคุมอุปกรณ์เอาต์พุต การออกแบบจึงต้องคำนึงถึงความถูกต้องของผลลัพธ์ เวลาตอบสนอง ความสม่ำเสมอในการทำงาน ทรัพยากรหน่วยความจำและพลังงาน การจัดการเมื่อเกิดข้อผิดพลาดสำหรับงานที่มีข้อกำหนดด้านเวลาจริง (real-time constraint) ผลลัพธ์ต้องถูกต้องและเกิดขึ้นภายในเวลาที่กำหนด หากพลาดกำหนดเวลา คุณภาพการควบคุมอาจลดลง แม้ค่าที่คำนวณได้จะถูกต้อง

ไมโครคอนโทรลเลอร์รวมหน่วยประมวลผล หน่วยความจำ และวงจรต่อพ่วง เช่น ขารับส่งสัญญาณดิจิทัล (GPIO) ตัวนับเวลา ตัวสร้างสัญญาณ PWM วงจรขัดจังหวะ (interrupt) และพอร์ตสื่อสารไว้ในชิปเดียว Arduino Uno R3 ซึ่งใช้ ATmega328P ดังรูปที่ 1 ตัว Arduino มีทรัพยากรเหล่านี้สำหรับเชื่อมต่ออุปกรณ์และควบคุมงานตามคาบ การควบคุมงานระดับล่างโดยไม่ต้องพึ่งระบบปฏิบัติการทั่วไปช่วยให้ตอบสนองต่อสัญญาณ encoder และอัปเดตคำสั่งมอเตอร์ได้โดยตรง อย่างไรก็ตาม ผู้ออกแบบยังต้องจัดสรรขา I/O หน่วยความจำ และเวลาประมวลผลให้เหมาะสม

คอมพิวเตอร์บอร์ดเดียว (single-board computer: SBC) เช่น Raspberry Pi ตัวอย่างดังรูปที่ 2 มีทรัพยากรประมวลผลและระบบปฏิบัติการที่เอื้อต่องานระดับสูง เช่น การจัดการภารกิจ ส่วนติดต่อผู้ใช้ การสื่อสารผ่านเครือข่าย และการประมวลผลข้อมูลจากเซนเซอร์ แม้ระบบปฏิบัติการทั่วไปจะช่วยให้พัฒนาซอฟต์แวร์ได้สะดวก แต่ก็ไม่ได้รับประกันความหน่วงหรือเวลาตอบสนองที่แน่นอนในทุกสถานการณ์ การมอบหมายงานควบคุมมอเตอร์ที่ต้องอัปเดตต่อเนื่องให้ไมโครคอนโทรลเลอร์ และให้งานจัดการภารกิจอยู่บน SBC จึงช่วยแยกขอบเขตและลดการพึ่งพากันระหว่างซอฟต์แวร์แต่ละระดับ



รูปที่ 1 บอร์ด Arduino UNO R3 ที่ใช้ ATmega328P เป็นตัวประมวลผลหลัก
(ที่มา: https://en.wikipedia.org/wiki/Arduino_Uno)



รูปที่ 2 บอร์ด Raspberry Pi 4
(ที่มา: https://en.wikipedia.org/wiki/Arduino_Uno)

2.1.2 การจัดการงานของระบบหุ่นยนต์

ระบบประกอบด้วย Raspberry Pi 4 Model B หน่วยความจำ 4 GB และ Arduino Uno R3 หนึ่งบอร์ด Arduino อ่านค่า encoder และ keypad ควบคุมมอเตอร์ซ้ายและขวาผ่านวงจรขับ L298N และสั่งแสดงลูกศรเลี้ยวผ่าน LED Matrix ขนาด 32x8 ส่วน Raspberry Pi ทำงาน POS/Kiosk จัดการสถานะภารกิจและตำแหน่ง waypoint รับข้อมูล LiDAR และแลกเปลี่ยนคำสั่งหรือข้อมูลสถานะกับ Arduino ผ่าน USB Serial ซอฟต์แวร์รองรับการเชื่อมต่อสองรูปแบบ ได้แก่ Serial โดยตรงและการส่งข้อมูลผ่าน ROS 2

การแบ่งหน้าที่นี้ทำให้งานควบคุมระดับโกลบอลแวร์ เช่น การอ่าน encoder และการควบคุมความเร็วล้อ อยู่บน Arduino ขณะที่การจัดลำดับภารกิจและการติดต่อผู้ใช้อยู่บน Raspberry Pi จอสัมผัสเป็นช่องทางหลักของ POS/Kiosk ส่วน keypad 4x4 เป็นช่องทางเพิ่มเติมที่ใช้งานทดแทนกันได้

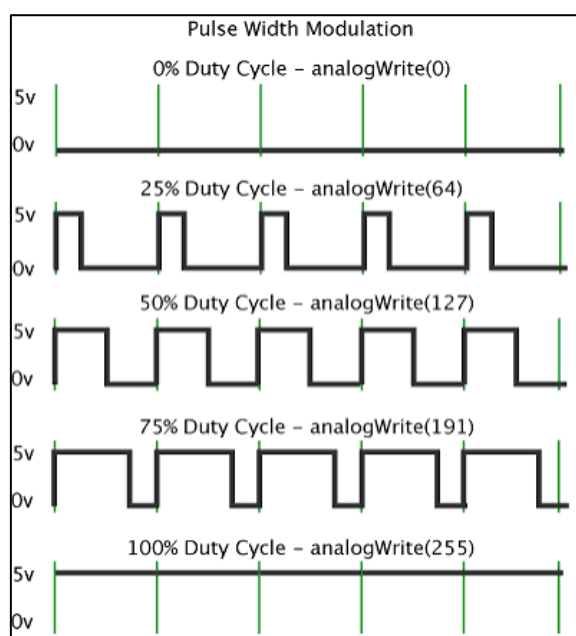
2.2 อินพุต เอาต์พุต และการควบคุมมอเตอร์

2.2.1 GPIO, interrupt และ PWM

ขา GPIO (general-purpose input/output) ใช้รับหรือส่งสถานะดิจิทัล เช่น อ่านปุ่มกำหนดทิศทางการหมุน หรือควบคุมสัญญาณ enable การอ่านอินพุตแบบวนตรวจ (polling) เหมาะกับสัญญาณที่เปลี่ยนไม่บ่อย แต่ encoder อาจสร้างพัลส์ต่อเนื่องระหว่างที่โปรแกรมทำงานส่วนอื่น วงจร interrupt แจ้งให้หน่วยประมวลผลจัดการเหตุการณ์เมื่อสถานะขาอินพุตเปลี่ยน โดยไม่ต้องรอรอบตรวจถัดไป จึงช่วยลดโอกาสพลาดพัลส์เมื่อเลือกใช้อย่างเหมาะสม

โครงการใช้ Pin Change Interrupt บนพอร์ต C ของ ATmega328P เพื่อติดตามสัญญาณ encoder โดยนับขอบขาขึ้นของเฟส A และอ่านระดับสัญญาณเฟส B เพื่อระบุทิศทาง เนื่องจาก interrupt และลูปหลักเข้าถึงตัวแปร tick ร่วมกัน การอ่านตัวแปรหลายไบต์จึงต้องป้องกันไม่ให้ค่าเปลี่ยนระหว่างคัลลอกในโครงการ firmware จึงปิด interrupt ชั่วคราวขณะอ่านค่า แล้วเปิดใช้งานอีกครั้งทันที

การปรับความกว้างพัลส์ (pulse-width modulation: PWM) เป็นการปรับสัดส่วนเวลาที่สัญญาณอยู่ในระดับสูงต่อหนึ่งคาบ หรือ duty cycle เพื่อควบคุมกำลังเฉลี่ยที่ส่งไปยังโหลด ฟังก์ชัน `analogWrite()` ของ Arduino สร้างสัญญาณ PWM บนขาที่รองรับ ไม่ได้สร้างแรงดันแอนะล็อกต่อเนื่องโดยตรง ในโครงการใช้ PWM ควบคุมขา enable ของวงจรขับ ส่วนขาดิจิทัลแยกกันใช้กำหนดชั่วและทิศทางการหมุนของมอเตอร์



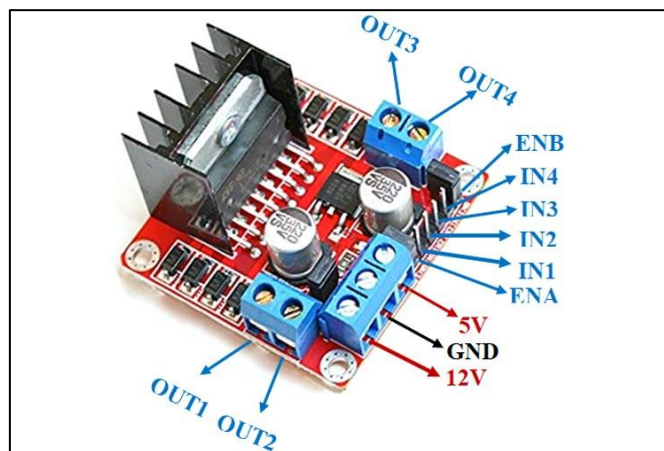
รูปที่ 3 ตัวอย่างการส่ง PWM ในระดับ Duty Cycle ต่าง ๆ

(ที่มา: <https://docs.arduino.cc/learn/microcontrollers/analog-output/>)

2.2.2 มอเตอร์กระแสตรงและวงจร H-bridge

มอเตอร์กระแสตรง (direct-current motor: DC motor) เปลี่ยนพลังงานไฟฟ้าให้เป็นการหมุนของเพลา ความเร็วและแรงบิดขึ้นอยู่กับแรงดัน กระแส โหลดเชิงกล และคุณสมบัติของมอเตอร์ การกลับขั้วแรงดันที่ขั้วมอเตอร์ทำให้ทิศทางการหมุนเปลี่ยนไป แต่หาไม่คอนโทรลเลอร์ไม่สามารถจ่ายกระแสให้มอเตอร์ได้โดยตรง จึงต้องใช้วงจรขับเคลื่อนเป็นตัวกลาง

วงจร H-bridge ใช้ชุดสวิตช์กำหนดทิศทางการกระแสและขั้วแรงดันที่จ่ายให้มอเตอร์ จึงรองรับการหมุนได้สองทิศทาง L298 เป็นวงจรขับเคลื่อนแบบ dual full-bridge สำหรับโหลดเหนี่ยวนำ เช่น มอเตอร์ DC ในงานนี้ Arduino จะมีหน้าที่ส่งสัญญาณกำหนดทิศทางและ PWM ไปยัง L298N เพื่อควบคุมล้อซ้ายและขวาแยกกัน



รูปที่ 4 มอดูล L298N H-Bridge

(ที่มา: <https://components101.com/modules/l293n-motor-driver-module>)

2.2.3 Encoder และการวัดความเร็ว

Rotary encoder แปลงการหมุนของเพลาให้เป็นพัลส์ดิจิทัล เมื่อนับจำนวนพัลส์ต่อรอบและจำนวนพัลส์ในช่วงเวลาที่กำหนด จะสามารถประมาณระยะทางและความเร็วเชิงเส้นของล้อได้ หากล้อมีรัศมี R และ encoder มีความละเอียด N tick ต่อรอบ ระยะทางที่ล้อเคลื่อนที่ต่อหนึ่ง tick คำนวณได้จาก

$$\Delta s_{tick} = \frac{2\pi R}{N}$$

หากจำนวน tick เปลี่ยนไป Δn ในช่วงเวลา Δt ระยะทางและความเร็วเฉลี่ยของล้อประมาณได้จาก

$$\Delta s = \Delta n \Delta s_{tick}, \approx \frac{\Delta s}{\Delta t}$$

ระบบนี้กำหนดรัศมีล้อ $R=0.065m$ และความละเอียด $N=1920 \text{ tick/rev}$ จึงได้ระยะทางเชิงทฤษฎีประมาณ $2.13 \times 10^{-4} m/tick$ หรือประมาณ 0.213 mm ต่อ tick ค่านี้เป็นความสัมพันธ์จากพารามิเตอร์ที่กำหนด ไม่ใช่ผลการวัดความแม่นยำของตัวรถ เนื่องจากค่าจริงยังขึ้นกับความละเอียด encoder การกำหนดขอบสัญญาณใน firmware และลักษณะการสัมผัสระหว่างล้อกับพื้น ซึ่งเป็นปัจจัยที่อาจทำให้เกิดความคลาดเคลื่อน

2.2.4 การควบคุมความเร็วแบบป้อนกลับ

ระบบควบคุมแบบป้อนกลับ (feedback control) นำค่าที่วัดได้มาเปรียบเทียบกับค่าเป้าหมาย แล้วนำความคลาดเคลื่อนที่ได้ไปปรับคำสั่งควบคุม ตัวควบคุม PID ประกอบด้วยส่วนสัดส่วน (proportional: P) ส่วนอินทิกรัล (integral: I) และส่วนอนุพันธ์ (derivative: D) สมการในโดเมนเวลาต่อเนื่องเขียนโดยย่อได้ดังนี้

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, e(t) = r(t) - y(t)$$

โดย $r(t)$ คือค่าเป้าหมาย $y(t)$ คือค่าที่วัดได้ และ $u(t)$ คือคำสั่งควบคุม ส่วน P ตอบสนองต่อความคลาดเคลื่อนปัจจุบัน ส่วน I สะสมความคลาดเคลื่อนตามเวลา และส่วน D พิจารณาอัตราการเปลี่ยนแปลงของความคลาดเคลื่อน การนำ PID ไปใช้บนไมโครคอนโทรลเลอร์เป็นการคำนวณแบบไม่ต่อเนื่อง ณ เวลาสุ่มตัวอย่าง T_s จึงประมาณอินทิกรัลด้วยผลรวมของตัวอย่าง และประมาณอนุพันธ์ด้วยผลต่างระหว่างตัวอย่าง การจำกัดเอดจ์พุตและผลรวมอินทิกรัลช่วยลดปัญหา integral windup เมื่อคำสั่งขับถึงขีดจำกัด

ในโครงงาน Arduino ใช้ตัวควบคุม PID แยกสำหรับล้อซ้ายและล้อขวา โดยอัปเดตวงควบคุมประมาณทุก 20 ms หรือ 50 Hz ความเร็วที่คำนวณจาก tick จะผ่านตัวกรองเอ็กซีโพเนนเชียลก่อนนำไปป้อนกลับ ส่วนอนุพันธ์คำนวณจากการเปลี่ยนแปลงของความเร็วที่วัดได้ นอกจากนี้ firmware ยังเพิ่ม feed-forward ตามความเร็วเป้าหมายและจำกัดผลรวมอินทิกรัลไว้ในช่วงที่กำหนด วิธีนี้ใช้ทั้งการชดเชยจากแบบจำลองอย่างง่ายและการแก้ความคลาดเคลื่อนจาก encoder โดยผลลัพธ์ยังขึ้นกับการปรับค่าให้เหมาะกับมอเตอร์และโหลดจริง

เมื่อนำหลักการดังกล่าวมาเขียนในรูปแบบไม่ต่อเนื่อง ให้ T_s เป็นช่วงเวลาสุ่มตัวอย่าง $v_{raw}[k]$ เป็นความเร็วที่คำนวณจาก tick และ $v_f[k]$ เป็นความเร็วหลังการกรอง การคำนวณใน Arduino สรุปได้โดยประมาณดังนี้

$$v_f[k] = 0.85v_f[k-1] + 0.15v_{raw}[k],$$

$$e[k] = r[k] - v_f[k]$$

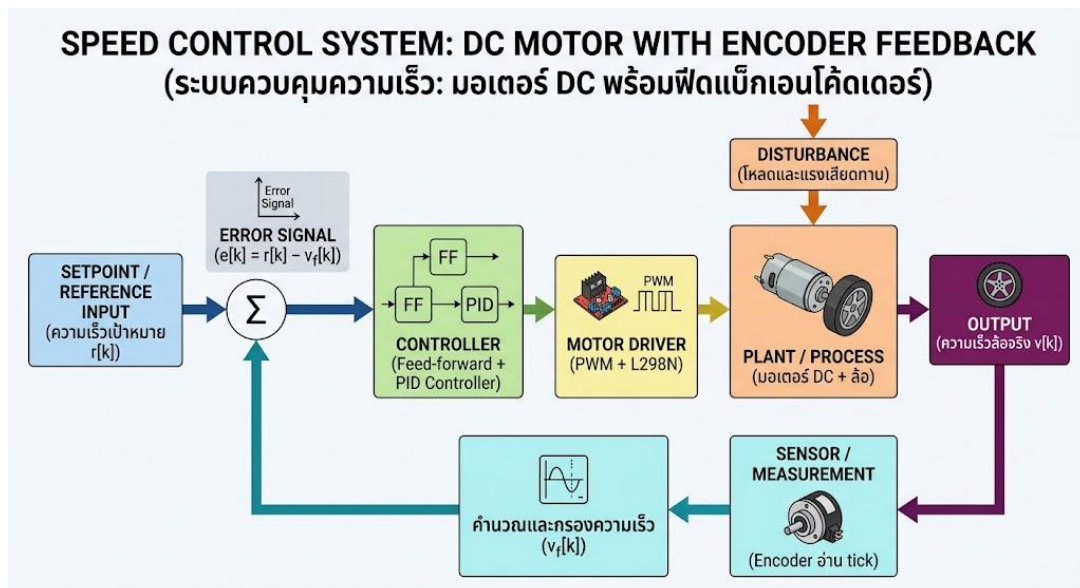
$$I[k] = \text{clip}(I[k-1] + e[k]T_s, -2, 2),$$

$$D_v[k] = \frac{v_f[k] - v_f[k-1]}{T_s}$$

$$u[k] = \frac{r[k]}{v_{cruise}} u_{ff} + K_p e[k] + K_i I[k] - K_d D_v[k]$$

ในสมการนี้ ถูกจำกัดให้อยู่ในช่วง PWM ที่ฮาร์ดแวร์รองรับ และกำหนดค่า PWM ขั้นต่ำเมื่อมีคำสั่งเคลื่อนที่ เพื่อไม่ให้ผลคำนวณที่มีค่าน้อยทำให้คำสั่งหายไป ค่า gain, feed-forward และขีดจำกัดต่าง ๆ เป็นค่าที่ตั้งไว้สำหรับ firmware ของโครงงาน จึงไม่ใช่ค่าทั่วไปที่รับรองสมรรถนะ กับมอเตอร์ทุกชนิด

เมื่อหุ่นยนต์เคลื่อนที่ตรง firmware เปรียบเทียบระยะสะสมของล้อทั้งสองเพื่อปรับ ตำแหน่งเป้าหมายแยกกันและลดการเบี่ยงเบนจากเส้นทางตรง ช่วงเริ่มต้นจะเพิ่มความเร็วเป้าหมายเป็นขั้น และลดความเร็วเมื่อเข้าใกล้ระยะปลายทาง หากมีการวางตำแหน่งเป้าหมายแต่ encoder ไม่แสดงการเคลื่อนที่ ระบบจะเพิ่ม PWM อย่างจำกัด และรายงานสถานะ fault เมื่อไม่พบการเคลื่อนที่ต่อเนื่องเกินเวลาที่กำหนด กลไกเหล่านี้ เป็นวิธีควบคุมและจัดการความผิดปกติของ firmware



รูปที่ 5 แผนภาพ Feedback Loop ของระบบควบคุมมอเตอร์ภายในหุ่นยนต์

2.3 การเคลื่อนที่แบบ Differential Drive และ Odometry

หุ่นยนต์แบบ Differential Drive ขับเคลื่อนด้วยล้อสองข้างที่ควบคุมความเร็วแยกจากกัน โดยล้อทั้งสองมีแกนหมุนร่วมกันและล้อประกอบช่วยพยุงตัวรถ ให้ v_L และ v_R เป็นความเร็วเชิงเส้นของล้อซ้ายและขวา และให้ d เป็นระยะระหว่างศูนย์กลางล้อขับ ความเร็วเชิงเส้นของจุดกึ่งกลางรถ v และความเร็วเชิงมุมรอบแกนตั้ง ω เป็นดังนี้

$$v = \frac{v_R + v_L}{2},$$

$$\omega = \frac{v_R - v_L}{d}$$

เมื่อความเร็วของล้อทั้งสองเท่ากัน หุ่นยนต์จะเคลื่อนที่เป็นเส้นตรงโดยประมาณ ส่วนความเร็วที่ต่างกันทำให้รถเคลื่อนที่เป็นแนวโค้ง หากล้อหมุนสวนทางกันด้วยความเร็วใกล้เคียงกัน รถจะหมุนรอบตัวเองแบบจำลองนี้ใช้แปลงคำสั่งการเคลื่อนที่เป็นความเร็วของล้อ และใช้ประมาณการเปลี่ยนตำแหน่งจาก encoder อย่างไรก็ตาม แบบจำลองเชิงจลนศาสตร์ไม่ได้อธิบายแรง มวล หรือพลวัตของมอเตอร์โดยละเอียด

Odometry คือการประมาณตำแหน่งและทิศทางจากการเคลื่อนที่สะสมของล้อ ในช่วงเวลาหนึ่งให้การเคลื่อนที่ของล้อซ้ายและขวาเป็น Δs_L และ Δs_R จะได้การเคลื่อนที่ของจุดกึ่งกลางและการเปลี่ยน heading ดังนี้

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2},$$

$$\Delta \theta = \frac{\Delta s_R - \Delta s_L}{b}$$

การอินทิเกรตตำแหน่งด้วยวิธี midpoint ประมาณได้จาก

$$x_{k+1} = x_k + \Delta s \cos(\theta_k + \Delta \theta / 2),$$

$$y_{k+1} = y_k + \Delta s \sin(\theta_k + \Delta \theta / 2),$$

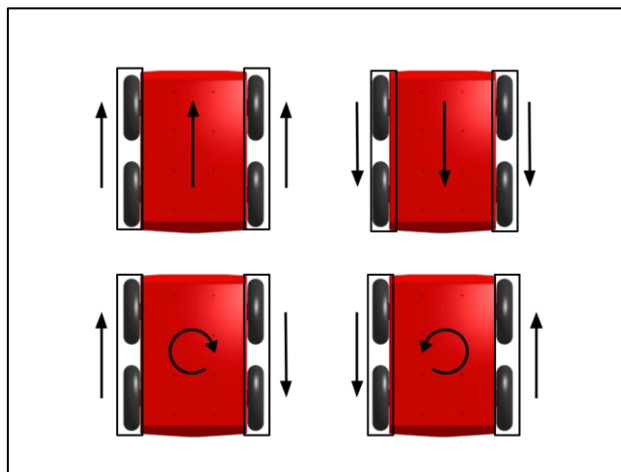
$$\theta_{k+1} = \theta_k + \Delta \theta$$

ในระบบนี้ Raspberry Pi ใช้ tick ซ้ายและขวาที่รับจาก Arduino เพื่อคำนวณ pose โดยกำหนดค่ารัศมีล้อ $R=0.065m$, ระยะห่างล้อ $b=0.343m$ และความละเอียด $N=1920 \text{ tick/rev}$ เส้นทางภารกิจใช้ waypoint ที่กำหนดตำแหน่งสถานีทางแยกโตะไ่ว์ล่วงหน้า จากนั้นใช้ตำแหน่งและ

heading จาก odometry เพื่อควบคุมช่วงทางและการเลี้ยว วิธีนี้มีโครงสร้างตรงไปตรงมา และเหมาะกับเส้นทางที่ทราบรูปแบบ แต่ไม่เทียบเท่ากับการรู้ตำแหน่งสัมบูรณ์ ของรถ

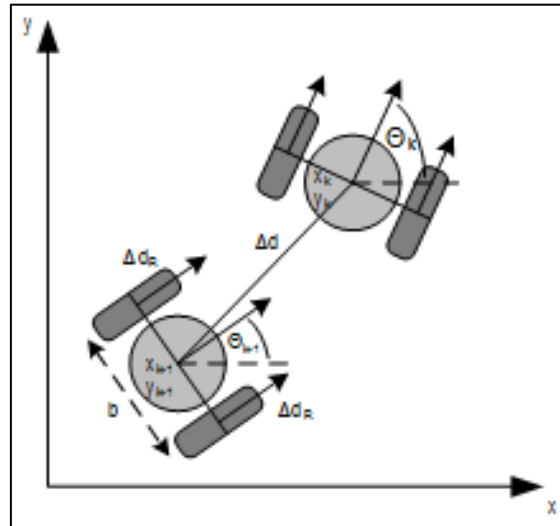
ในการออกแบบโจทย์สำหรับทดลอง จะกำหนดกรอบพิกัดของเส้นทางกำหนดสถานีเริ่มต้นเป็น $(0,0)$ และทิศออกจากสถานีเป็นแกน $+X$ ค่าเริ่มต้นใน configuration กำหนดทางแยกที่ $(2.0,0)$ m และกำหนดตำแหน่งโต๊ะ 1 กับโต๊ะ 2 ที่ $(2.0,+0.6)$ m และ $(2.0,-0.6)$ m ตามลำดับตัวควบคุม waypoint ใช้เป้าหมายเหล่านี้เพื่อแยกการเดินทางเป็นช่วงตรงและช่วงปรับ heading โดยใช้ pose ปัจจุบัน จาก odometry เป็น feedback เส้นทางจึงเป็นเส้นทางที่ระบุไว้ล่วงหน้า ไม่ใช่การค้นหาเส้นทางอิสระจากแผนที่ในทุกครั้งที่เดินทาง เพราะถือว่าในโจทย์การทดสอบเป็นร้านอาหารขนาดเล็ก ไม่จำเป็นต้องมีการคำนวณทิศทางซับซ้อน

เนื่องจาก odometry คำนวณตำแหน่งจากการเคลื่อนที่สะสม (dead reckoning) ความคลาดเคลื่อน จึงเพิ่มขึ้นตามระยะทาง ความคลาดเคลื่อนเชิงระบบอาจเกิดจากเส้นผ่านศูนย์กลางล้อซ้ายและขวาที่ไม่เท่ากัน หรือจากระยะห่างล้อจริงที่ต่างจากค่าที่ใช้คำนวณ ขณะที่การลื่นไถล การกระจายน้ำหนัก และพื้นผิวทำให้เกิดความคลาดเคลื่อนที่เปลี่ยนแปลงตามสภาวะ จึงมีขั้นตอนการแก้ไข โดยอ้างอิงจากงานของ Borenstein และ Feng ซึ่งวิเคราะห์วิธีวัดและแก้ความคลาดเคลื่อนเชิงระบบจากขนาดล้อและระยะห่างล้อ จึงสนับสนุนความจำเป็นในการสอบ เทียบพารามิเตอร์กับตัวรถจริง



รูปที่ 6 ลักษณะการขับเคลื่อนด้วยวิธี Differential Drive โดยการสั่งให้ล้อไปทางเดียวกันคือการไปข้างหน้า หรือไปหลัง ส่วนการสั่งให้ล้อขับเคลื่อนตรงข้ามกันด้วยความเร็วเท่ากันจะทำให้หุ่นยนต์หมุนรอบตัวเอง

(ที่มา: https://existentialrobotics.org/RobotProvingGrounds/algorithms/control/content/diffy_drive/)



รูปที่ 7 การคำนวณตำแหน่งของหุ่นยนต์บนแผนที่โดยหลักการ Odometry โดยจะใช้ค่า รัศมีล้อ ระยะห่างระหว่างล้อ ระยะทางที่ล้อขับแต่ละข้าง โดยได้ผลลัพธ์ออกมาคือตำแหน่ง และมุมของหุ่นยนต์
(ที่มา: <https://hackaday.io/project/158496-imcoders/log/147068-robot-odometry>)

2.4 เซนเซอร์และการตรวจจับสิ่งกีดขวาง

2.4.1 Keypad 4x4 และตัวขยายพอร์ต PCF8574

Keypad แบบเมทริกซ์จัดปุ่มเป็นแถวและคอลัมน์ ทำให้ไมโครคอนโทรลเลอร์อ่านปุ่มหลายปุ่มได้โดยใช้สายสัญญาณน้อยกว่าการต่อปุ่มแต่ละปุ่มแยกกัน PCF8574 เป็นตัวขยายพอร์ตดิจิทัล 8 บิตที่สื่อสารผ่าน บัส I2C ซึ่งใช้สายข้อมูล SDA และสายนาฬิกา SCL การต่อ keypad ผ่านตัวขยายพอร์ตช่วยลดจำนวนขา GPIO ที่ต้องใช้และสงวนขาอื่นไว้สำหรับ encoder และมอเตอร์

ในโครงงาน PCF8574 เชื่อมต่อสาย SDA และ SCL กับขา A4 และ A5 ของ Arduino Uno ตามลำดับ โดยใช้ address เริ่มต้น 0x20 กำหนดขา P7-P4 เป็นแถวและ P3-P0 เป็นคอลัมน์ เมื่อมีการกดปุ่ม Arduino จะส่งข้อความเหตุการณ์ที่ขึ้นต้นด้วย KEY: พร้อมอักขระของปุ่มไปยัง Raspberry Pi จากนั้น POS จะตีความคำสั่งตามสถานะภารกิจ



รูปที่ 8 Keypad แบบเมทริกซ์ขนาด 4x4
(ที่มา: <https://electropeak.com/flat-keypad-4x4>)



รูปที่ 9 PCF8574 สำหรับการเชื่อมต่อ Keypad
(ที่มา: <https://www.letscontrolit.com/wiki/index.php/PCF8574>)

2.4.2 LiDAR และเขตหยุดเพื่อความปลอดภัย

LiDAR แบบระยะวัดระยะถึงวัตถุจากหลายมุมและส่งผลเป็นข้อมูลเชิงขั้ว ข้อมูล LaserScan ของ ROS 2 ระบุขอบเขตมุม ช่วงเพิ่มของมุมต่อหนึ่งตัวอย่าง ช่วงระยะที่ใช้งานได้ และค่าระยะของลำแสงแต่ละตัวอย่างข้อมูลนี้อาจใช้แสดงสภาพแวดล้อม สร้างแผนที่ หรือตรวจเช็คด้านหน้าเพื่อสั่งหยุด การเลือกใช้งานขึ้นกับวัตถุประสงค์และอัลกอริทึมของระบบ การติดตั้ง LiDAR เพียงอย่างเดียวจึงไม่ได้ทำให้หุ่นยนต์นำทางหรือหลบสิ่งกีดขวางได้ครบถ้วน

หากให้ระยะของลำแสงตัวที่ i เป็น r_i และมุมเป็น $\phi_i = \text{angle}_{\min} + i \text{ angle}_{\max}$ จุดวัดในระนาบสามารถเขียนในพิกัดคาร์ทีเซียนได้เป็น $x_i = r_i \cos \phi_i$ และ $y_i = r_i \sin \phi_i$ การแปลงนี้ช่วยเชื่อมข้อมูลมุมกับตำแหน่งที่ตรวจพบเมื่อพิจารณาเทียบกับกรอบพิกัดของเซนเซอร์และตัวรถ

ซอฟต์แวร์ตรวจจับจาก LiDAR พิจารณาเฉพาะพื้นที่รูปกรวยด้านหน้าหุ่นยนต์ โดยคัดค่าที่ไม่ใช่ตัวเลข ค่าที่ไม่เป็นจำนวนจำกัด หรือค่าระยะที่อยู่นอกช่วงที่กำหนด แล้วนับจำนวนจุดภายในเขตหยุด ค่าเริ่มต้นคือ มุมครึ่งกรวย 35 องศา ระยะหยุด 0.30 m ระยะใกล้สุดที่พิจารณา 0.22 m และกำหนดให้พบอย่างน้อยสาม จุดก่อนรายงานสิ่งกีดขวาง ค่าระยะหยุดปรับได้ผ่านตัวแปรสภาพแวดล้อม เมื่อพบสิ่งกีดขวาง ตัวควบคุม waypoint จะหยุดคำสั่งเดินหน้าและรอจนพื้นที่กลับมาโล่ง โดยระบบต้องได้รับข้อมูล LiDAR ผ่าน ROS 2 ในหัวข้อ /scan

วิธีการกำหนดจำนวนจุดขั้นต่ำช่วยลดการตอบสนองต่อค่าผิดปกติจากลำแสงเพียงจุดเดียว อย่างไรก็ตามระบบนับจำนวนจุดทั้งหมดที่ผ่านเงื่อนไขในกรวย โดยไม่ได้จัดกลุ่มจุดหรือกำหนดว่าจุดต้อง ต่อเนื่องกัน ตามมุม จึงใช้จำนวนจุดนี้ระบุรูปทรงหรือแยกวัตถุไม่ได้ นอกจากนี้ระยะหยุดเป็นค่าเกณฑ์คงที่ ไม่ได้คำนวณจากความเร็วปัจจุบัน ระยะเบรก หรือความหน่วงในการสื่อสาร

ดังนั้น กลไกนี้ทำหน้าที่เป็น virtual bumper ที่สั่งหยุดเมื่อข้อมูล LiDAR ผ่านเกณฑ์ที่กำหนด ขอบเขตการทำงานไม่ครอบคลุมการจำแนกวัตถุ การวางแผนเส้นทางหลบหลีก หรือการรับรองระยะหยุด ภายใต้อุปกรณ์ การประเมินในสภาพใช้งานจริงยังต้องพิจารณาดำเนินการติดตั้ง ชนิดพื้นผิว รูปร่างของวัตถุ และความเร็วของหุ่นยนต์



รูปที่ 10 RPLiDAR A1 สำหรับการตรวจจับวัตถุ

(ที่มา: <https://core-electronics.com.au/slamtec-rplidar-a1-360-laser-range-scanner.html>)

2.5 การสื่อสารและการจัดการสถานะ

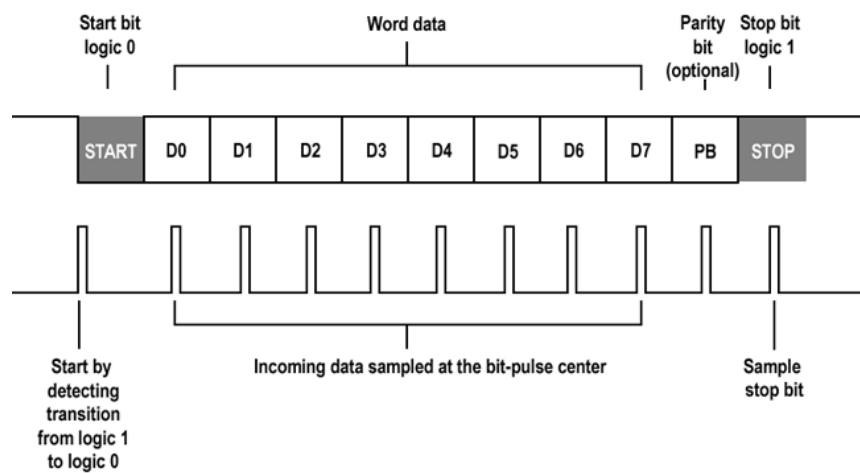
2.5.1 Serial communication และ ROS 2

การสื่อสารแบบอนุกรม (serial communication) ส่งข้อมูลเป็นลำดับระหว่างอุปกรณ์ ผู้ส่งและผู้รับ ต้องใช้อัตราบิตและรูปแบบการรอบข้อมูลที่สอดคล้องกัน Arduino กำหนดอัตราบิตด้วยคำสั่ง `Serial.begin()` ในโครงงาน Arduino เชื่อมต่อกับ Raspberry Pi ผ่าน USB Serial ที่ 115200 baud และใช้การขึ้นบรรทัดใหม่เป็นตัวแบ่งข้อความ วิธีนี้ช่วยให้โปรแกรมปลายทางรับข้อมูลที่ละบรรทัดและจำแนกประเภทจากค่านำหน้าได้

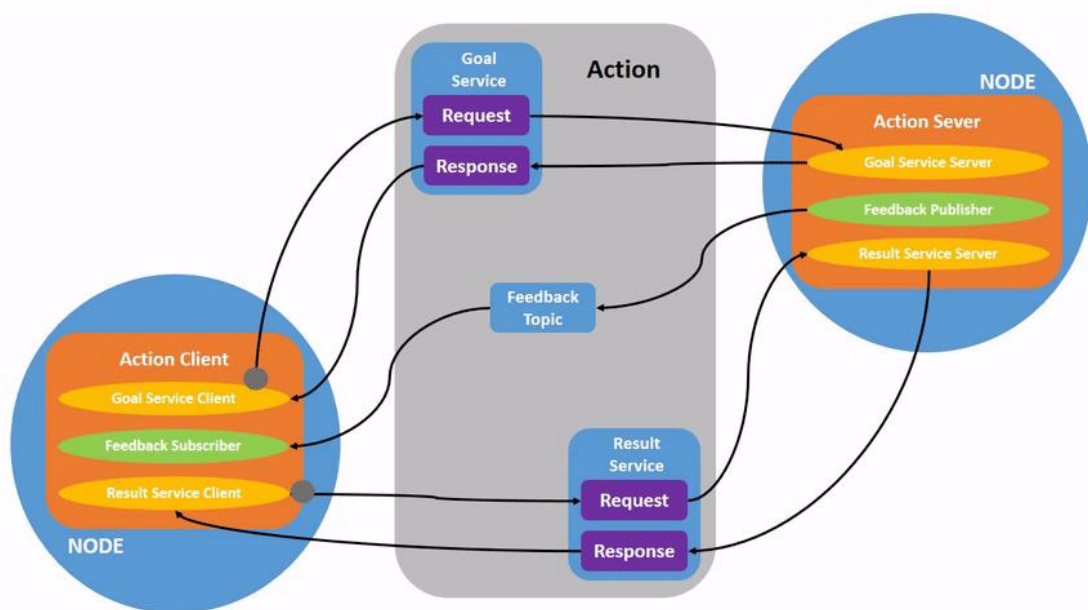
ข้อความจาก Arduino ประกอบด้วยข้อมูล encoder ในรูปแบบ `ENCODER:<left>,<right>` ข้อความสถานะ `STATUS:<สถานะ>` และเหตุการณ์จาก keypad `KEY:<char>` ส่วน Raspberry Pi ส่งคำสั่งเดินหน้า เลี้ยว หยุด กำหนดความเร็วต่อเนื่อง และควบคุมไฟเลี้ยว เมื่อเลือกใช้ ROS 2 ตัวเชื่อมต่อซอฟต์แวร์จะแลกเปลี่ยนข้อมูล Serial กับ topic เช่น `/cmd_vel`, `/odom` และ `/keypad/key` ส่วนการเชื่อมต่อแบบ Serial โดยตรงให้ซอฟต์แวร์หลักอ่านพอร์ตของ Arduino โดยตรง ทั้งสองรูปแบบเป็นทางเลือกของช่องทางสื่อสารในระดับซอฟต์แวร์และใช้ฮาร์ดแวร์ควบคุมชุดเดียวกัน

คำแนะนำและตัวค้นช่วยให้โปรแกรมแยกประเภทข้อความก่อนแปลงข้อมูล โปรแกรมจะละเว้นบรรทัดที่ไม่ตรงรูปแบบหรือแปลงเป็นตัวเลขไม่ได้ ส่วนตัวเชื่อมต่อ ROS 2 จะตรวจสอบความใหม่ของข้อมูล encoder ก่อนรายงานสถานะพร้อมรับคำสั่ง วิธีเหล่านี้ช่วยตรวจสอบข้อผิดพลาดด้านรูปแบบและการขาดข้อมูลบางกรณี แต่ยังคงใช้อัตราบิตตรงกันและจัดการข้อมูลที่อ่านมาไม่ครบตามพฤติกรรมของพอร์ตสื่อสาร

หน้าจอ POS/Kiosk ติดต่อกับเซิร์ฟเวอร์ POS ผ่าน HTTP ส่วน keypad ส่งเหตุการณ์เข้าสู่เซิร์ฟเวอร์เดียวกัน การใช้ข้อมูลร่าง (draft) ชุดเดียวกันทำให้การป้อนข้อมูลจากหน้าจอและ keypad ปรับปรุงรายการภารกิจเดียวกันได้ อย่างไรก็ตามช่องทางที่พร้อมใช้งานขึ้นกับการตั้งค่า backend และซอฟต์แวร์ที่เริ่มทำงานในแต่ละครั้ง



รูปที่ 11 แสดงโครงสร้างการส่ง Data ระดับ Bit ของการสื่อสารแบบ Serial Communication UART
(ที่มา: <https://developer.electricimp.com/resources/uart>)



รูปที่ 12 แผนภาพการทำงานของ ROS Action: Client ส่งเป้าหมายให้ Server, รับความคืบหน้าผ่าน Topic และรับผลลัพธ์เมื่อทำงานเสร็จ

(ที่มา: <https://docs.ros.org/en/dashing/Tutorials/Understanding-ROS2-Actions.html>)

2.5.2 Finite State Machine และการทำงานตามเวลา

Finite State Machine (FSM) เป็นแบบจำลองพฤติกรรมที่ประกอบด้วยสถานะและเงื่อนไขการเปลี่ยนสถานะ แบบจำลองนี้เหมาะกับงานที่มีลำดับและเงื่อนไขชัดเจน เพราะช่วยระบุว่าแต่ละเหตุการณ์เกิดขึ้นได้ในสถานะใด และเมื่อเกิดข้อผิดพลาดระบบควรเข้าสู่สถานะใด โครงการใช้ FSM ระดับภารกิจเพื่อจัดการการรับรายการ เริ่มเดินทาง รอผู้ใช้นั้นการรับอาหาร ตรวจสอบรายการที่เหลือ เดินทางกลับ และจัดการข้อผิดพลาด ส่วน firmware แยกสถานะคำสั่งเคลื่อนที่ เช่น idle, forward, turn และ velocity ออกจากสถานะของภารกิจ การแบ่งระดับนี้ช่วยให้ตรรกะการให้บริการไม่ปะปนกับการควบคุมมอเตอร์

ในเชิงแบบจำลอง FSM ระดับภารกิจอธิบายได้ด้วยชุดสถานะ S เหตุการณ์นำเข้า E ฟังก์ชันเปลี่ยนสถานะ δ และสถานะเริ่มต้น การเปลี่ยนสถานะจะเกิดขึ้นเมื่อระบบได้รับเหตุการณ์ที่เหมาะสมกับสถานะปัจจุบัน เช่น รับภารกิจใหม่เมื่อรอคำสั่ง หรือยืนยันรับอาหารเมื่อถึงช่วงรอรับอาหาร หากการนำทางไม่สำเร็จหรือข้อมูลภารกิจไม่สมบูรณ์ ระบบเปลี่ยนไปสู่สถานะข้อผิดพลาด และสั่งหยุดการเคลื่อนที่ การจัดรูปแบบเช่นนี้ทำให้เงื่อนไขปกติและเงื่อนไขผิดพลาดสามารถตรวจสอบแยกกันได้ ทั้งยังช่วยให้การทดสอบภารกิจจำลองลำดับเหตุการณ์ของผู้ใช้ได้โดยไม่ต้องพึ่งวงจรมอเตอร์ทุกกรณี

อีกทั้งระบบใช้ watchdog, timeout และสถานะ fault เพื่อจัดการเมื่อข้อมูลหรือการเคลื่อนที่ขาดหาย ตัวอย่างเช่น ระบบหยุดคำสั่งความเร็วต่อเนื่องเมื่อไม่ได้รับคำสั่งใหม่ภายในช่วงเวลาที่กำหนด และเมื่อ encoder ไม่แสดงความคืบหน้า ระบบจะเพิ่มแรงขับอย่างจำกัดหรือหยุดพร้อมรายงานสถานะผิดพลาด

2.6 ระบบจ่ายไฟและวงจรขับเคลื่อน

แหล่งจ่ายหลักของหุ่นยนต์คือแบตเตอรี่ลิเธียม 12 V 15 Ah พร้อมสวิตช์และบัลลัสสำหรับกระจายไฟวงจร L298N ขับมอเตอร์ ส่วนตัวแปลงไฟ DC-DC แบบ buck และตัวแปลง DC-DC อีกชุดที่ติดตั้งในระบบทำหน้าที่ปรับแรงดันให้เหมาะกับอุปกรณ์อิเล็กทรอนิกส์ เนื่องจากกระแสที่มอเตอร์ใช้เปลี่ยนตามภาระการออกแบบแหล่งจ่ายจึงต้องคำนึงถึงผลของโหลดมอเตอร์ต่อแรงดันของระบบ

การประเมินภาระทางไฟฟ้าเบื้องต้นพิจารณากำลังจากความสัมพันธ์ $P=VI$ และพลังงานจากผลคูณของกำลังกับเวลา สำหรับแบตเตอรี่ที่ระบุ 12 V และ 15 Ah จึงคำนวณความจุพลังงานได้เป็น 180 Wh โดยประมาณภายใต้เงื่อนไขปกติ อย่างไรก็ตาม ค่านี้ไม่ใช่พลังงานที่ใช้ได้จริงทั้งหมดและไม่เพียงพอที่จะระบุระยะเวลาทำงาน เพราะขึ้นกับประสิทธิภาพตัวแปลง กระแสของมอเตอร์และคอมพิวเตอร์ ภาระหน้าจอละเอียดเงื่อนไขการคายประจุ



รูปที่ 13 แบตเตอรี่ลิเทียม 12V 15A ที่ใช้ในโครงการนี้
(หมายเหตุ: ภาพถ่ายโดยผู้จัดทำ)

2.7 สรุป

บทนี้อธิบายสถาปัตยกรรมระบบฝังตัวและการแบ่งหน้าที่ระหว่าง Arduino กับ Raspberry Pi หลักการรับสัญญาณ encoder การควบคุมมอเตอร์แบบป้อนกลับ และแบบจำลอง Differential Drive สำหรับคำนวณ odometry นอกจากนี้ยังกล่าวถึงข้อจำกัดของการประมาณตำแหน่งแบบ dead reckoning การรับข้อมูลจาก keypad และ LiDAR การสื่อสารผ่าน Serial/ROS 2 และการจัดลำดับภารกิจด้วย FSM เนื้อหาเหล่านี้เชื่อมโยงเส้นทางข้อมูลจากผู้ใช้และเซนเซอร์ไปสู่คำสั่งควบคุมการเคลื่อนที่ หุ่นยนต์เดินทางตาม waypoint ที่กำหนดไว้ ใช้ encoder odometry เพื่อประมาณตำแหน่ง และใช้ LiDAR guard สั่งหยุดเมื่อพบวัตถุในเขตด้านหน้าตามเงื่อนไขที่ตั้งไว้ ผู้ใช้ยืนยันการรับอาหารผ่าน POS/Kiosk หรือ keypad

บทที่ 3

วิธีการดำเนินการ

3.1 การทำงานโดยภาพรวม

ระบบหุ่นยนต์ส่งอาหารต้นแบบแบ่งการทำงานเป็นสองส่วนหลัก ได้แก่ Raspberry Pi 4 ซึ่งประมวลผลภารกิจและจัดการส่วนติดต่อผู้ใช้ และ Arduino Uno R3 ซึ่งควบคุมอุปกรณ์ระดับฮาร์ดแวร์ จอสัมผัสแสดงระบบ POS สำหรับกำหนดเส้นทางและโต๊ะปลายทาง รวมถึงยืนยันการวางและการรับอาหาร ผู้ใช้สามารถใช้ Keypad 4x4 เป็นช่องทางป้อนคำสั่งแทนการสัมผัสได้เช่นกัน โดยทั้งสองช่องทางอ้างอิงรายการภารกิจชุดเดียวกัน

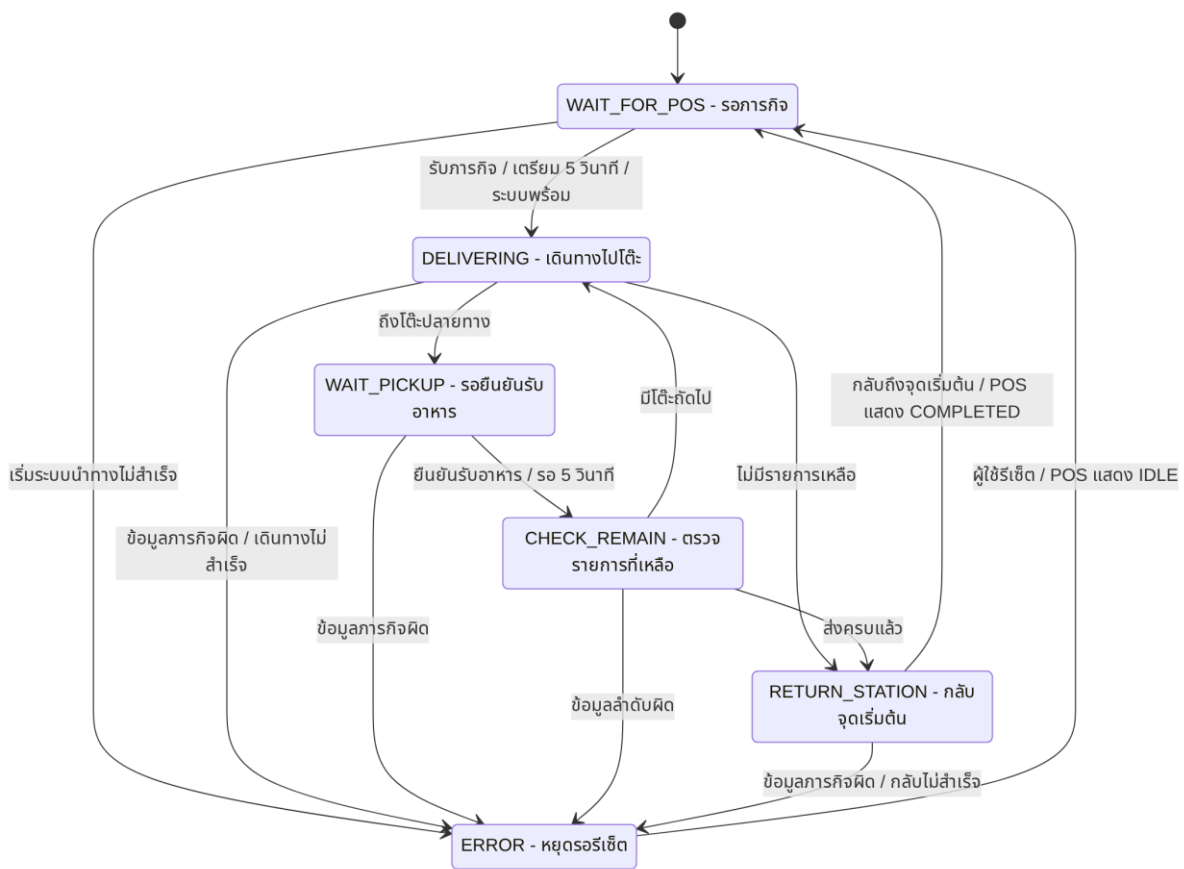
เมื่อผู้ใช้นั้นรายการ ระบบ POS ส่งภารกิจให้โปรแกรมควบคุมบน Raspberry Pi จัดลำดับการทำงานตาม Finite State Machine (FSM) และกำหนดจุดนำทาง (Waypoint) สำหรับโต๊ะที่เลือก Raspberry Pi ส่งคำสั่งความเร็วผ่าน ROS 2 และ USB Serial ไปยัง Arduino Uno เพื่อขับมอเตอร์ผ่านวงจร L298N ขณะเดียวกัน Arduino อ่านค่า Encoder ของล้อทั้งสองข้างและส่งค่ากลับเพื่อคำนวณการเคลื่อนที่ของหุ่นยนต์

LiDAR ใช้ตรวจสอบสิ่งกีดขวางระหว่างเดินทาง หากพบวัตถุอยู่ในระยะที่กำหนด ระบบจะหยุดการเคลื่อนที่ชั่วคราวและเดินทางต่อเมื่อเส้นทางปลอดภัย ส่วน SLAM ใช้สร้างและแสดงแผนที่สภาพแวดล้อมในการพัฒนาและตรวจสอบระบบ โดยการเดินทางตามภารกิจในรุ่นนี้ยังอาศัย Waypoint ที่กำหนดไว้ ปัจจุบันยังไม่ได้ให้ SLAM วางเส้นทางอัตโนมัติ เพราะข้อจำกัดด้านพื้นที่ที่ใช้ทดสอบ Arduino Uno ควบคุมจอ MAX7219 Dot Matrix ให้แสดง ลูกศรตามทิศทางการเลี้ยว เมื่อถึงโต๊ะผู้ใช้นั้นการรับอาหาร ผ่านจอ POS หรือ Keypad จากนั้นหุ่นยนต์จึงไป โต๊ะถัดไปหรือกลับจุดเริ่มต้น

3.1.1 ลำดับสถานะการส่งอาหาร

โปรแกรมควบคุมหลักเริ่มจากสถานะ WAIT_FOR_POS เพื่อรอภารกิจจาก POS เมื่อได้รับรายการระบบหยุดและแสดง PREPARING บนหน้าจอเป็นเวลา 5 วินาที จากนั้นตรวจสอบความพร้อมของระบบนำทางก่อนเข้าสู่ DELIVERING เพื่อเดินทางไปยังโต๊ะปลายทาง เมื่อถึงโต๊ะจะเข้าสู่ WAIT_PICKUP และรอให้ผู้ใช้นั้นการรับอาหารผ่านจอ POS หรือ Keypad หลังยืนยัน ระบบแสดง PICKUP_DELAY และหยุดอีก 5 วินาทีก่อนเข้าสู่ CHECK_REMAIN เพื่อตรวจรายการที่เหลือ หากมีโต๊ะถัดไปจะกลับไป DELIVERING แต่หากส่งครบแล้วจะเข้าสู่ RETURN_STATION เพื่อเดินทางกลับจุดเริ่มต้น เมื่อกลับถึงระบบแสดง COMPLETED

และกลับสู่ WAIT_FOR_POS หากระบบนำทางทำงานไม่สำเร็จ โปรแกรมจะเข้าสู่ ERROR และรอคำสั่งรีเซ็ต ดังรูปที่ 14



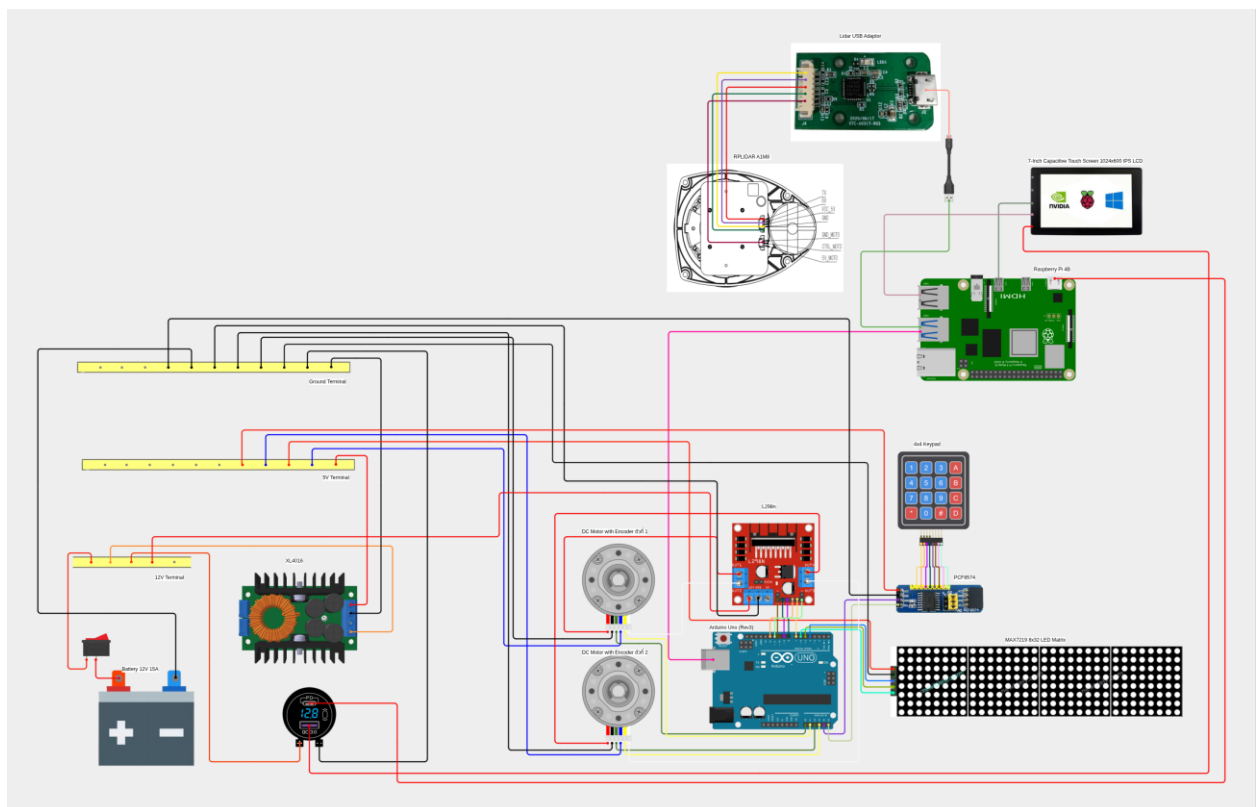
รูปที่ 14 แผนภาพ FSM หลักของระบบหุ่นยนต์ส่งอาหาร

การยกเลิกภารกิจทำได้ระหว่างการเตรียมเดินทาง การเคลื่อนที่ และการรอยืนยันรับอาหารโดยโปรแกรมจะสั่งหยุดรถ ล้างข้อมูลภารกิจ และกลับสู่ WAIT_FOR_POS ส่วนการตรวจสอบสิ่งกีดขวางด้วย LiDAR เป็นเงื่อนไขหยุดชั่วคราวระหว่างการเดินทางของ DELIVERING และ RETURN_STATION ไม่ใช่สถานะใหม่ของ FSM หลัก เมื่อทางโล่ง หุ่นยนต์จึงเดินทางต่อในเส้นทางเดิม

3.2 การทำงานของฮาร์ดแวร์

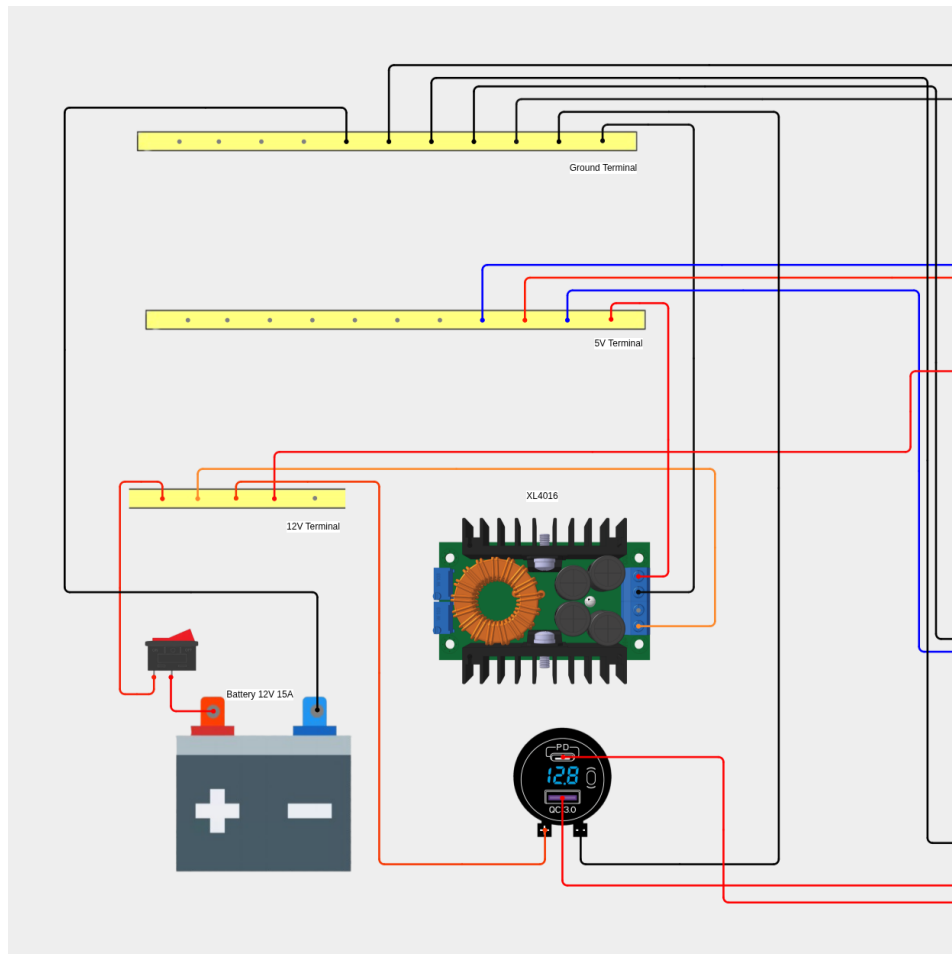
3.2.1 ผังการเชื่อมต่อและระบบจ่ายไฟ

รูปที่ 15 แสดงองค์ประกอบของระบบต้นแบบ ได้แก่ แบตเตอรี่และชุดจ่ายไฟ Raspberry Pi 4 จอสัมผัส LiDAR Arduino Uno วงจรขับมอเตอร์ มอเตอร์พร้อม Encoder Keypad ผ่าน PCF8574 และจอ Dot Matrix โดย Raspberry Pi เชื่อมต่อ Arduino และ LiDAR ผ่าน USB จอสัมผัสรับสัญญาณภาพผ่าน HDMI และข้อมูลสัมผัสผ่าน USB



รูปที่ 15 ผังการเชื่อมต่ออุปกรณ์โดยรวมของหุ่นยนต์

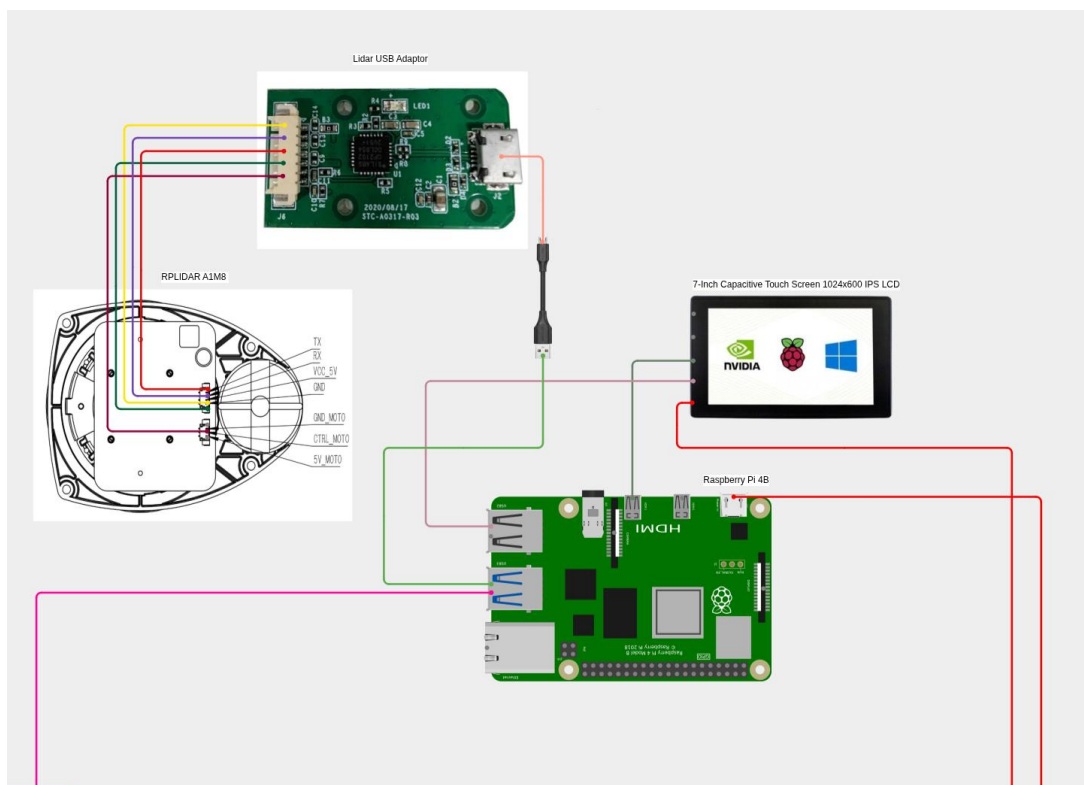
แบตเตอรี่ 12 V จ่ายพลังงานให้ส่วนขับเคลื่อน และส่งผ่านวงจรลดแรงดันสำหรับส่วนควบคุมและอุปกรณ์แสดงผล สายกราวด์ของวงจรที่สื่อสารหรือควบคุมร่วมกันต้องอ้างอิงจุดกราวด์เดียวกัน และต้องตรวจสอบแรงดันจริงที่เข้าอุปกรณ์ก่อนเปิดระบบ โดยเฉพาะ Raspberry Pi และจอ Dot Matrix ที่ต้องใช้แรงดัน 5 V ที่เหมาะสม



รูปที่ 16 ส่วนแหล่งจ่ายไฟและการกระจายแรงดัน

3.2.2 Raspberry Pi จอสัมผัส และ LiDAR

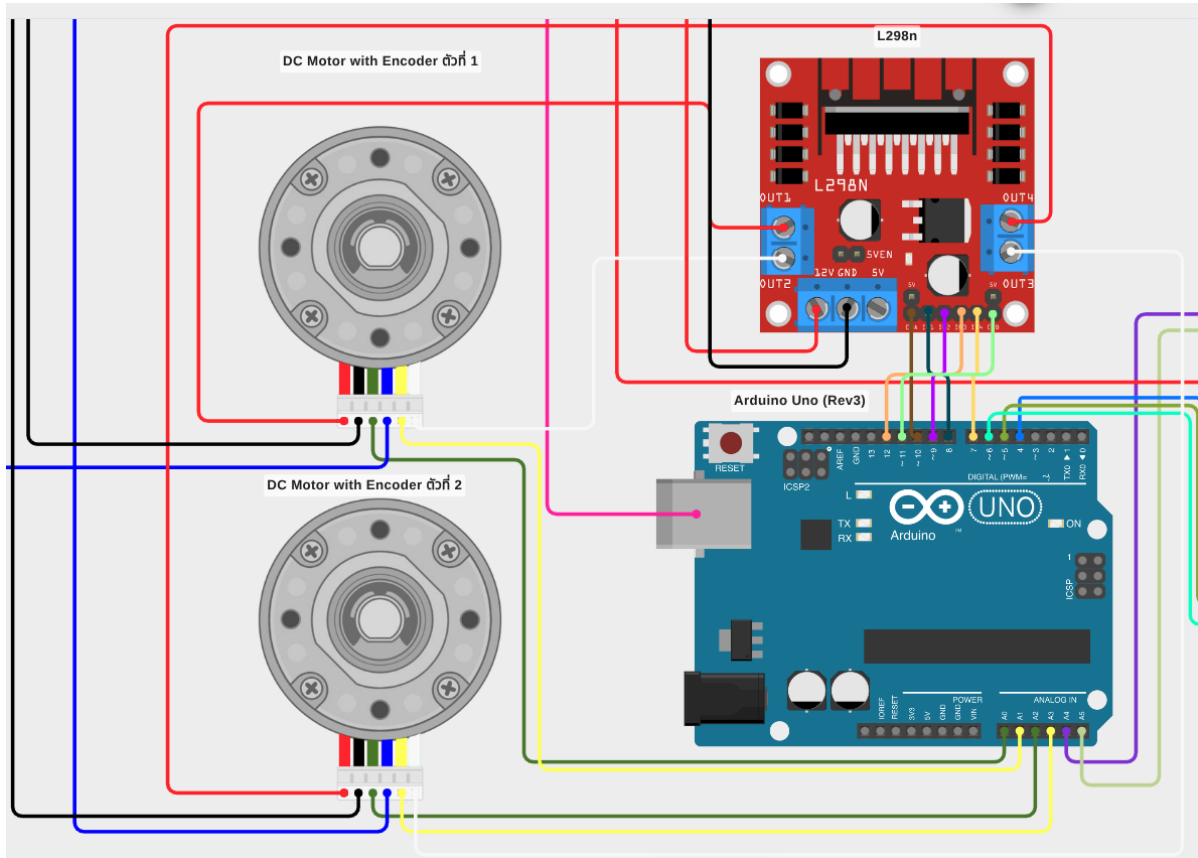
Raspberry Pi 4 ทำหน้าที่รัน POS โปรแกรมควบคุมการกิจ ROS 2 และส่วนรับข้อมูล LiDAR จอสัมผัสขนาด 7 นิ้วใช้แสดงรายการอาหาร สถานะการกิจ และปุ่มยืนยันการรับอาหาร การเชื่อมต่อจอใช้สาย Micro HDMI สำหรับภาพและ USB สำหรับสัญญาณสัมผัส ส่วน RPLIDAR A1M8 เชื่อมต่อกับ Raspberry Pi ผ่านอะแดปเตอร์ USB ดังแสดงในรูปที่ 17



รูปที่ 17 การเชื่อมต่อ Raspberry Pi จอสัมผัส และ LiDAR

3.2.3 Arduino Uno ระบบขับเคลื่อน และ Encoder

Arduino Uno รับคำสั่งจาก Raspberry Pi ทาง USB Serial ที่อัตรา 115,200 บิตต่อวินาที และส่งสัญญาณควบคุมไปยัง L298N เพื่อขับมอเตอร์สองข้างแบบ Differential Drive สัญญาณจาก Encoder ของมอเตอร์ซ้ายและขวาเข้าสู่ขานาล็อกซึ่งกำหนดให้ทำหน้าที่เป็นอินพุตดิจิทัล Arduino ใช้ข้อมูลดังกล่าวในการควบคุมความเร็วและส่งค่ากลับให้ Raspberry Pi สำหรับคำนวณระยะทางและ ทิศทางการเคลื่อนที่

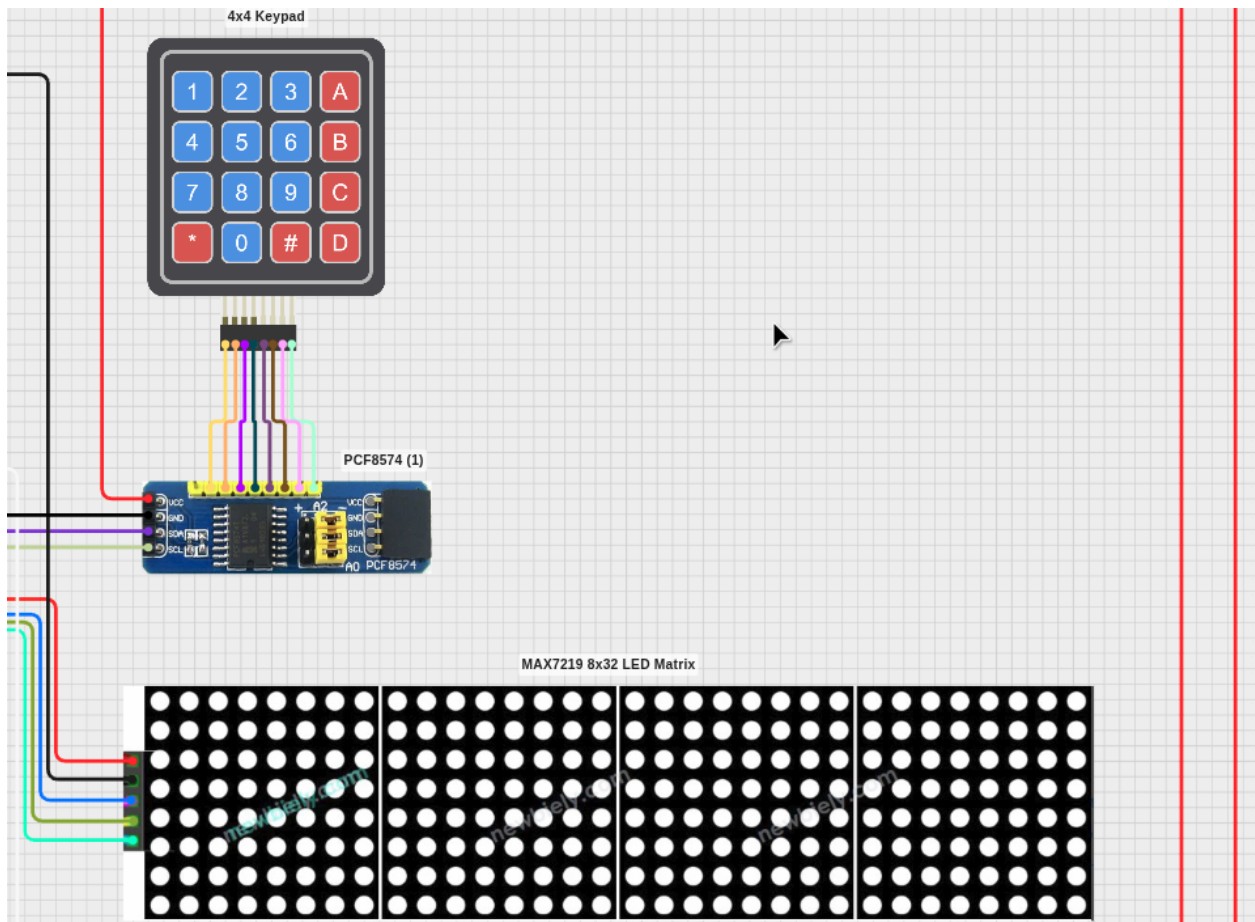


รูปที่ 18 การเชื่อมต่อ Arduino Uno วงจรขับเคลื่อนมอเตอร์ และ Encoder

3.2.4 Keypad และจอ MAX7219 Dot Matrix

Keypad 4x4 เชื่อมต่อผ่านโมดูลขยายขา PCF8574 ซึ่งสื่อสารกับ Arduino Uno ด้วย I2C โดยใช้ A4 เป็น SDA และ A5 เป็น SCL ค่าที่อยู่เริ่มต้นของโมดูลโปรแกรมคือ 0x20 การกดปุ่มถูกแปลงเป็น ข้อความ KEY:<ปุ่ม> ส่งให้ Raspberry Pi เพื่อใช้ควบคุม POS สำหรับเลือกชั้นวาง โต๊ะปลายทาง และยืนยัน รับอาหาร

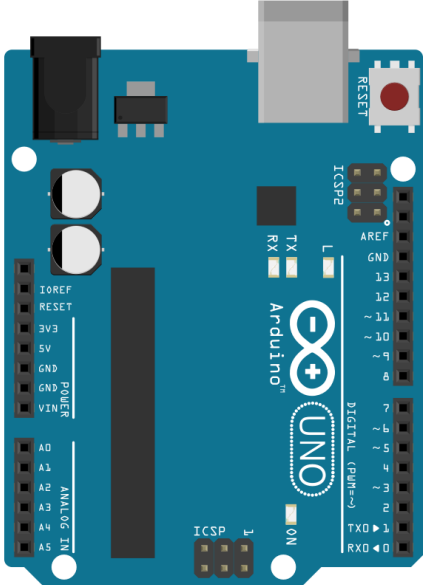
จอ MAX7219 Dot Matrix ขนาด 32x8 จุดประกอบด้วย 4 โมดูล รับสัญญาณ DIN, CS และ CLK จาก Arduino Uno เพื่อแสดงลูกศรเลี้ยวซ้ายหรือขวาตามคำสั่งของระบบ โดยจอนี้เป็นไฟแสดงทิศทาง ไม่ใช่จอแสดงรายการอาหาร



รูปที่ 19 การเชื่อมต่อ Keypad, PCF8574 และจอ MAX7219

3.2.5 การกำหนดการใช้งานของ Arduino Uno

รูปที่ 20 จัดวางภาพ Arduino Uno ไว้กลางตาราง โดยแสดงชื่อขาและหน้าที่สองด้านตาม รูปแบบผัง GPIO สีพื้นใช้แยกกลุ่มการใช้งาน ได้แก่ จอ Dot Matrix, มอเตอร์, Encoder, I2C และระบบ จ่ายไฟ รายการขาอ้างอิงโปรแกรมควบคุมรุ่นปัจจุบัน

			AREF	ไม่ได้ใช้งานในระบบ
			GND	กราวด์ร่วม
ไม่ได้ใช้งานในระบบ	IOREF		D13	ไม่ได้ใช้งานในระบบ
ไม่ได้ใช้งานในระบบ	RESET		D12	L298N IN3 มอเตอร์ขวา
ไม่ได้ใช้งานในระบบ	3V3		D11	L298N ENB มอเตอร์ขวา
ไฟเลี้ยงวงจรลอจิก	5V		D10	L298N ENA มอเตอร์ซ้าย
กราวด์ร่วม	GND		D9	L298N IN2 มอเตอร์ซ้าย
กราวด์ร่วม	GND		D8	L298N IN1 มอเตอร์ซ้าย
ไม่ได้ใช้งานในระบบ	VIN		D7	L298N IN4 มอเตอร์ขวา
Encoder ขวา ช่อง B	A0		D6	MAX7219 CLK
Encoder ขวา ช่อง A	A1		D5	MAX7219 CS/LOAD
Encoder ซ้าย ช่อง B	A2		D4	MAX7219 DIN
Encoder ซ้าย ช่อง A	A3		D3	ไม่ได้ใช้งานในระบบ
I2C SDA - PCF8574	A4		D2	ไม่ได้ใช้งานในระบบ
I2C SCL - PCF8574	A5		D1	TX ผ่าน USB Serial
			D0	RX ผ่าน USB Serial
			USB Type-B เชื่อม Raspberry Pi 4	
Raspberry Pi 4 ← USB Serial → Arduino Uno R3				

รูปที่ 20 การกำหนดขา GPIO ของ Arduino Uno กับอุปกรณ์ในระบบ

ภายใน PCF8574 กำหนด P7–P4 เป็นสายแถว และ P3–P0 เป็นสายคอลัมน์ของ Keypad ส่วนขา D0/D1 เป็นขาสื่อสาร Serial ของบอร์ด จึงไม่ควรต่ออุปกรณ์อื่นร่วมระหว่างใช้งาน USB Serial ขาไฟเลี้ยงให้ต่อเฉพาะแรงดันที่ตรวจสอบตามผังจริงแล้ว

3.2.6 อุปกรณ์หลักที่ใช้ในระบบ

อุปกรณ์แต่ละชิ้นถูกเลือกตามหน้าที่ในระบบ โดย Raspberry Pi รับงานประมวลผลและส่วนติดต่อผู้ใช้ ส่วน Arduino รับงานที่ต้องตอบสนองกับสัญญาณมอเตอร์และ Encoder โดยตรง รายการสำคัญสรุปในตารางที่ 3.1

อุปกรณ์	จำนวน	หน้าที่ในระบบ
Raspberry Pi 4 Model B	1	รัน POS, FSM, ROS 2 และโปรแกรมนำทาง
Arduino Uno R3	1	ควบคุมมอเตอร์ อ่าน Encoder และ Keypad
RPLIDAR A1M8	1	ตรวจระยะสิ่งกีดขวางและรับข้อมูลสแกน
จอสัมผัส 7 นิ้ว	1	แสดง POS และรับคำสั่งจากผู้ใช้
L298N Motor Driver	1	ขับมอเตอร์ซ้ายและขวา
มอเตอร์ DC พร้อม Encoder	2	ขับเคลื่อนและวัดการหมุนของล้อ
MAX7219 Dot Matrix 32×8	1 ชุด	แสดงลูกศรทิศทางการเลี้ยว
Keypad 4×4 และ PCF8574	1 ชุด	ป้อนข้อมูลไปยัง POS ผ่าน I2C
แบตเตอรี่ 12 V และวงจรลดแรงดัน	1 ชุด	จ่ายไฟให้ส่วนขับเคลื่อนและส่วนควบคุม

ตารางที่ 3.1 อุปกรณ์หลักและหน้าที่ของแต่ละส่วน

3.3 การพัฒนาโปรแกรมและหลักการทำงาน

Arduino Uno ซึ่งรับผิดชอบการเคลื่อนที่และอุปกรณ์รอบตัว แล้วจึงอธิบาย Raspberry Pi รับผิดชอบรายการส่งอาหารและการตัดสินใจระดับระบบ ตัวอย่างโค้ดเป็นการคัดเฉพาะช่วงที่เห็นหลักการทำงานจากไฟล์ปัจจุบัน ไม่ใช่โปรแกรมเต็มทั้งไฟล์

Source Code ฉบับเต็ม: <https://github.com/AungpaoK/EmbeddedProj>

3.3.1 โปรแกรมควบคุมบน Arduino Uno

3.3.1.1 การควบคุมมอเตอร์ผ่าน L298N

รถใช้มอเตอร์สองตัวขับเคลื่อนซ้ายและขวาแยกจากกัน Arduino จึงต้องกำหนดทั้งทิศทางและกำลังขับของมอเตอร์แต่ละข้าง ฟังก์ชัน driveMotors รับค่า PWM แบบมีเครื่องหมาย หากค่าเป็นบวกให้หมุนไปข้างหน้า หากเป็นลบให้กลับทิศ และหากเป็นศูนย์ให้หยุดมอเตอร์ ดูได้ดังโค้ดที่ 3.1 แสดงการแปลงคำสั่งดังกล่าวเป็นสัญญาณ สำหรับ L298N

```
void driveMotors(int leftPWM, int rightPWM) {
    int pwmL = constrain(abs(leftPWM), 0, 255);
    if (leftPWM > 0) { digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW); }
    else if (leftPWM < 0) { digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH); }
    else { digitalWrite(IN1, LOW); digitalWrite(IN2, LOW); }
    analogWrite(ENA, pwmL);

    int pwmR = constrain(abs(rightPWM), 0, 255);
    if (rightPWM > 0) { digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH); }
    else if (rightPWM < 0) { digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW); }
    else { digitalWrite(IN3, LOW); digitalWrite(IN4, LOW); }
    analogWrite(ENB, pwmR);
}
```

โค้ดที่ 3.1 ฟังก์ชันควบคุมทิศทางและ PWM ของมอเตอร์

คำสั่ง digitalWrite ใช้กำหนดทิศทางผ่านขา IN1–IN4 ส่วน analogWrite ใช้ส่ง PWM ไปยัง ENA และ ENB โดยจำกัดค่าอยู่ระหว่าง 0–255 การขับมอเตอร์แยกซ้ายและขวาทำให้รถเดินหน้า เลี้ยวหรือหมุนอยู่กับที่ได้

3.3.1.2 การอ่าน Encoder และส่งค่าการหมุนของล้อ

Encoder ของแต่ละล้อส่งสัญญาณสองเฟสให้ Arduino เพื่อนับการหมุนของล้อ ก่อนส่งข้อมูล โปรแกรมคัดลอกตัวนับของล้อทั้งสองขณะปิด interrupt ชั่วคราว เพื่อไม่ให้ค่าเปลี่ยนกลางการอ่าน จากนั้นส่งจำนวน tick สะสมทุก 100 มิลลิวินาที ในรูปแบบ ENCODER:<ซ้าย>,<ขวา> ดังโค้ดที่ 3.2

```
const unsigned long ENCODER_PRINT_MS = 100;

void readTicks(long &leftOut, long &rightOut) {
    noInterrupts();
    long rawL = leftEncoderTicks;
    long rawR = rightEncoderTicks;
    interrupts();
    leftOut = LEFT_ENC_INVERT ? -rawL : rawL;
    rightOut = RIGHT_ENC_INVERT ? -rawR : rawR;
}

// ส่วนส่งข้อมูลใน loop()
if (now - lastEncoderPrint >= ENCODER_PRINT_MS) {
    lastEncoderPrint = now;
    long L, R;
    readTicks(L, R);
    Serial.print("ENCODER:");
    Serial.print(L);
    Serial.print(",");
    Serial.println(R);
}
```

โค้ดที่ 3.2 การส่งค่า Encoder ผ่าน Serial

ส่วนแรกของโค้ดอ่านค่า Encoder ทั้งสองข้างและปรับเครื่องหมายตามทิศการติดตั้ง ส่วนที่อยู่ใน loop() ตรวจสอบเวลาครบ 100 มิลลิวินาทีแล้วส่งค่าออกทาง Serial Raspberry Pi จึงนำจำนวน tick ไปคำนวณ odometry ต่อได้

3.3.1.3 การอ่าน Keypad ผ่าน PCF8574

Keypad 4x4 ต่อผ่าน PCF8574 โดยกำหนดแถวและคอลัมน์ให้ตรงกับปุ่มบนแผง Arduino เริ่มบัส I²C แล้วตรวจสอบการกดปุ่มเป็นระยะ หากพบปุ่มใหม่จะส่ง KEY:<ปุ่ม> ผ่าน Serial ไปยัง Raspberry Pi ดังโค้ดที่ 3.3

```
constexpr byte ROWS = 4;
constexpr byte COLS = 4;
constexpr byte PCF8574_ADDRESS = 0x20;

// ลำดับขาตรงกับ PCF8574 adapter ที่ใช้ในโปรเจกต์
byte rowPins[ROWS] = {7, 6, 5, 4};
byte colPins[COLS] = {3, 2, 1, 0};
```

```

char keys[ROWS][COLS] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

Keypad_I2C keypad(
    makeKeymap(keys),
    rowPins,
    colPins,
    ROWS,
    COLS,
    PCF8574_ADDRESS
);

inline void begin() {
    Wire.begin();
    keypad.begin();
    keypad.setDebounceTime(40);
}

inline char readKey() {
    return keypad.getKey();
}

inline void updateAndSendToPi(Stream &serialPort) {
    static unsigned long lastScanMs = 0;
    const unsigned long nowMs = millis();
    if (nowMs - lastScanMs < 10) {
        return;
    }
    lastScanMs = nowMs;

    char key = readKey();

    if (key != NO_KEY) {
        serialPort.print(F("KEY:"));
        serialPort.println(key);
    }
}

```

โค้ดที่ 3.3 การอ่าน Keypad และส่งข้อมูล KEY

ในตัวอย่างนี้ PCF8574 ใช้ที่อยู่ 0x20 แถวของ Keypad ต่อกับ P7-P4 และคอลัมน์ต่อกับ P3-P0 ฟังก์ชัน updateAndSendToPi() ตรวจสอบทุก 10 มิลลิวินาที โดยไม่หน่วงการทำงานของวงควบคุมมอเตอร์

3.3.1.4 การแสดงลูกศรบน MAX7219

จอ Dot Matrix ประกอบด้วย MAX7219 สี่โมดูลเรียงเป็นพื้นที่ 32x8 จุด โค้ดที่ 3.4 แสดงการกำหนดจอ รูปแบบลูกศร และการเขียนจุดลงในแต่ละโมดูล ซึ่งช่วยให้เห็นว่าคำสั่งไฟเลี้ยวเปลี่ยนเป็นภาพบนจออย่างไร

```
MD_MAX72XX matrix(
    MD_MAX72XX::FC16_HW,
    LED_MATRIX_DIN_PIN,
    LED_MATRIX_CLK_PIN,
    LED_MATRIX_CS_PIN,
    MAX7219_COUNT
);

uint8_t arrowRight[8] = {
    0b00011000,
    0b00001100,
    0b00000110,
    0b11111111,
    0b11111111,
    0b00000110,
    0b00001100,
    0b00011000
};

uint8_t arrowLeft[8] = {
    0b00011000,
    0b00110000,
    0b01100000,
    0b11111111,
    0b11111111,
    0b01100000,
    0b00110000,
    0b00011000
};

void drawArrow(uint8_t module, const uint8_t picture[]) {
    uint8_t firstColumn = module * 8;

    for (uint8_t row = 0; row < 8; row++) {
        for (uint8_t column = 0; column < 8; column++) {
            bool ledOn = bitRead(picture[row], 7 - column);
            matrix.setPoint(row, firstColumn + column, ledOn);
        }
    }
}

// ส่วนเลือกภาพใน turnIndicatorUpdate()
if (currentSignal == TURN_RIGHT) {
    drawArrow(module, arrowRight);
} else {
```

```
drawArrow(MAX7219_COUNT - 1 - module, arrowLeft);
}
```

โค้ดที่ 3.4 การกำหนดรูปและแสดงลูกศรบน MAX7219

ข้อมูล arrowLeft และ arrowRight กำหนดจุดที่ต้องติดในแต่ละแถว ฟังก์ชัน drawArrow() เขียนภาพลงในโมดูลที่เลือก ส่วน turnIndicatorUpdate() เลือกทิศของลูกศรตามคำสั่งจาก Raspberry Pi และอัปเดตจอโดยไม่ใช้ delay ค้างในรอบควบคุม

3.3.1.5 การหยุดมอเตอร์เมื่อคำสั่งขาดช่วง

คำสั่งความเร็ว V: ต้องส่งซ้ำระหว่างที่รถเคลื่อนที่ หาก Raspberry Pi หยุดส่งคำสั่ง Arduino ใช้เวลาที่รับคำสั่งล่าสุดตรวจว่าการสื่อสารขาดช่วงหรือไม่ โค้ดที่ 3.5 แสดงการตรวจเวลาและการควบคุมความเร็วล้อภายในฟังก์ชันเดียวกัน

```
const unsigned long VELOCITY_TIMEOUT_MS = 300; // ตัดมอเตอร์ทันทีหาก Serial ขาดหายไป 300ms

void executeVelocity(float dt) {
    // Safety Watchdog: หากไม่ได้รับคำสั่ง V: ภายใน 300ms ให้หยุดมอเตอร์ทันที
    if (millis() - lastVelocityCmdTime > VELOCITY_TIMEOUT_MS) {
        targetLeftSpeed = 0.0f;
        targetRightSpeed = 0.0f;
        driveMotors(0, 0);
        currentCmd = CMD_IDLE;
        return;
    }

    long leftTicks, rightTicks;
    readTicks(leftTicks, rightTicks);

    float pwmL = pidLeft.compute(leftTicks, targetLeftSpeed, dt);
    float pwmR = pidRight.compute(rightTicks, targetRightSpeed, dt);

    if (encoderStallDetected()) {
        stopForEncoderStall();
        return;
    }

    driveMotors((int)pwmL, (int)pwmR);
}
```

โค้ดที่ 3.5 เงื่อนไขหยุดมอเตอร์เมื่อขาดคำสั่ง

หากเกิน 300 มิลลิวินาทีโดยไม่มีคำสั่งใหม่ ฟังก์ชันจะตั้งความเร็วเป้าหมายเป็นศูนย์ หยุดมอเตอร์ และออกจากโหมดควบคุมความเร็วทันที เมื่อยังได้รับคำสั่งตามปกติ โปรแกรมจะอ่าน Encoder คำนวณ PWM จาก PID และตรวจการติดขัดก่อนส่งค่าไปยัง L298N

3.3.2 โปรแกรมควบคุมบน Raspberry Pi

3.3.2.1 การรับข้อมูล Serial จาก Arduino

Raspberry Pi อ่านข้อมูลจาก Arduino ผ่าน USB Serial ที่ละบรรทัด โค้ดที่ 3.6 แสดงการแยกข้อความ ENCODER: และ KEY: ในรอบรับข้อมูลเดียวกัน ค่าตัวนับล้อใช้คำนวณ odometry ส่วนปุ่มที่ถูกต้องจะส่งต่อไปยัง POS ผ่าน ROS 2

```
# ส่วนรับข้อมูลใน _serial_read_loop()
raw = self._ser.readline()
if not raw:
    continue

last_rx_time = time.monotonic()
line = raw.decode("utf-8", errors="replace").strip()
if not line:
    continue

if line.startswith("ENCODER:"):
    parts = line[8:].split(",")
    if len(parts) == 2:
        try:
            l_ticks = int(parts[0])
            r_ticks = int(parts[1])
        except ValueError:
            logger.warning("Malformed encoder line from Arduino: %r", line)
            continue

        if not encoder_seen:
            logger.info("Received encoder stream from Arduino: %s", line)
            encoder_seen = True
            self._last_encoder_time = time.monotonic()
            self._update_odometry(l_ticks, r_ticks)
        else:
            logger.warning("Malformed encoder line from Arduino: %r", line)
    elif line.startswith("KEY:"):
        key = line[4:].strip().upper()
        if len(key) == 1 and key in "0123456789ABCD*#":
            message = String()
            message.data = key
            self._keypad_pub.publish(message)
            logger.info("Arduino keypad: %s", key)
        else:
            logger.warning("Malformed keypad line from Arduino: %r", line)
```

โค้ดที่ 3.6 การแยกข้อมูล Encoder และ Keypad จาก Serial

โปรแกรมตรวจจำนวนค่าและชนิดข้อมูลก่อนใช้ ENCODER: และตรวจว่าปุ่มอยู่ในชุด 0-9, A-D, * หรือ # ก่อนเผยแพร่เป็นเหตุการณ์ Keypad การแยกด้วยค่านำหน้าช่วยให้ข้อมูลทั้งสองชนิดใช้สาย USB เส้นเดียวกันได้

3.3.2.2 ระบบ POS และการรับรายการส่งอาหาร

จอสัมผัสเป็นช่องทางหลักในการเลือกชั้นวางและโต๊ะปลายทาง ส่วน Keypad เป็นช่องทางสำรองที่แก็รารายการร่างชุดเดียวกันบน POS เมื่อเลือกโต๊ะ โปรแกรมปัจจุบันบันทึกสถานะยืนยันอาหารของชั้นนั้นไว้ด้วย ผู้ใช้ยังสามารถเปลี่ยนสถานะดังกล่าวก่อนเริ่มงานได้ เมื่อรายการผ่านการตรวจสอบ POS จึงส่งภารกิจให้ FSM รับไปทำงาน ส่วนการรับอาหารที่โต๊ะต้องให้ผู้ใช้กดยืนยันผ่านจอหรือ Keypad ไม่ได้ใช้เซนเซอร์ตรวจอาหารจริง

ปุ่ม	หน้าที่ขณะรอรับรายการ	หน้าที่เมื่อถึงโต๊ะ
A / B	เลือกชั้น 2 / ชั้น 1	ไม่เปลี่ยนภารกิจ
1 / 2	เลือกโต๊ะ 1 / โต๊ะ 2 ให้ชั้นที่เลือก	ไม่เปลี่ยนภารกิจ
C	เปลี่ยนสถานะยืนยันว่ามีอาหารบนชั้น	ไม่เปลี่ยนภารกิจ
D / *	ล้างรายการชั้นที่เลือก / ล้างทั้งหมด	ไม่เปลี่ยนภารกิจ
#	เริ่มภารกิจเมื่อรายการพร้อม	ยืนยันว่ารับอาหารแล้ว

ตารางที่ 3.2 ความหมายของปุ่ม Keypad ตามสถานะ POS

เมื่อระบบอยู่ในสถานะ ERROR ปุ่ม * ใช้ขอรีเซ็ตภารกิจ แต่ในสถานะเดินทางปุ่มอื่นจะไม่เปลี่ยนรายการส่งอาหาร วิธีนี้ลดโอกาสเกิดพิษณะรถกำลังเคลื่อนที่ ผู้ใช้ยังมองเห็นสถานะที่หน้าจอ POS แม้เลือกป้อนข้อมูลผ่าน Keypad

เมื่อผู้ใช้เลือกโต๊ะหรือกดเริ่มภารกิจ หน้าจอ POS จะส่งคำสั่งไปยังระบบควบคุมผ่าน API ภายใน Raspberry Pi โค้ดที่ 3.7 แสดงส่วนที่แปลงหมายเลขชั้นบนจอเป็นหมายเลขชั้นจริง ส่งโต๊ะที่เลือกและส่งคำสั่งเริ่มงาน

```
function physicalShelfForUi(shelf) {
    return 3 - shelf;
}

async function assignTable(shelf, tableId) {
    try {
```

```

const physicalShelf = physicalShelfForUi(shelf);
await requestJson("/api/pos/action", {
  method: "POST",
  body: JSON.stringify({ action: "set_table", shelf: physicalShelf, table_id:
tableId }),
});
const snapshot = await requestJson("/api/state");
const selection = snapshot.draft && (snapshot.draft[String(physicalShelf)] ||
snapshot.draft[physicalShelf]);
// Older running controllers still require the placement flag; newer ones set it with
the table.
if (selection && selection.table_id === tableId && selection.loaded_confirmed !== true)
{
  await requestJson("/api/pos/action", {
    method: "POST",
    body: JSON.stringify({ action: "toggle_loaded", shelf: physicalShelf }),
  });
}
await refreshState();
} catch (error) {
  showSetupMessage(error.message, "error");
}
}

async function startMission() {
  const selected = [1, 2].filter((shelf) => draft[shelf].table_id !== null);
  if (selected.length === 0) {
    showSetupMessage("เลือกโต๊ะอย่างน้อยหนึ่งโต๊ะก่อนเริ่ม", "error");
    return;
  }

  startPending = true;
  renderDraft();
  try {
    await requestJson("/api/pos/action", {
      method: "POST",
      body: JSON.stringify({ action: "start" }),
    });
    setSidebarOpen(false);
    showSetupMessage("รับรายการแล้ว กำลังเริ่มภารกิจ");
    await refreshState();
  } catch (error) {
    showSetupMessage(error.message, "error");
  } finally {
    startPending = false;
    renderDraft();
  }
}

```

โค้ดที่ 3.7 การเลือกโต๊ะและเริ่มภารกิจจากหน้าจอ POS

หลังส่งคำสั่ง หน้าจออ่านสถานะล่าสุดกลับมาแสดง จึงเห็นรายการเดียวกับที่ Keypad แก้ไขไว้
หากยังไม่ได้เลือกโต๊ะ ระบบจะไม่ส่งคำสั่งเริ่มภารกิจ

ก่อนรับภารกิจ โปรแกรมตรวจว่ามีรายการหนึ่งหรือสองรายการ แต่ละรายการระบุชั้นและโต๊ะที่ระบับรองรับ และมีสถานะยืนยันอาหารแล้ว จากนั้นจึงสร้างรหัสภารกิจและนำรายการเข้าคิวให้ FSM รับไปทำงาน ดังโค้ดที่ 3.8

```
def _validate_orders(raw_orders: object) -> list[dict]:
    if not isinstance(raw_orders, list) or not 1 <= len(raw_orders) <= 2:
        raise PosRequestError(400, "ต้องระบุรายการส่งอาหาร 1 ถึง 2 รายการ")

    orders: list[dict] = []
    seen_shelves: set[int] = set()
    for item in raw_orders:
        if not isinstance(item, dict):
            raise PosRequestError(400, "รูปแบบรายการส่งอาหารไม่ถูกต้อง")

        shelf = item.get("shelf")
        table_id = item.get("table_id")
        loaded_confirmed = item.get("loaded_confirmed")
        if type(shelf) is not int or shelf not in (1, 2):
            raise PosRequestError(400, "ชั้นต้องเป็น 1 หรือ 2")
        if type(table_id) is not int or table_id not in (1, 2):
            raise PosRequestError(400, "โต๊ะต้องเป็น 1 หรือ 2")
        if loaded_confirmed is not True:
            raise PosRequestError(400, f"กรุณายืนยันว่าของวางบนชั้น {shelf} แล้ว")
        if shelf in seen_shelves:
            raise PosRequestError(400, f"เลือกชั้น {shelf} ซ้ำไม่ได้")

        seen_shelves.add(shelf)
        orders.append(
            {
                "shelf": shelf,
                "table_id": table_id,
                "loaded_confirmed": True,
            }
        )

    return sorted(orders, key=lambda order: order["shelf"])

def submit_mission(self, raw_orders: object) -> dict:
    orders = self._validate_orders(raw_orders)
    with self._lock:
        if self._snapshot["state"] not in {"IDLE", "COMPLETED"}:
            raise PosRequestError(409, "หุ่นยนต์กำลังทำงานหรือรอการกู้คืน")

        mission_id = uuid.uuid4().hex
        mission = {"mission_id": mission_id, "orders": orders}
        try:
            self.missions.put_nowait(mission)
        except queue.Full as exc:
            raise PosRequestError(409, "มีงานรอเริ่มอยู่แล้ว") from exc

    self._pickup_pending = None
    self.cancel_event.clear()
```

```

self._keypad_sidebar_open = False
self._keypad_sidebar_pending = False
self._snapshot = {
    "state": "PREPARING",
    "mission_id": mission_id,
    "orders": [dict(order) for order in orders],
    "current_order_index": None,
    "message": "รับรายการแล้ว กำลังเตรียมหุ่นยนต์",
    "error": None,
    "obstacle_detected": False,
}
self._clear_draft_locked()
self._setup_message = "รับรายการแล้ว กำลังเริ่มภารกิจ"
logger.info("Accepted POS mission %s with %d order(s)", mission_id, len(orders))
return {"mission_id": mission_id, "state": "PREPARING"}

```

โค้ดที่ 3.8 การตรวจรายการและส่งภารกิจจาก POS ให้ FSM

การตรวจขั้นที่ซ้ำกันป้องกันการส่งรายการซ้ำจากชั้นเดียว ส่วนคิวที่รับภารกิจได้ครั้งละหนึ่งงาน ป้องกันการเริ่มงานใหม่ซ้อนระหว่างที่หุ่นยนต์กำลังทำงาน เมื่อรถถึงโต๊ะ POS จะแสดงปุ่มยืนยันรับอาหาร

โค้ดที่ 3.9 ตรวจว่าคำยืนยันตรงกับรหัสภารกิจและลำดับโต๊ะที่กำลังรออยู่ ก่อนส่งเหตุการณ์ให้ FSM ดำเนินงานต่อ

```

def confirm_pickup(self, mission_id: object, order_index: object) -> dict:
    with self._lock:
        snapshot = self._snapshot
        if snapshot["state"] != "WAITING_PICKUP":
            raise PosRequestError(409, "หุ่นยนต์ไม่ได้รับการยืนยันรับอาหาร")
        if mission_id != snapshot["mission_id"]:
            raise PosRequestError(409, "รายการยืนยันนี้เป็นของภารกิจเก่า")
        if type(order_index) is not int or order_index != snapshot["current_order_index"]:
            raise PosRequestError(409, "รายการยืนยันนี้ไม่ตรงกับโต๊ะปัจจุบัน")

        confirmation = (mission_id, order_index)
        if self._pickup_pending == confirmation:
            raise PosRequestError(409, "รายการนี้ได้รับการยืนยันแล้ว")

        self._pickup_pending = confirmation
        self.pickup_confirmations.put_nowait(confirmation)
        logger.info(
            "Pickup confirmed for mission %s order %d",
            mission_id,
            order_index,
        )
    return {"accepted": True}

```

โค้ดที่ 3.9 การตรวจคำยืนยันรับอาหารจาก POS

หากคำยืนยันเป็นของภารกิจเก่า โต้ะผิดลำดับ หรือรายการเดียวกันได้รับการยืนยันแล้ว ระบบจะไม่รับคำสั่งนั้น เพื่อลดการข้ามขั้นตอนระหว่างส่งอาหาร

3.3.2.3 การจัดลำดับงานด้วย FSM

FSM เลือกฟังก์ชันที่ทำงานตามสถานะปัจจุบันและตรวจคำขอยกเลิกก่อนทุกครั้ง โค้ดที่ 3.10 แสดงทั้งจุดเลือกสถานะและตัวอย่างการเปลี่ยนสถานะหลังส่งอาหารถึงโต้ะ เพื่อให้เห็นว่าระบบตัดสินใจไปโต้ะถัดไปหรือกลับจุดเริ่มต้นอย่างไร

```
def _step(self) -> None:
    if self._pos.cancel_event.is_set():
        self._finish_cancelled()
        return
    match self._state:
        case State.WAIT_FOR_POS:
            self._state_wait_for_pos()
        case State.DELIVERING:
            self._state_delivering()
        case State.WAIT_PICKUP:
            self._state_wait_pickup()
        case State.CHECK_REMAIN:
            self._state_check_remain()
        case State.RETURN_STATION:
            self._state_return_station()
        case State.ERROR:
            self._state_error()

def _state_check_remain(self) -> None:
    if self._current_order_index is None:
        self._latch_error("ไม่พบลำดับรายการส่ง")
        return

    completed_table = self._orders[self._current_order_index].table_id
    next_index = self._current_order_index + 1
    while (
        next_index < len(self._orders)
        and self._orders[next_index].table_id == completed_table
    ):
        next_index += 1

    if next_index < len(self._orders):
        self._current_order_index = next_index
        next_order = self._orders[next_index]
        self._shelf lcd_print(0, "Next delivery...")
        self._shelf lcd_print(1, f"Table {next_order.table_id}")
        self._state = State.DELIVERING
        return

    self._state = State.RETURN_STATION
```

โค้ดที่ 3.10 การเลือกสถานะและตรวจรายการส่งอาหารที่เหลือ

เมื่อผู้ใช้นัยการรับอาหารและครบเวลารอก่อนเคลื่อนที่ FSM จะเข้าสู่ CHECK_REMAIN ฟังก์ชันนี้ข้ามรายการที่ปลายทางเป็นโต๊ะเดิม หากยังมีโต๊ะอื่นจะกลับไปสถานะ DELIVERING แต่ถ้าส่งครบทุกรายการจะเปลี่ยนเป็น RETURN_STATION

3.3.2.4 การนำทางตาม Waypoint

การนำทางใช้จุดเริ่มต้น จุดแยก และโต๊ะทั้งสองเป็นตำแหน่งที่กำหนดไว้ล่วงหน้า ก่อนเคลื่อนที่โปรแกรมตรวจสอบความพร้อมของข้อมูลตำแหน่งและบันทึกมุมหันเริ่มต้น จากนั้นเลือกทิศทางไปโต๊ะที่กำหนดโดยใช้ระยะทางและมุมหันจาก odometry ตรวจสอบการเคลื่อนที่ ฟังก์ชันหลักที่ใช้ในขั้นตอนนี้แสดงในโค้ดที่ 3.11–3.15

```
def begin_mission(self) -> bool:
    """Wait for healthy motion feedback and capture the mission heading."""
    self.last_error = None
    deadline = self._clock() + self.preflight_timeout_s
    while self._clock() < deadline:
        if self._cancel_event.is_set():
            self._motion.stop_continuous()
            return False
        if self._is_ready() and self._is_pose_fresh():
            _x, _y, heading = self._get_pose()
            self._start_heading = normalize_angle(heading)
            self._location = "home"
            self._active = True
            logger.info(
                "[Route] Mission frame captured at heading %.1f degrees.",
                math.degrees(self._start_heading),
            )
            return True
        self._motion.stop_continuous()
        self._sleep(0.1)

    return self._fail("ไม่พบ /arduino/ready และ /odom ที่พร้อมใช้งานภายใน 12 วินาที")
```

โค้ดที่ 3.11 ฟังก์ชัน begin_mission() ตรวจสอบความพร้อมและบันทึกมุมหันเริ่มต้น

ฟังก์ชัน begin_mission() รอจน Arduino และข้อมูล odometry พร้อมใช้งาน แล้วบันทึกมุมหัน ณ จุดเริ่มต้นเป็นทิศทางอ้างอิงของภารกิจ หากยังไม่พร้อมภายในเวลาที่กำหนด โปรแกรมจะไม่สั่งให้หุ่นยนต์ออกเดินทาง

เมื่อเริ่มภารกิจแล้ว ฟังก์ชัน go_to_table() เลือกทิศทางและระยะของโต๊ะปลายทาง หากรถอยู่ที่จุดเริ่มต้นจะเดินทางถึงจุดแยกก่อนเลี้ยวและไปยังโต๊ะ แต่ถ้ากำลังอยู่ที่โต๊ะอีกฝั่ง จะคำนวณระยะเดินทางข้ามไปยังโต๊ะใหม่

```

def go_to_table(self, table_id: int) -> bool:
    """Move from the current logical route location to a selected table."""
    if table_id not in (1, 2):
        return self._fail(f"ไม่รู้จักโต๊ะ {table_id}")
    if self._start_heading is None:
        return self._fail("ยังไม่ได้เริ่ม mission frame")

    destination = f"table_{table_id}"
    if self._location == destination:
        logger.info("[Route] Already at table %d; no movement required.", table_id)
        return True

    side = 1.0 if table_id == 1 else -1.0
    target_heading = normalize_angle(self._start_heading + side * math.pi / 2.0)
    target_distance = TABLE1_Y if table_id == 1 else TABLE2_Y

    if self._location == "home":
        if not self.drive_forward(JUNCTION_X, self._start_heading):
            return False
        if not self.turn_to_heading(target_heading):
            return False
        if not self.drive_forward(target_distance, target_heading):
            return False
    elif self._location in {"table_1", "table_2"}:
        previous_table = 1 if self._location == "table_1" else 2
        previous_distance = TABLE1_Y if previous_table == 1 else TABLE2_Y
        if not self.turn_to_heading(target_heading):
            return False
        if not self.drive_forward(previous_distance + target_distance, target_heading):
            return False
    else:
        return self._fail(f"ตำแหน่งเส้นทางไม่ถูกต้อง: {self._location}")

    self._location = destination
    return True

```

โค้ดที่ 3.12 ฟังก์ชัน go_to_table() กำหนดเส้นทางไปยังโต๊ะ

แต่ละช่วงของเส้นทางคืนค่า True เมื่อทำสำเร็จและ False เมื่อไม่สำเร็จ

ฟังก์ชันจะอัปเดตตำแหน่งเชิงตรรกะของรถหลังเดินทางถึงโต๊ะเท่านั้น

ฟังก์ชัน drive_forward() ส่งความเร็วเดินทางเป็นช่วงสั้น ๆ พร้อมอ่าน odometry เพื่อติดตามระยะที่วิ่งจริงและแก้ความคลาดเคลื่อนของมุมหัน

```

def drive_forward(
    self,
    distance: float,
    target_heading: float,
    *,
    timeout_s: Optional[float] = None,
) -> bool:
    """Drive a measured distance while correcting yaw drift."""

```

```

if distance <= ARRIVAL_TOLERANCE_M:
    return True
if not self._feedback_healthy():
    return self._fail("Arduino หรือ odometry ไม่พร้อมก่อนเริ่มเดินทาง")

target_heading = normalize_angle(target_heading)
last_x, last_y, _heading = self._get_pose()
traveled = 0.0
timeout_s = timeout_s or (distance / max(self.linear_speed, 0.05)) * 2.5 + 5.0
deadline = self._clock() + timeout_s
last_progress_time = self._clock()
last_progress_distance = 0.0

logger.info(
    "[Route] Drive %.2fm at heading %.1f degrees.",
    distance,
    math.degrees(target_heading),
)

while self._active and not self._cancel_event.is_set() and traveled < distance:
    if not self._feedback_healthy():
        return self._fail("Arduino หรือ odometry ขาดการตอบสนองระหว่างเดินทาง")
    if self._clock() > deadline:
        return self._fail(f"เดินทางไม่ครบระยะ {traveled:.2f}/{distance:.2f} เมตร")

    self._report_obstacle_state()
    if self._safety.is_obstacle_detected:
        paused_at = self._clock()
        self._motion.stop_continuous()
        self._sleep(self.control_period)
        paused_for = self._clock() - paused_at
        deadline += paused_for
        last_progress_time += paused_for
        continue

    x, y, heading = self._get_pose()
    heading_error = normalize_angle(target_heading - heading)
    correction = 0.0
    if abs(heading_error) > math.radians(1.0):
        correction = max(-0.5, min(0.5, 1.8 * heading_error))
    if abs(correction) < 0.12:
        correction = math.copysign(0.12, correction)

    if not self._motion.drive_continuous(self.linear_speed, correction):
        return self._fail("ส่งคำสั่งความเร็วเดินทางไม่สำเร็จ")

    self._sleep(self.control_period)
    x, y, _heading = self._get_pose()
    traveled += math.hypot(x - last_x, y - last_y)
    last_x, last_y = x, y

    now = self._clock()
    if traveled - last_progress_distance >= 0.01:
        last_progress_distance = traveled
        last_progress_time = now

```

```

elif now - last_progress_time > 4.0:
    return self._fail("ไม่พบความคืบหน้าจาก odometry ระหว่างเดินทางเกิน 4 วินาที")

self._motion.stop_continuous()
if self._cancel_event.is_set():
    return False
if traveled >= max(0.0, distance - ARRIVAL_TOLERANCE_M):
    logger.info("[Route] Straight leg completed: %.2fm.", traveled)
    return True
return self._fail(f"เดินทางไม่ครบระยะ {traveled:.2f}/{distance:.2f} เมตร")

```

โค้ดที่ 3.13 ฟังก์ชัน drive_forward() ควบคุมการเดินทาง

หาก LiDAR พบสิ่งกีดขวาง ฟังก์ชันจะหยุดมอเตอร์และรออยู่ในช่วงทางเดิม เมื่อทางโล่งจึงเดินทางต่อ

โดยไม่นับเวลาที่หยุดรอเป็นเวลาที่ใช้เดินทาง ถ้าข้อมูล odometry

ไม่คืบหน้าหรืออุปกรณ์ขาดการตอบสนองจะคืนค่าความล้มเหลว

ฟังก์ชัน turn_to_heading() เปรียบเทียบมุมหันปัจจุบันกับมุมเป้าหมาย ถ้าคลาดเคลื่อนเกินค่าที่ยอมรับจึงกำหนดทิศทางการเลี้ยว แล้วเรียก _turn_to_heading_active() เพื่อควบคุมการหมุน

```

def turn_to_heading(
    self,
    target_heading: float,
    *,
    tolerance_degrees: float = 2.5,
    timeout_s: Optional[float] = None,
) -> bool:
    target_heading = normalize_angle(target_heading)
    _x, _y, heading = self._get_pose()
    initial_error = normalize_angle(target_heading - heading)
    tolerance = math.radians(tolerance_degrees)
    if abs(initial_error) <= tolerance:
        self._motion.set_turn_intent(TURN_OFF)
        self._motion.stop_continuous()
        return True

    if not self._feedback_healthy():
        return self._fail("Arduino หรือ odometry ไม่พร้อมก่อนเริ่มหมุน")

    direction = TURN_LEFT if initial_error > 0.0 else TURN_RIGHT
    try:
        self._motion.set_turn_intent(direction)
        return self._turn_to_heading_active(
            target_heading,
            tolerance_degrees=tolerance_degrees,
            timeout_s=timeout_s,
        )
    finally:
        self._motion.set_turn_intent(TURN_OFF)

def _turn_to_heading_active(

```

```

self,
target_heading: float,
*,
tolerance_degrees: float,
timeout_s: Optional[float],
) -> bool:
    """Turn in place to an absolute mission-relative odometry heading."""
    if not self._feedback_healthy():
        return self._fail("Arduino หรือ odometry ไม่พร้อมก่อนเริ่มหมุน")

    target_heading = normalize_angle(target_heading)
    _x, _y, heading = self._get_pose()
    initial_error = normalize_angle(target_heading - heading)
    tolerance = math.radians(tolerance_degrees)
    if abs(initial_error) <= tolerance:
        self._motion.stop_continuous()
        return True

    timeout_s = timeout_s or (abs(initial_error) / max(self.max_angular_speed, 0.1)) * 3.0 +
6.0
    deadline = self._clock() + timeout_s
    last_progress_time = self._clock()
    best_remaining = abs(initial_error)
    slowdown_range = math.radians(45.0)
    minimum_turn_speed = min(self.max_angular_speed, 0.60)

    logger.info(
        "[Route] Turn to heading %.1f degrees (error %.1f degrees).",
        math.degrees(target_heading),
        math.degrees(initial_error),
    )

    while self._active and not self._cancel_event.is_set():
        if not self._feedback_healthy():
            return self._fail("Arduino หรือ odometry ขาดการตอบสนองระหว่างหมุน")
        if self._clock() > deadline:
            return self._fail("หมุนไม่ถึง heading เป้าหมายภายในเวลาที่กำหนด")

        _x, _y, heading = self._get_pose()
        error = normalize_angle(target_heading - heading)
        remaining = abs(error)

        if remaining <= tolerance:
            self._motion.stop_continuous()
            self._sleep(0.15)
            if not self._feedback_healthy():
                return self._fail("odometry ขาดการตอบสนองขณะตรวจมุมหลังหยุด")
            _x, _y, settled_heading = self._get_pose()
            settled_error = normalize_angle(target_heading - settled_heading)
            if abs(settled_error) <= tolerance:
                logger.info(
                    "[Route] Turn completed at %.1f degrees.",
                    math.degrees(settled_heading),
                )
            return True

```

```

        best_remaining = abs(settled_error)
        last_progress_time = self._clock()
        continue

    now = self._clock()
    if remaining < best_remaining - math.radians(1.5):
        best_remaining = remaining
        last_progress_time = now
    elif now - last_progress_time > 4.0:
        return self._fail("ไม่พบการเปลี่ยนมุมจาก odometry เกิน 4 วินาที")

    if remaining >= slowdown_range:
        turn_speed = self.max_angular_speed
    else:
        progress = max(
            0.0,
            min(1.0, (remaining - tolerance) / (slowdown_range - tolerance)),
        )
        turn_speed = minimum_turn_speed + (
            self.max_angular_speed - minimum_turn_speed
        ) * progress

    angular_speed = math.copysign(turn_speed, error)
    if not self._motion.drive_continuous(0.0, angular_speed):
        return self._fail("ส่งคำสั่งความเร็วหมุนไม่สำเร็จ")
    self._sleep(self.control_period)

self._motion.stop_continuous()
if self._cancel_event.is_set():
    return False
return self._fail("ภารกิจนำทางถูกยกเลิก")

```

โค้ดที่ 3.14 ฟังก์ชัน turn_to_heading() และ _turn_to_heading_active() ควบคุมการหมุน

ระหว่างหมุน ฟังก์ชัน _turn_to_heading_active() อ่านมุมจาก odometry ขึ้น

ลดความเร็วเมื่อเข้าใกล้มุมเป้าหมาย และตรวจมุมอีกครั้งหลังหยุด ส่วน turn_to_heading()

ปิดสัญญาณไฟเขียวเมื่อจบการทำงาน ไม่ว่าจะสำเร็จหรือเกิดข้อผิดพลาด

เมื่อส่งอาหารครบ ฟังก์ชัน return_home() วางลำดับการหมุนและเดินหน้า จากโต๊ะผ่านจุดแยก
กลับไปยังจุดเริ่มต้น

```

def return_home(self) -> bool:
    """Return through the junction and finish facing out toward the aisle."""
    if self._start_heading is None:
        return self._fail("ยังไม่ได้เริ่ม mission frame")

    if self._location == "home":
        return self.turn_to_heading(self._start_heading)
    if self._location not in {"table_1", "table_2"}:
        return self._fail(f"ตำแหน่งเส้นทางไม่ถูกต้อง: {self._location}")

```

```

table_id = 1 if self._location == "table_1" else 2
side = 1.0 if table_id == 1 else -1.0
table_distance = TABLE1_Y if table_id == 1 else TABLE2_Y
heading_to_junction = normalize_angle(self._start_heading - side * math.pi / 2.0)
heading_to_kitchen = normalize_angle(self._start_heading + math.pi)

if not self.turn_to_heading(heading_to_junction):
    return False
if not self.drive_forward(table_distance, heading_to_junction):
    return False
if not self.turn_to_heading(heading_to_kitchen):
    return False
if not self.drive_forward(JUNCTION_X, heading_to_kitchen):
    return False
if not self.turn_to_heading(self._start_heading):
    return False

self._location = "home"
self._active = False
return True

```

โค้ดที่ 3.15 ฟังก์ชัน return_home() เดินทางกลับจุดเริ่มต้น

เมื่อรถกลับถึงและหันในทิศเริ่มต้นแล้ว โปรแกรมจึงเปลี่ยนตำแหน่งเชิงตรรกะเป็น home และสิ้นสุด

การควบคุมเส้นทางของภารกิจนั้น

3.3.2.5 การตรวจสิ่งกีดขวางด้วย LiDAR

LiDAR ส่งข้อมูลระยะหลายมุมให้ Raspberry Pi ฟังก์ชัน update_scan() คัดกรองค่าที่ใช้ไม่ได้หรืออยู่นอกช่วงวัด ปรับมุมตามการติดตั้ง LiDAR แล้วพิจารณาเฉพาะกรวยด้านหน้ารถดังโค้ดที่ 3.16

```

def update_scan(
    self,
    ranges: list[float],
    angle_min: float,
    angle_max: float,
    angle_increment: float,
    range_min: float = 0.05,
    range_max: float = 12.0,
) -> bool:
    """
    ประมวลผลข้อมูลการสแกน (ใช้ได้ทั้งกับ ROS 2 LaserScan หรือ Raw Data)
    """
    obstacle_count = 0
    min_dist = float("inf")
    min_angle = 0.0

    yaw_offset_rad = math.radians(self.yaw_offset_deg)
    half_cone_rad = math.radians(self.front_cone_deg)

    for i, dist in enumerate(ranges):
        # ตรวจสอบค่าระยะที่สมเหตุสมผล

```

```

if math.isnan(dist) or math.isinf(dist):
    continue
if dist < max(range_min, self.min_clearance_m) or dist > range_max:
    continue

# คำนวณมุมของลำแสงนี้ เทียบกับหน้ารถ (Robot Forward = 0 rad)
raw_angle = angle_min + i * angle_increment
robot_angle = raw_angle + yaw_offset_rad

# ปรับมุมให้อยู่ในช่วง [-π, +π]
robot_angle = (robot_angle + math.pi) % (2.0 * math.pi) - math.pi

# ตรวจสอบว่าอยู่ในกรวยด้านหน้าหรือไม่ (-half_cone <= angle <= +half_cone)
if abs(robot_angle) <= half_cone_rad:
    if dist <= self.stop_distance_m:
        obstacle_count += 1
        if dist < min_dist:
            min_dist = dist
            min_angle = math.degrees(robot_angle)

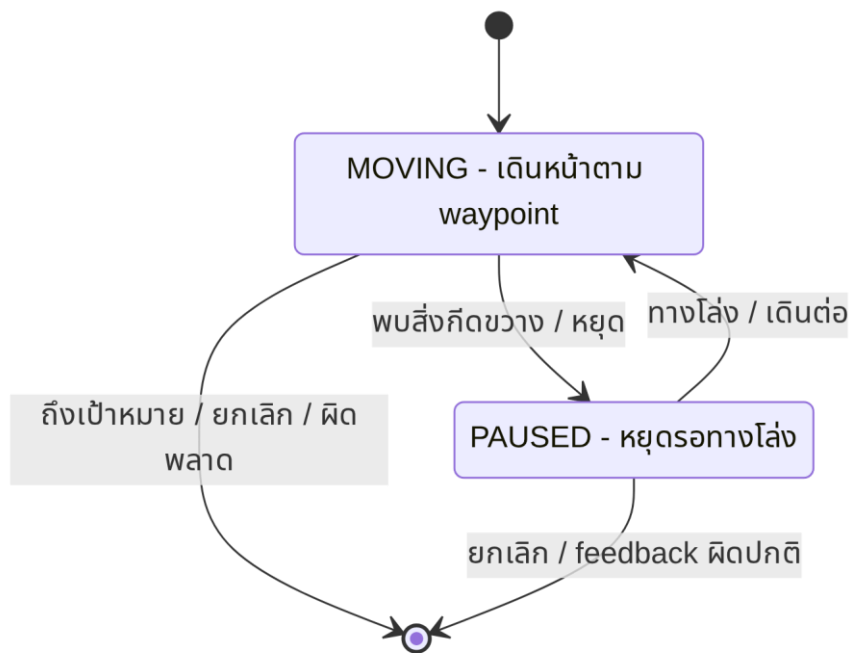
if obstacle_count >= self.min_obstacle_points:
    if not self._obstacle_detected:
        logger.warning(
            f"[LiDAR Safety] OBSTACLE DETECTED at {min_dist:.2f}m ({min_angle:.1f}°)"
        )
        self._obstacle_detected = True
        self._closest_distance = min_dist
        self._closest_angle_deg = min_angle
    else:
        if self._obstacle_detected:
            logger.info("[LiDAR Safety] Obstacle cleared. Path is open.")
        self._obstacle_detected = False
        self._closest_distance = float("inf")
        self._closest_angle_deg = 0.0

return self._obstacle_detected

```

โค้ดที่ 3.16 การคัดกรองข้อมูล LiDAR และเปลี่ยนสถานะสิ่งกีดขวาง

เมื่อพบสิ่งกีดขวางระหว่างเดินทาง โปรแกรมนำทางจะสั่งหยุดมอเตอร์และรออยู่ในช่วงเส้นทางเดิม โดยยังคงรับข้อมูล LiDAR ต่อ หากทางโล่งจึงสั่งเดินทางจากระยะที่เหลือ การหยุดดังกล่าวไม่เปลี่ยนสถานะ FSM หลักหรือถือว่าหุ่นยนต์เดินทางถึงโตะแล้ว ลำดับการหยุดและเดินต่อแสดงในรูปที่ 21



รูปที่ 21 แผนภาพการหยุดและเดินต่อเมื่อ LiDAR พบสิ่งกีดขวาง

3.3.2.6 ROS 2 และการใช้ SLAM

ROS 2 ใช้เป็นช่องทางแลกเปลี่ยนข้อมูลระหว่างส่วนรับ LiDAR, odometry และชุดคำสั่งเคลื่อนที่ ข้อมูลสแกนและตำแหน่งที่ได้ยังนำไปใช้กับ SLAM เพื่อสร้างหรือแสดงแผนที่ระหว่างการพัฒนา อย่างไรก็ตาม การกิจส่งอาหารของต้นแบบนี้ยังเดินตามเส้นทาง Waypoint ที่กำหนดไว้ ไม่ได้ให้ SLAM หรือ Nav2 เลือกเส้นทางไปโต๊ะโดยอัตโนมัติ การแยกบทบาทนี้ช่วยให้รายงานอธิบายความสามารถของระบบตรงกับสิ่งที่โปรแกรมทำจริง

3.3.3 รูปแบบข้อมูลที่ใช้สื่อสารระหว่าง Raspberry Pi กับ Arduino

3.3.3.1 ช่องทางเชื่อมต่อและรูปแบบข้อความ

Raspberry Pi 4 เชื่อมต่อกับ Arduino Uno ผ่านสาย USB โดยใช้การสื่อสารแบบ Serial ที่อัตรา 115,200 บิตต่อวินาที ข้อมูลที่แลกเปลี่ยนเป็นข้อความหนึ่งบรรทัดต่อหนึ่งรายการ และปิดท้ายด้วยอักขระขึ้นบรรทัดใหม่ (\n) ตัวอย่างเช่น V:0.250,0.250\n โปรแกรม Arduino อ่านข้อความจนถึงตัวจบบรรทัดแล้วจึงพิจารณาคำนำหน้า ส่วน Raspberry Pi อ่านข้อความตอบกลับที่ละบรรทัดในลักษณะเดียวกัน วิธีนี้ทำให้ตรวจสอบข้อมูลได้จากหน้าต่าง Serial และแยกประเภทข้อมูลได้โดยไม่ต้องถอดรหัส

การอ่าน Keypad ผ่าน I2C เป็นการเชื่อมต่อเฉพาะระหว่าง Arduino กับโมดูล PCF8574 โดยใช้ขา A4 (SDA) และ A5 (SCL) ไม่ได้เป็นสายสื่อสารระหว่าง Raspberry Pi กับ Arduino เมื่อ Arduino ตรวจพบการกดปุ่มแล้วจึงเปลี่ยนเหตุการณ์นั้นเป็นข้อความ KEY:<ปุ่ม> เพื่อส่งผ่าน USB Serial ไปยัง Raspberry Pi

ข้อความหลายชนิดใช้พอร์ต Serial เส้นเดียวกัน จึงอาจได้รับค่า Encoder หรือปุ่ม Keypad ระหว่างที่กำลังรอผลคำสั่งเคลื่อนที่ โปรแกรมฝั่งรับต้องตรวจค่านำหน้า ENCODER:, KEY: และ STATUS: ก่อนนำข้อมูลไปใช้

3.3.3.2 คำสั่งจาก Raspberry Pi ไปยัง Arduino

Raspberry Pi ส่งคำสั่งให้ Arduino ควบคุมมอเตอร์และไฟแสดงทิศทางตามภารกิจที่กำลังทำ ตารางที่ 3.3 สรุปรูปแบบคำสั่งและผลตอบกลับ โดยคำสั่งความเร็วต่อเนื่องใช้หน่วยเมตรต่อวินาที ส่วนคำสั่งเดินหน้าและหมุน เป็นคำสั่งแบบมีเป้าหมายที่ Arduino แจ้งเมื่อทำเสร็จ

คำสั่ง	ตัวอย่าง	การทำงานและผลตอบกลับ
V:<ซ้าย>,<ขวา>	V:0.250,0.250	กำหนดความเร็วล้อซ้ายและขวา หน่วย m/s; ค่าติดลบคือทิศกลับ ต้องส่งซ้ำระหว่างวิ่ง คำสั่งนี้ไม่มีข้อความตอบรับเฉพาะ
FORWARD:<เมตร>	FORWARD:1.250	วิ่งตรงตามระยะที่กำหนด โดย firmware รับค่ามากกว่า 0 แต่น้อยกว่า 20 เมตร; เมื่อเสร็จส่ง STATUS:DONE
TURN:<องศา>	TURN:-90.0	หมุนตามมุมที่กำหนด ค่าบวกคือเลี้ยวซ้ายหรือทวนเข็มนาฬิกา ค่าลบคือเลี้ยวขวาหรือตามเข็มนาฬิกา; เมื่อเสร็จส่ง STATUS:DONE
INDICATOR:<ทิศ>	INDICATOR:LEFT	กำหนดลูกศร LEFT, RIGHT หรือ OFF บน MAX7219; ค่าถูกต้องตอบ STATUS:INDICATOR
STOP	STOP	หยุดมอเตอร์ ยกเลิกคำสั่งที่กำลังทำ และล้าง motion fault; ตอบ STATUS:DONE

ตารางที่ 3.3 คำสั่งที่ Raspberry Pi ส่งไปยัง Arduino Uno

ขณะควบคุมความเร็วต่อเนื่อง Raspberry Pi ส่ง V: เป็นระยะ หากต้องการหยุดและล้าง fault สามารถส่ง V:0.000,0.000 ได้ แต่คำสั่งนี้ไม่ส่ง STATUS:DONE กลับมา ต่างจาก STOP ที่มีข้อความตอบรับ

หาก Arduino ไม่ได้รับ V: ใหม่ภายใน 300 มิลลิวินาที จะสั่งหยุดมอเตอร์โดยอัตโนมัติ เงื่อนไขนี้เป็นชั้นป้องกันกรณีการสื่อสารขาดช่วง ไม่ใช่ตัวแทนการตรวจสิ่งกีดขวางด้วย LiDAR

3.3.3.3 ข้อมูลและสถานะที่ Arduino ส่งกลับ

Arduino ส่งข้อมูลการหมุนของล้อ เหตุการณ์จาก Keypad และสถานะการทำงานกลับผ่านพอร์ตเดียวกัน ค่า Encoder เป็นจำนวน tick สะสมของล้อซ้ายและขวา ส่งทุก 100 มิลลิวินาที ส่วน KEY: ส่งเมื่อพบการกดปุ่มใหม่ ไม่ใช่ข้อมูลที่ส่งซ้ำทุกครั้งที่โปรแกรมวนรอบ ตารางที่ 3.4 แสดงความหมายของข้อความแต่ละประเภท

ข้อมูลจาก Arduino	ตัวอย่าง	ความหมาย
ENCODER:<ซ้าย>,<ขวา>	ENCODER:1234,-1228	จำนวน tick สะสมของล้อซ้ายและขวา เป็นจำนวนเต็มมีเครื่องหมาย ใช้คำนวณ odometry
KEY:<ปุ่ม>	KEY:A	อักขระจาก Keypad ที่กดใหม่ อยู่ในชุด 0-9, A-D, * และ # แล้วส่งต่อให้ POS
STATUS:READY	STATUS:READY	Arduino เริ่มทำงานและพร้อมรับคำสั่ง
STATUS:DONE	STATUS:DONE	คำสั่ง FORWARD:, TURN: หรือ STOP ทำงานเสร็จ
STATUS:INDICATOR	STATUS:INDICATOR	รับคำสั่งไฟเลี้ยวที่ถูกต้องแล้ว
STATUS:ERROR	STATUS:ERROR	คำสั่งไม่รู้จำ หรือข้อมูลคำสั่งถูกปฏิเสธ
STATUS:STALL	STATUS:STALL	พบว่ามอเตอร์ถูกสั่งให้เคลื่อนที่ แต่ Encoder ไม่คืบหน้าตามเกณฑ์ จึงตัดกำลังขับ

ตารางที่ 3.4 ข้อมูลที่ Arduino Uno ส่งกลับไปยัง Raspberry Pi

เมื่อได้รับ STATUS:STALL ระบบต้องไม่ปล่อยให้รถทำงานต่อจากคำสั่งเดิม การล้าง fault ของ firmware ทำได้ด้วย STOP หรือคำสั่งความเร็วศูนย์ จากนั้นจึงตรวจสอบสาเหตุ เช่น ล้อติดขัด สาย Encoder หลวม หรือแรงดันจ่ายมอเตอร์ไม่เพียงพอ ก่อนเริ่มการเคลื่อนที่อีกครั้ง ส่วน STATUS:ERROR บอกว่าคำสั่งไม่ถูกต้อง แต่ไม่ได้รับรายละเอียดข้อผิดพลาดเป็นรหัสแยก

3.3.3.4 การนำโปรโตคอลไปใช้ในโปรแกรมควบคุม

การสื่อสารระหว่าง Raspberry Pi กับ Arduino เป็นข้อความผ่าน USB Serial ตามรูปแบบในตารางที่ 3.3 และ 3.4 ตัวอย่างเช่น เมื่อให้ล้อทั้งสองเคลื่อนที่ด้วยความเร็วเท่ากัน Raspberry Pi จะส่ง V:0.250,0.250 พร้อมอักขระขึ้นบรรทัดใหม่ Arduino อ่านข้อความนี้ แล้วนำค่าความเร็วซ้ายและขวาไปควบคุมมอเตอร์ คำสั่ง V: ไม่มี STATUS:DONE ตอบกลับ จึงต้องส่งค่าความเร็วซ้ำระหว่างเคลื่อนที่ และส่งความเร็วเป็นศูนย์เมื่อต้องการหยุด

ระหว่างนั้น Arduino ส่ง ENCODER:<ซ้าย>,<ขวา> ทุก 100 มิลลิวินาที เพื่อให้ Raspberry Pi คำนวณระยะทางและมุมหัน หากมีการกด Keypad จะส่ง KEY:<ปุ่ม> เช่น KEY:A ไปให้ POS ติดตามสถานะปัจจุบัน ข้อความทั้งสองชนิดใช้สาย USB เส้นเดียวกัน โปรแกรมจึงแยกจากค่านำหน้า ENCODER: และ KEY: ก่อนนำไปใช้ ไม่ว่าคำสั่งความเร็วจะมาจากโปรแกรมควบคุมโดยตรงหรือผ่าน ROS 2 ข้อความที่ Arduino ได้รับทาง USB ยังคงเป็นรูปแบบ V: ตามตารางที่ 3.3

3.4 ขั้นตอนการประกอบและตรวจสอบระบบ

1. ติดตั้งมอเตอร์ ล้อ และชิ้นวางอาหารเข้ากับฐานหุ่นยนต์ จากนั้นกำหนดตำแหน่งของแบตเตอรี่ Raspberry Pi, Arduino Uno, L298N, ชุดแปลงแรงดัน และแผงกระจายไฟ โดยจัดให้สายไฟและสายสัญญาณไม่สัมผัสล้อหรือส่วนที่เคลื่อนที่
2. ติดตั้งแบตเตอรี่ 12 V สวิตช์ และแผงกระจายไฟตามแผนผังวงจร แยกสายไฟสำหรับมอเตอร์ออกจากสายที่นำไปยังชุดแปลงแรงดัน และต่อกราวด์ของวงจรที่ใช้สัญญาณร่วมกัน ก่อนเชื่อมต่ออุปกรณ์อิเล็กทรอนิกส์ให้ตรวจขั้วบวก ขั้วลบ และการลัดวงจรของสายแต่ละเส้น
3. ต่อชุดแปลงแรงดันเข้ากับแหล่งจ่าย 12 V แล้วปรับและวัดแรงดันขาออกให้ตรงกับพิกัดของอุปกรณ์ที่รับไฟ โดยตรวจทั้งขณะยังไม่ต่อโหลดและหลังต่อโหลด การจ่ายไฟให้ Raspberry Pi, Arduino, จอสัมผัส และโมดูล 5 V ต้องตรวจตามขั้วและพิกัดของอุปกรณ์จริง รวมถึงตรวจการตั้งจัมเปอร์ 5V-EN ของ L298N ให้สอดคล้องกับวิธีจ่ายไฟที่ใช้
4. ต่อมอเตอร์ทั้งสองเข้ากับขาเอาต์พุตของ L298N และต่อสายควบคุมจาก Arduino โดยขา D8, D9 และ D10 ใช้กำหนดทิศทางและความเร็วของมอเตอร์ซ้าย ส่วน D12, D7 และ D11 ใช้กับมอเตอร์ขวา จากนั้นตรวจว่าสายมอเตอร์ซ้ายและขวาดตรงกับตำแหน่งล้อจริง

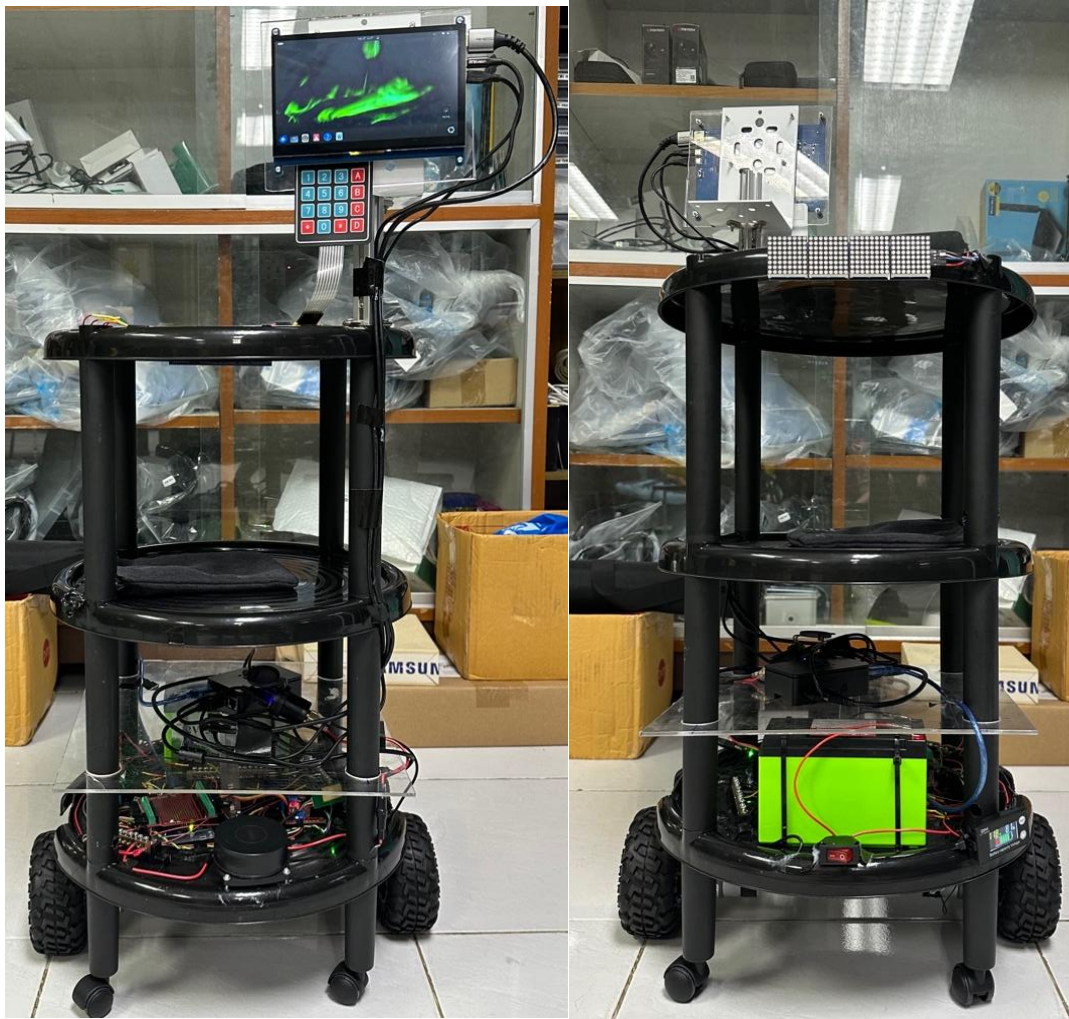
5. ต่อสายไฟและสายสัญญาณของ Encoder ทั้งสองตัวเข้ากับ Arduino โดยใช้ขา A0–A3 สำหรับสัญญาณสองเฟสของมอเตอร์ซ้ายและขวา ทดสอบหมุนล้อทีละข้างเพื่อตรวจสอบว่าค่า Encoder เปลี่ยนตามล้อที่หมุน และตรวจทิศทางการนับก่อนทดสอบการเคลื่อนที่
6. ต่อ Keypad 4x4 เข้ากับโมดูล PCF8574 แล้วต่อโมดูลเข้ากับ Arduino ผ่านบัส I²C ที่ขา A4 (SDA) และ A5 (SCL) พร้อมสายไฟและกราวด์ ตรวจสอบที่อยู่ของโมดูลซึ่งกำหนดไว้เป็น 0x20 และทดสอบว่าการกดแต่ละปุ่มส่งค่าตรงกับปุ่มที่กด
7. ต่อจอ MAX7219 Dot Matrix เข้ากับ Arduino โดยใช้ขา D4 สำหรับ DIN, D5 สำหรับ CS และ D6 สำหรับ CLK พร้อมต่อไฟเลี้ยงและกราวด์ ทดสอบการแสดงผลลูกศรซ้าย ลูกศรขวา และสัญลักษณ์หยุด ก่อนให้แสดงผลตามคำสั่งขณะหุ่นยนต์ทำงาน
8. ติดตั้ง LiDAR บนตัวหุ่นยนต์โดยกำหนดทิศด้านหน้าให้ตรงกับทิศการเคลื่อนที่ ต่อ LiDAR เข้ากับบอร์ดแปลงสัญญาณและเชื่อมต่อบอร์ดดังกล่าวกับ Raspberry Pi ผ่าน USB จากนั้นตรวจสอบว่าระบบอ่านข้อมูลระยะได้ต่อเนื่อง และไม่มีส่วนของโครงรถบังมุมตรวจจับด้านหน้า
9. ต่อจอสัมผัสเข้ากับ Raspberry Pi โดยใช้สาย Micro HDMI สำหรับภาพและสาย USB สำหรับข้อมูลสัมผัส พร้อมจ่ายไฟให้จอตามข้อกำหนดของอุปกรณ์ จากนั้นเชื่อม Arduino Uno กับ Raspberry Pi ผ่าน USB และตรวจสอบว่า Raspberry Pi มองเห็นอุปกรณ์ทั้ง Arduino, LiDAR และจอสัมผัส
10. ทดสอบวงจรควบคุมทีละส่วนก่อนให้ล้อสัมผัสพื้น เริ่มจากตรวจแรงดันไฟเลี้ยงและการสื่อสาร Serial แล้วทดสอบค่า Encoder, ปุ่ม Keypad, การแสดงผล Dot Matrix และการหมุนของมอเตอร์ด้วยความเร็วต่ำ หากล้อหมุนผิดทิศหรือค่า Encoder ไม่สัมพันธ์กับล้อ ให้แก้การต่อสายหรือการกำหนดค่าก่อนทดสอบขั้นต่อไป
11. เปิดระบบ POS และโปรแกรมควบคุมบน Raspberry Pi ทดลองกำหนดเส้นทางและไต่ะปลายทางผ่านจอสัมผัส รวมทั้งทดสอบการกด Keypad เพื่อตรวจสอบว่าข้อมูลที่ผู้ใช้ป้อนเปลี่ยนสถานะของรายการเดียวกัน และคำสั่งจาก Raspberry Pi ส่งถึง Arduino ได้ถูกต้อง
12. ทดสอบการเดินทางตาม Waypoint ในพื้นที่จำลอง โดยเริ่มจากการไปยังไต่ะหนึ่งไต่ะก่อน แล้วทดสอบหลายไต่ะตามลำดับ ระหว่างที่หุ่นยนต์เดินทาง ให้นำสิ่งกีดขวางเข้าและออกจากบริเวณด้านหน้า LiDAR เพื่อตรวจการหยุดรอและเดินทางต่อ สุดท้ายทดสอบการยืนยันรับอาหาร การไปไต่ะถัดไป และการกลับจุดเริ่มต้นเมื่อส่งอาหารครบ

บทที่ 4

ผลการดำเนินงาน

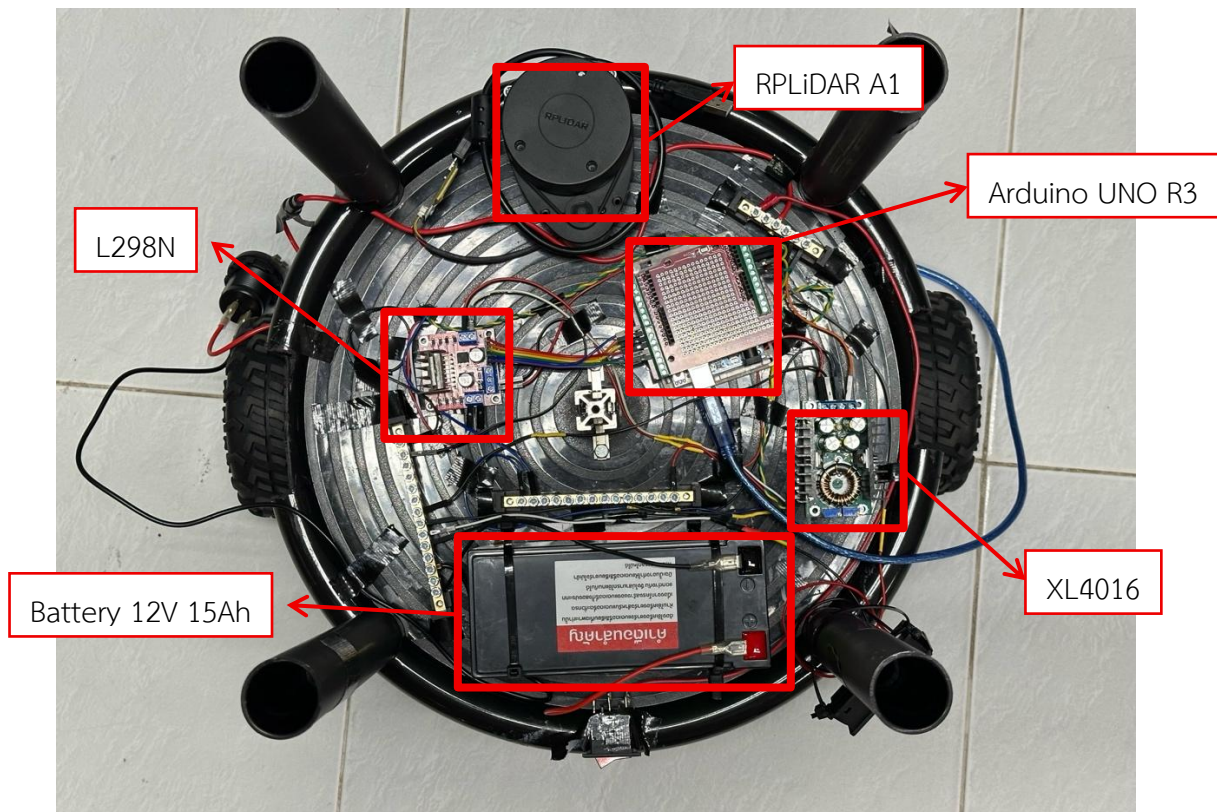
4.1 ผลการประกอบหุ่นยนต์และการเชื่อมต่อระบบ

4.1.1. ภาพหุ่นยนต์ต้นแบบหลังประกอบ



รูปที่ 22 - 23 ด้านหน้าของหุ่นยนต์ และด้านหลังของหุ่นยนต์

4.1.2. การติดตั้ง Raspberry Pi, Arduino, LiDAR, มอเตอร์ และแบตเตอรี่



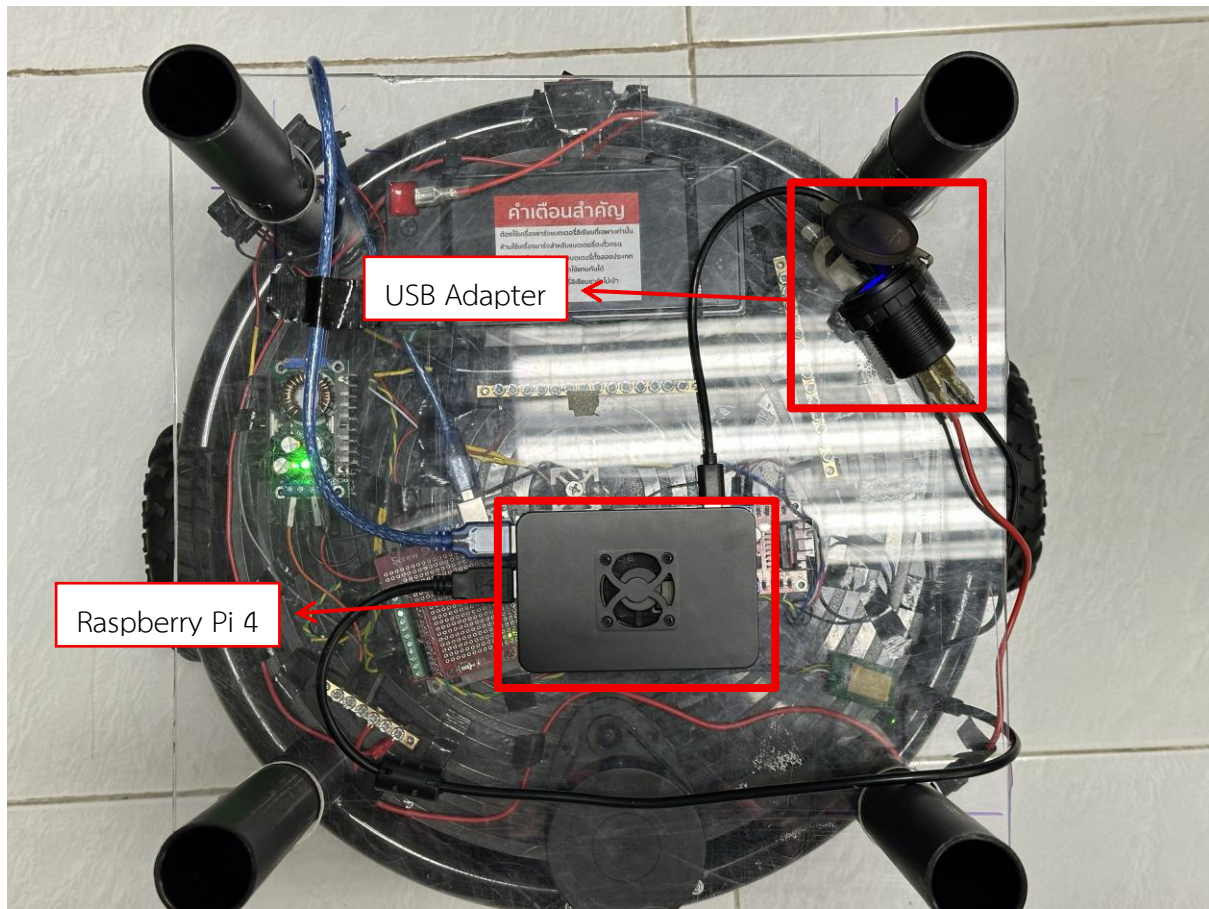
รูปที่ 24 ภาพรวมของหุ่นยนต์ในชั้นที่ 1

ในชั้นที่ 1 ของหุ่นยนต์จะประกอบไปด้วยอุปกรณ์ดังนี้

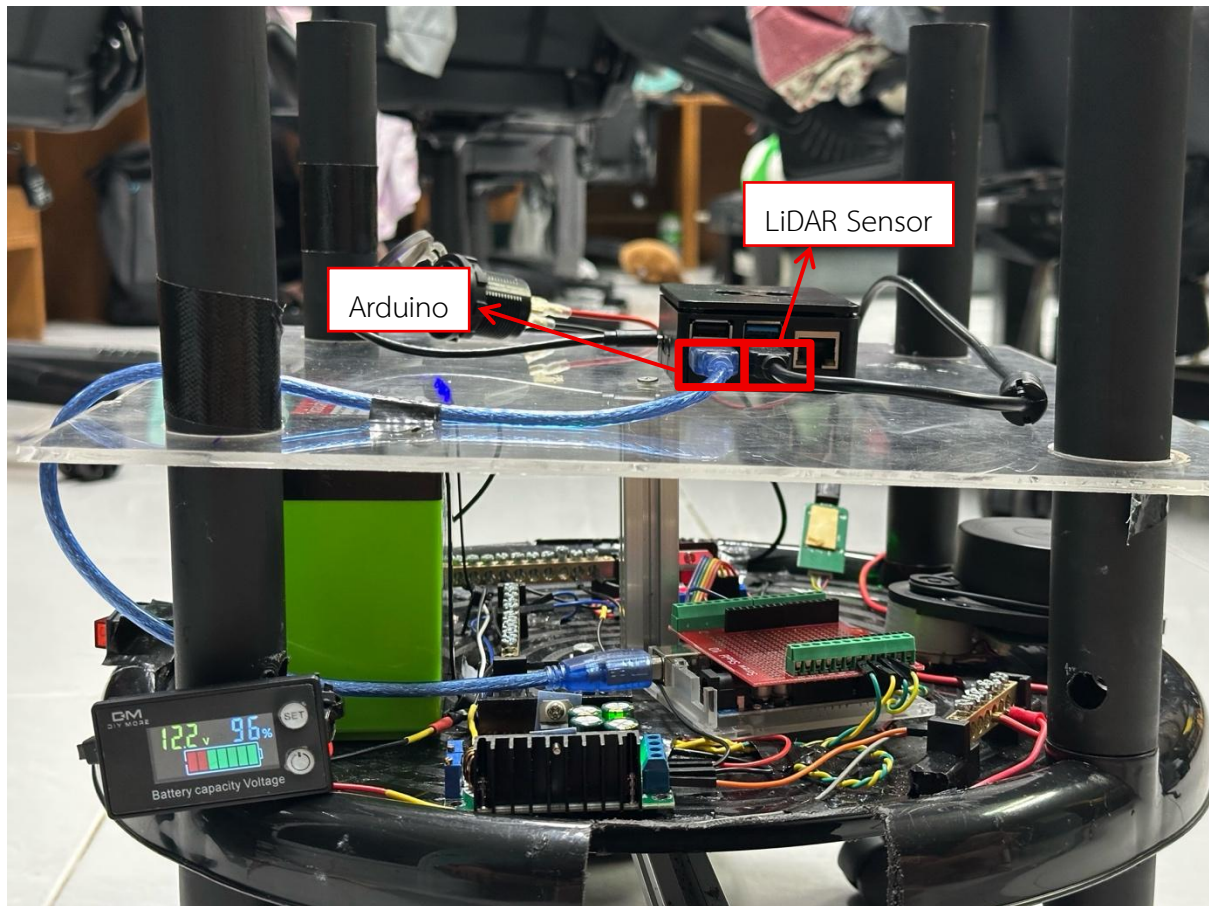
- L298N Motor Driver – ใช้ในการขับ DC Motor ทั้ง 2
- XL4016 DC-DC step-down voltage regulator – สำหรับ Step down 12V จาก Battery ให้เป็น 5V
- Arduino UNO R3 – Microcontroller หลักที่ใช้ในการควบคุม Motor
- RPLiDAR A1 – ใช้สำหรับตรวจสอบสิ่งกีดขวาง
- Battery 12V 15Ah – แหล่งจ่ายไฟหลักของหุ่นยนต์
- และประกอบไปด้วย Terminal 3 อัน ได้แก่
- 12V Terminal – แหล่งจ่ายไฟ 12V ต่อกับ 12V ของ Battery
- 5V Terminal – แหล่งจ่ายไฟ 5V ที่มาจาก OUTPUT ของ XL4016

- GND Terminal - ทำหน้าที่เป็น Ground รวม ต่อกับ GND ของ Battery

ภาพการเชื่อมต่อวงจรและการสื่อสารระหว่าง Raspberry Pi กับ Arduino



รูปที่ 25 ภาพรวมของหุ่นยนต์ในชั้นที่ 1.5 top view



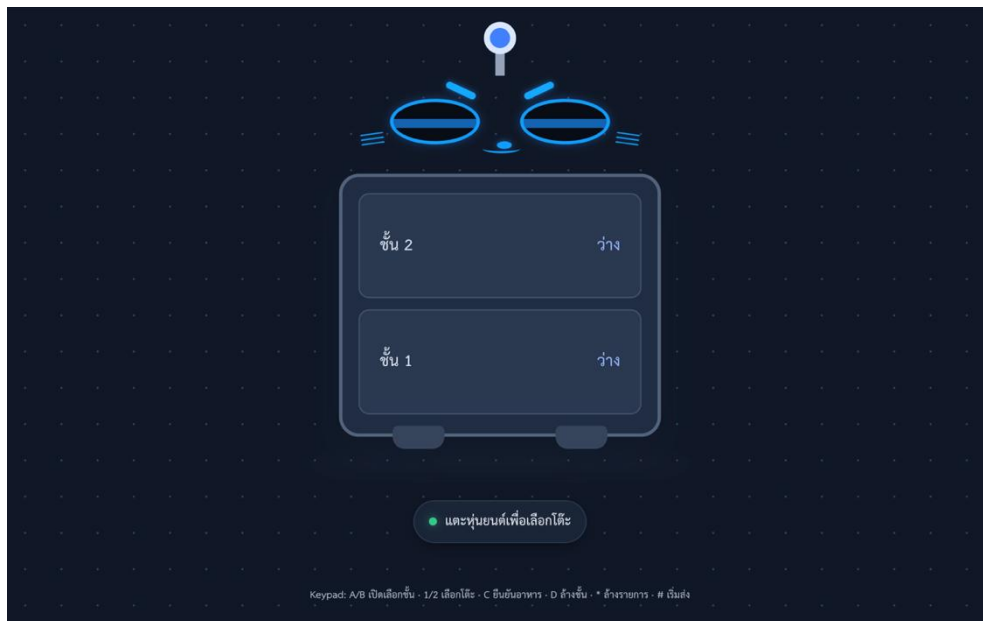
รูปที่ 26 ภาพรวมของหุ่นยนต์ในชั้นที่ 1.5 top view

ในชั้นที่ 1.5 ของหุ่นยนต์จะประกอบไปด้วยอุปกรณ์ดังนี้

- Raspberry Pi 4 – ใช้ในการอ่าน LiDAR และ สั่ง Arduino ให้เคลื่อนที่

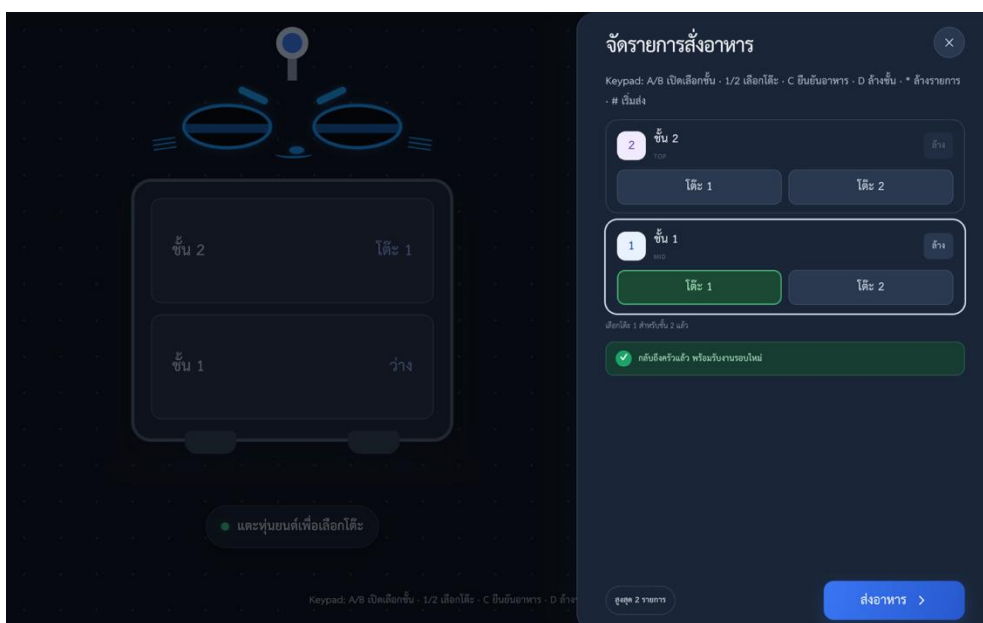
4.2 ผลการทำงานของจอสัมผัส POS/Kiosk และ Keypad

4.2.1. การเลือกชั้นวางอาหาร

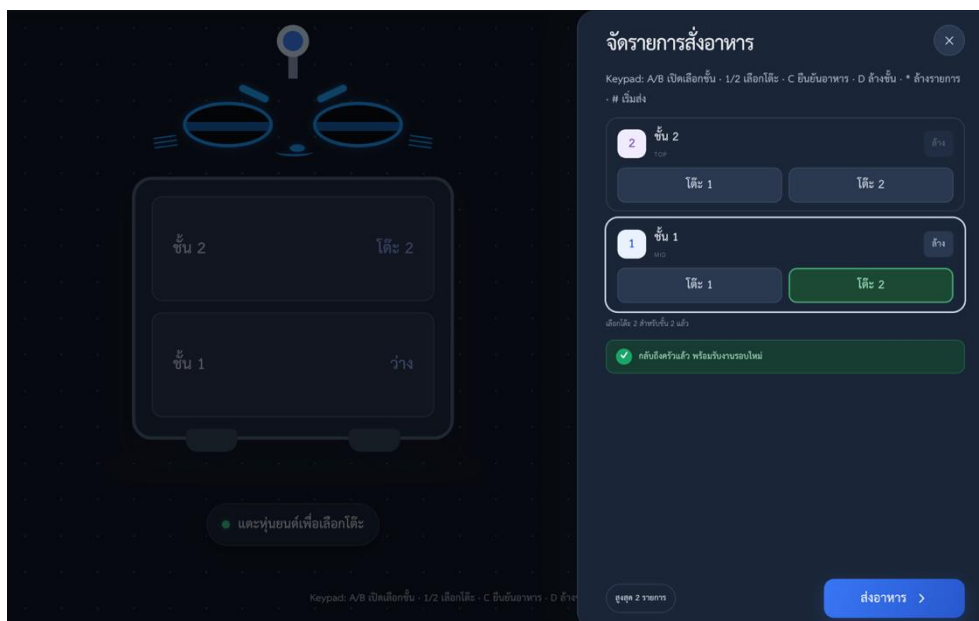


รูปที่ 27 หน้าจอ Kiosk แสดงสถานะชั้นวางอาหาร ให้ผู้ใช้แตะหุ่นยนต์เพื่อเริ่มเลือกโต๊ะปลายทาง เมื่อเริ่มหุ่นยนต์จะแสดงหน้าจอสำหรับการเลือกชั้นสำหรับใส่อาหาร เมื่อกด A หรือ B บน Keypad หรือใช้นิ้วในการเลือกชั้นบนหน้าจอ จะทำให้มี Sidebar สำหรับการจัดรายการสั่งอาหารแสดงออกมา

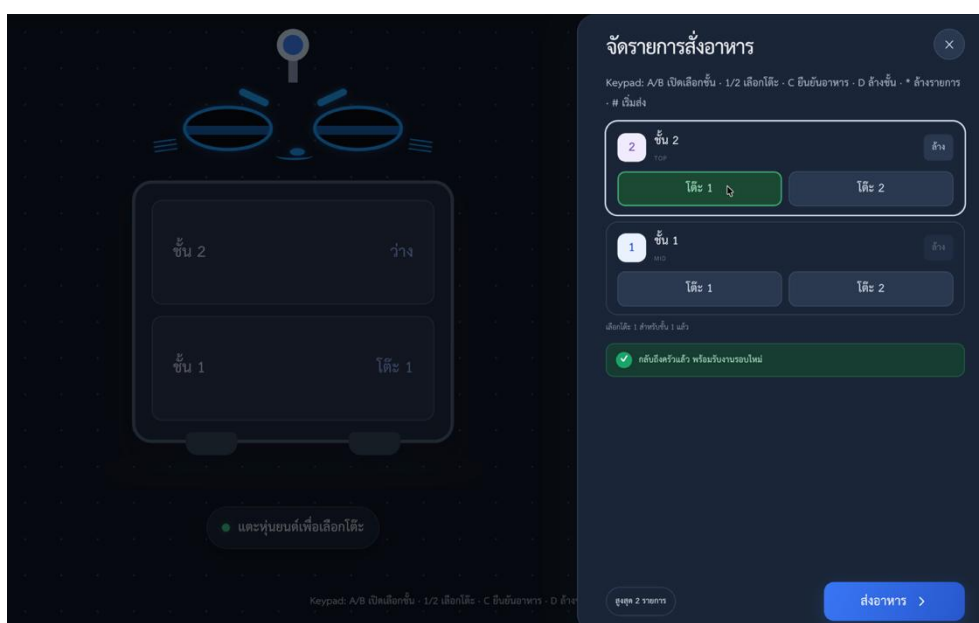
4.2.2. การเลือกโต๊ะปลายทาง



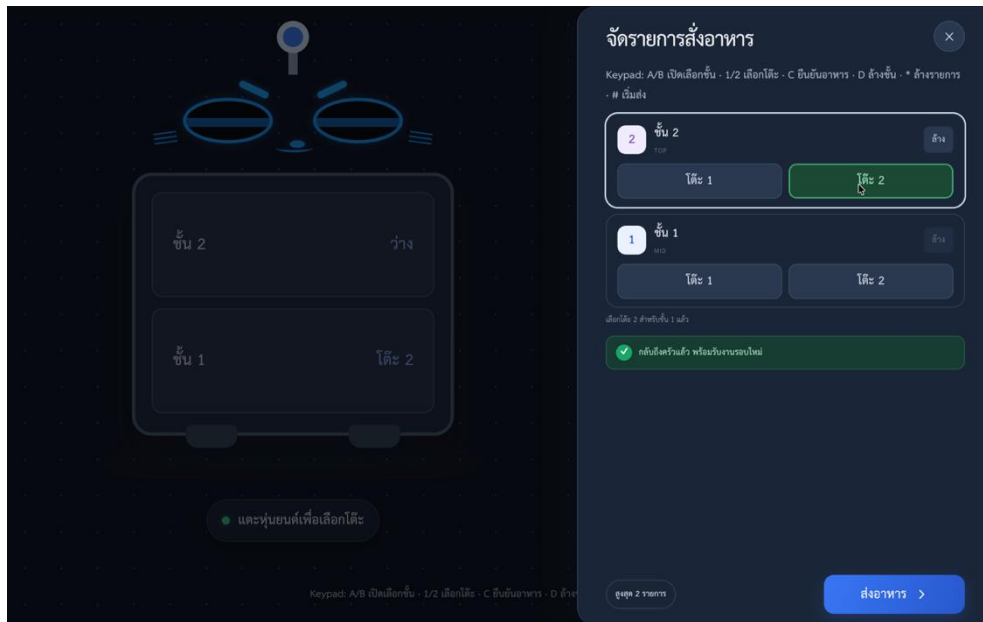
รูปที่ 28 หน้าจอจัดรายการสั่งอาหาร แสดงการเลือกโต๊ะที่ 1 เป็นปลายทางสำหรับชั้นวางที่ 1



รูปที่ 29 หน้าจอจัดรายการสั่งอาหาร แสดงการเลือกโต๊ะที่ 2 เป็นปลายทางครบสำหรับชั้นวางที่ 1



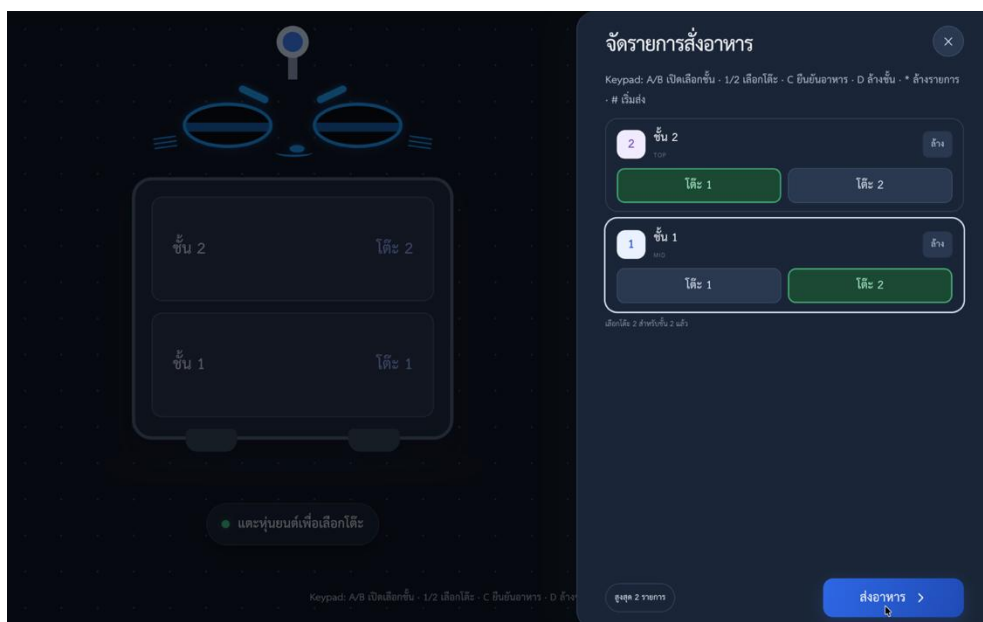
รูปที่ 30 หน้าจอจัดรายการสั่งอาหาร แสดงการเลือกโต๊ะที่ 1 เป็นปลายทางสำหรับชั้นวางที่ 2



รูปที่ 31 หน้าจอจัดรายการสั่งอาหาร แสดงการเลือกโต๊ะที่ 2 เป็นปลายทางสำหรับชั้นว่างที่ 2

ใน Sidebar สำหรับการจัดรายการสั่งอาหารจะมี 2 ส่วน คือส่วนของการเลือกชั้นที่จะวางอาหาร และปลายทางที่หุ่นยนต์จะต้องไปส่งอาหาร

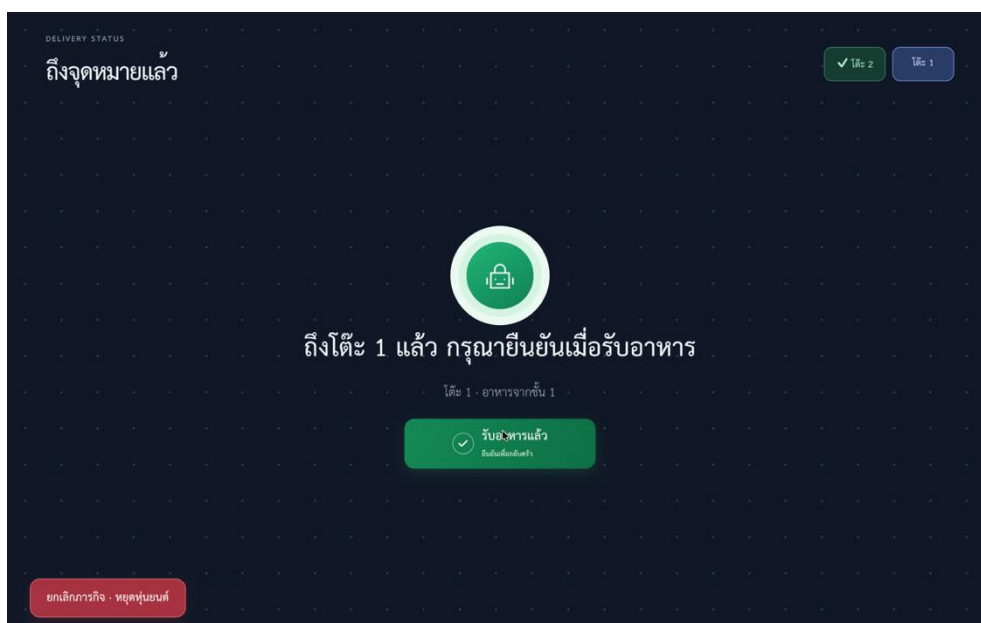
4.2.3. การยืนยันอาหารและเริ่มภารกิจ



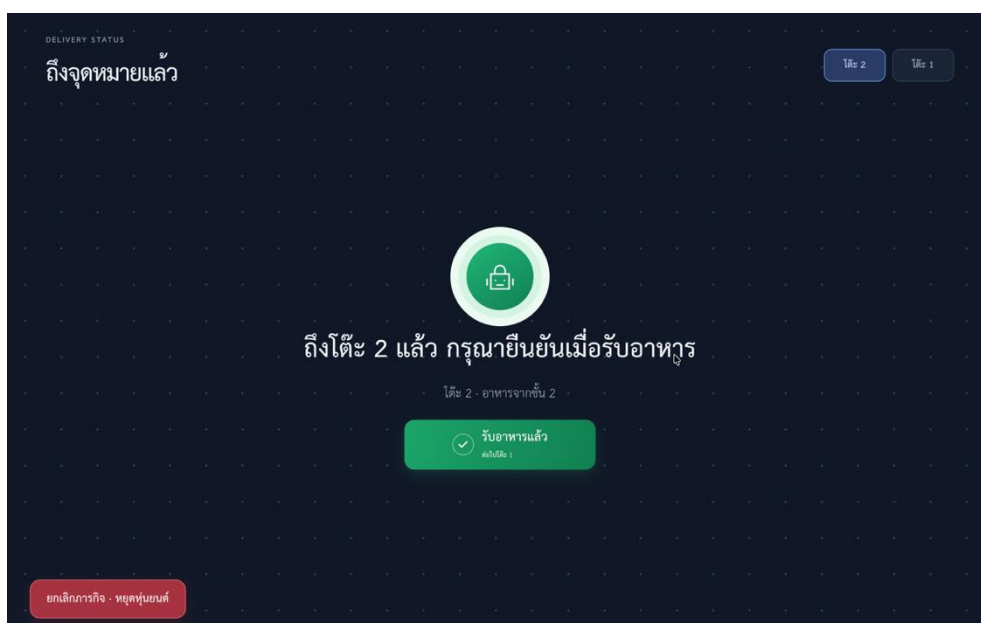
รูปที่ 32 หน้าจอยืนยันรายการอาหารครบถ้วนก่อนกดปุ่ม "ส่งอาหาร" เพื่อเริ่มภารกิจ

เมื่อกดส่งอาหารหุ่นยนต์จะมีการรอเป็นเวลา 5 วินาที ก่อนที่จะออกเดินทางไปยังปลายทางที่ได้รับมอบหมายไว้

4.2.4. การยืนยันรับอาหารผ่านจอสัมผัสหรือ Keypad



รูปที่ 33 หน้าจอแสดงสถานะการส่งอาหาร เมื่อหุ่นยนต์เดินทางถึงโต๊ะที่ 1 และรอการยืนยันรับอาหารจากผู้ใช้



รูปที่ 34 หน้าจอแสดงสถานะการส่งอาหาร เมื่อหุ่นยนต์เดินทางถึงโต๊ะที่ 2 หลังยืนยันรับอาหารที่โต๊ะแรกแล้ว

เมื่อถึงยังโต๊ะปลายทางจะมีการให้กดรับอาหาร หลังจากกดรับอาหารแล้ว หุ่นยนต์ก็จะเดินทางไปยังปลายทางต่อไป

4.3 ผลการทดสอบการเคลื่อนที่ของหุ่นยนต์

4.3.1. การเดินหน้า ถอยหลัง และหยุด



รูปที่ 35-36 หุ่นยนต์ตอนเริ่มเดินหน้าไปยังจุด Junction และหุ่นยนต์ระหว่างที่กำลังเดินหน้าไปยังจุด Junction ตามลำดับ

4.3.2. การหมุนอยู่กับที่



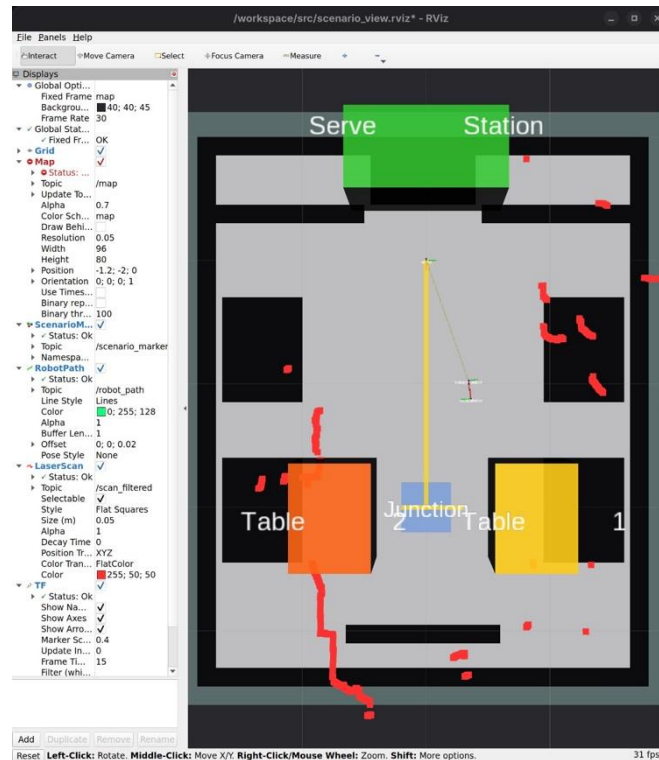
รูปที่ 37-38 หุ่นยนต์เตรียมพร้อมที่จะหมุนไปทางด้านซ้าย และ หุ่นยนต์ระหว่างที่กำลังหมุนไปทางด้านซ้าย ตามลำดับ



รูปที่ 39-40 หุ่นยนต์หมุนไปทางด้านซ้ายเสร็จสิ้น และ หุ่นยนต์เตรียมพร้อมที่จะหมุนไปทางด้านขวา ตามลำดับ



รูปที่ 41-42 หุ่นยนต์ระหว่างที่กำลังหมุนไปทางด้านขวา และ หุ่นยนต์หมุนไปทางด้านขวาเสร็จสิ้น ตามลำดับ



รูปที่ 45 แผนที่ห้องหุ่นยนต์ระหว่างเดินไปยังจุด Junction โปรแกรม Rviz2

4.3.5. การแสดงลูกศรบนจอ LED MATRIX



รูปที่ 46 ไฟเลี้ยวของหุ่นยนต์เมื่อจะทำการเลี้ยวขวา



รูปที่ 47 ไฟเลี้ยวของหุ่นยนต์เมื่อจะทำการเลี้ยวซ้าย

4.4 ผลการทำงานของระบบตรวจจับสิ่งกีดขวางและความปลอดภัย

4.4.1. การหยุดเมื่อ LiDAR ตรวจพบสิ่งกีดขวาง



รูปที่ 48 หุ่นยนต์หยุดเคลื่อนที่เมื่อเจอสิ่งกีดขวางขนาดเล็ก



รูปที่ 49 หุ่นยนต์หยุดเคลื่อนที่เมื่อเจอสิ่งกีดขวางขนาดใหญ่



รูปที่ 50 หน้าจอของหุ่นยนต์เมื่อเจอสิ่งกีดขวาง

4.4.2. การเดินทางต่อเมื่อเส้นทางปลอดภัย



รูปที่ 51 หน้าจอของหุ่นยนต์เมื่อไม่พบสิ่งกีดขวาง



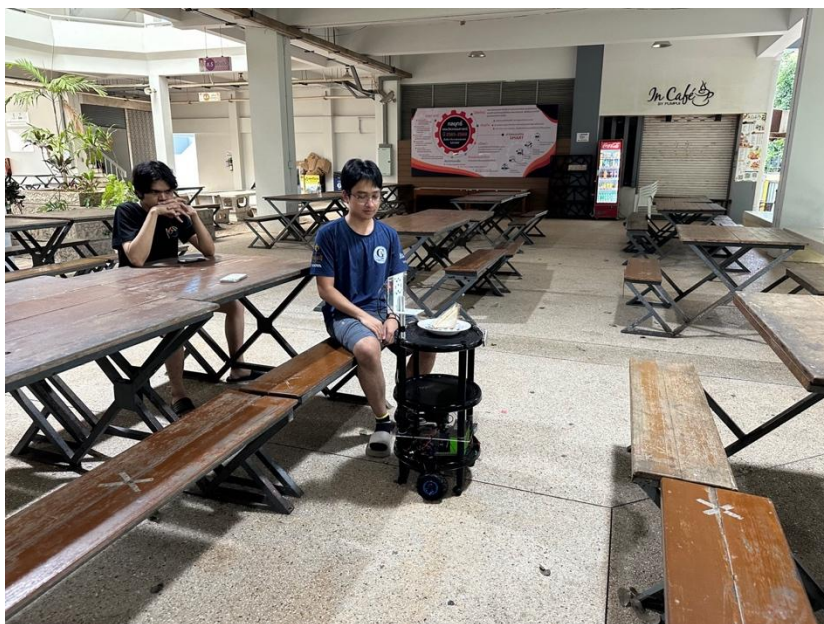
รูปที่ 52 หุ่นยนต์เคลื่อนที่ต่อหลังจากไม่มีสิ่งกีดขวาง

4.5 ผลการทดสอบภารกิจส่งอาหาร

4.5.1. การส่งอาหารไปยังโต๊ะเดียว

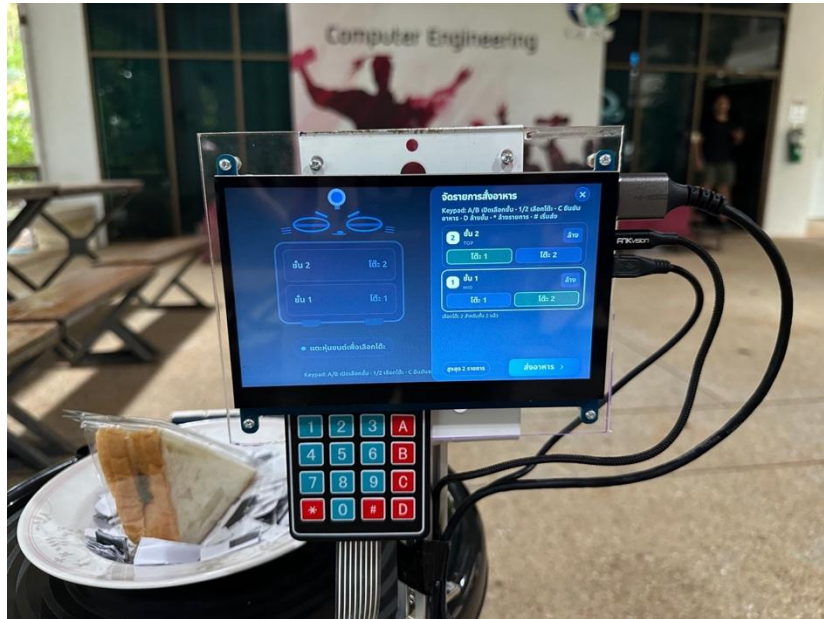


รูปที่ 53 หุ่นยนต์ต้นแบบขณะทดสอบส่งอาหารไปยังโต๊ะเดียว



รูปที่ 54 หุ่นยนต์เคลื่อนที่เข้าใกล้โต๊ะเป้าหมายพร้อมอาหารบนถาด ระหว่างการทดสอบส่งอาหารไปยังโต๊ะเดียว

4.5.2. การส่งอาหารไปยังหลายโต๊ะ



รูปที่ 55 หุ่นยนต์แสดงหน้าจอจัดการรายการสั่งอาหารสำหรับหลายโต๊ะ พร้อมอาหารวางบนถาดก่อนเริ่มภารกิจ



รูปที่ 56 หุ่นยนต์ระหว่างทดสอบภารกิจส่งอาหารไปยังหลายโต๊ะ



รูปที่ 57 หุ่นยนต์เดินทางระหว่างจุดส่งอาหารในภารกิจส่งอาหารหลายโต๊ะ



รูปที่ 58 หุ่นยนต์กำลังเดินทางไปยังอีกโต๊ะ หลังจากส่งอาหารให้โต๊ะแรกเสร็จ

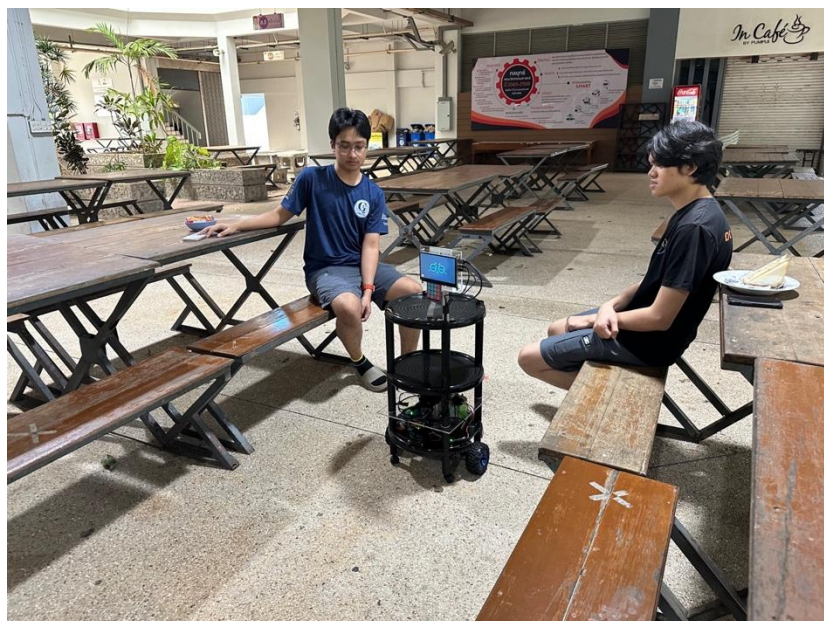
4.5.3. การรออนินยัรับอาหาร



รูปที่ 59 หุ่นยนต์รออนินยัรับอาหารจากผู้ที่ใช้โต๊ะปลายทาง

ถ้าหุ่นยนต์ถึงโต๊ะที่กำหนดแล้ว จะทำการแสดงหน้าอินยัรับอาหาร

4.5.4. การกลับมายังจุดเริ่มต้น



รูปที่ 60 หุ่นยนต์ระหว่างเดินทางกลับจุดเริ่มต้นหลังส่งอาหารเสร็จสิ้น



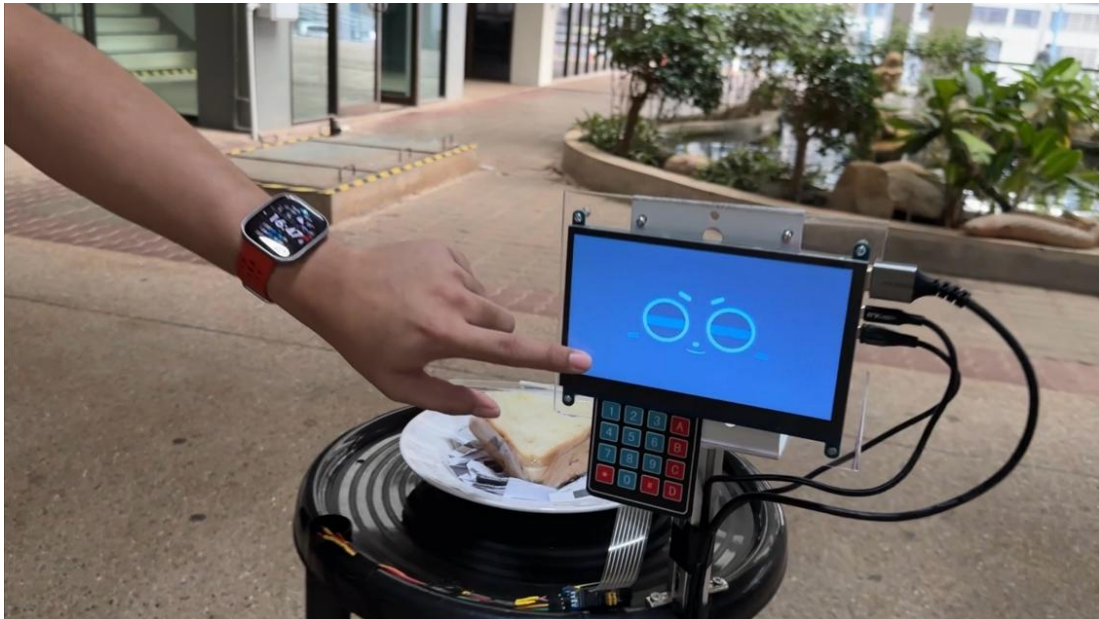
รูปที่ 61 หุ่นยนต์เดินทางกลับจุดเริ่มต้น มุมมองที่สองระหว่างการทดสอบ



รูปที่ 62 หุ่นยนต์กลับถึงจุดเริ่มต้นและพร้อมรับภารกิจใหม่

หลังจากส่งอาหารเสร็จสิ้น หุ่นยนต์จะเดินทางกลับไปยังจุดเริ่มต้น แล้วหมุนตัว 180 องศาจะถือว่าจบการส่งอาหาร 1 ครั้ง

4.5.5 การยกเลิกหรือรีเซ็ตภารกิจ



รูปที่ 63 การกดปุ่มบนตัวหุ่นยนต์เพื่อยกเลิกภารกิจ หน้าจอแสดงสถานะรอคำสั่ง



รูปที่ 64 หน้าจอหุ่นยนต์หลังกดยกเลิกภารกิจ กลับสู่สถานะพร้อมทำงาน

เมื่อมีการกดปุ่มยกเลิกภารกิจในระหว่างการส่ง สถานะของหุ่นยนต์จะกลับไปยังสถานะพร้อมรับออเดอร์

4.6 สรุปผลการทดสอบระบบโดยรวม

ลำดับ	รายการทดสอบ	วิธีทดสอบ	ผลที่คาดหวัง	ผลลัพธ์จริง
1	การเปิดระบบและเชื่อมต่ออุปกรณ์	เปิด Raspberry Pi, Arduino, LiDAR และจอสัมผัส	อุปกรณ์ทุกส่วนพร้อมทำงาน	ทำงานได้
2	การรับคำสั่งจากจอสัมผัส	เลือกชั้นวางและโต๊ะผ่าน POS/Kiosk	ระบบบันทึกข้อมูลถูกต้อง	ทำงานได้
3	การรับคำสั่งจาก Keypad	กดปุ่มเลือกชั้น โต๊ะ และเริ่มงาน	ระบบแปลคำสั่งได้ถูกต้อง	ทำงานได้
4	การเริ่มภารกิจส่งอาหาร	สร้างรายการอาหารแล้วกดเริ่มภารกิจ	หุ่นยนต์เริ่มเดินทาง	ทำงานได้
5	การสื่อสารระหว่าง Raspberry Pi กับ Arduino	ตรวจสอบการส่งคำสั่งและข้อมูล Encoder	รับส่งข้อมูลได้ต่อเนื่อง	ทำงานได้
6	การเดินหน้า ถอยหลัง และหยุด	สั่งให้หุ่นยนต์เคลื่อนที่แต่ละรูปแบบ	หุ่นยนต์เคลื่อนที่ตามคำสั่ง	ทำงานได้
7	การเลี้ยวซ้ายและขวา	สั่งให้หุ่นยนต์เลี้ยวตามทิศทาง	หุ่นยนต์เลี้ยวถูกต้องทิศทาง	ทำงานได้
8	การเดินทางตาม Waypoint	กำหนดจุดหมายและสั่งให้หุ่นยนต์เดินทาง	หุ่นยนต์ไปถึงจุดหมายตามลำดับ	ทำงานได้
9	การทำงานของ Encoder และ Odometry	เปรียบเทียบระยะทางจริงกับค่าที่คำนวณ	ระบบคำนวณตำแหน่งได้ใกล้เคียงความจริง	ทำงานได้
10	การตรวจจับสิ่งกีดขวางด้วย LiDAR	วางสิ่งกีดขวางด้านหน้าหุ่นยนต์	หุ่นยนต์หยุดและเดินต่อเมื่อทางปลอดภัย	ทำงานได้
11	การแสดงผลบนจอ LED MATRIX	สั่งให้หุ่นยนต์เลี้ยวซ้ายและขวา	จอแสดงลูกศรถูกต้อง	ทำงานได้

ลำดับ	รายการทดสอบ	วิธีทดสอบ	ผลที่คาดหวัง	ผลลัพธ์จริง
12	การยืนยันรับอาหาร	ยืนยันผ่านจอสัมผัสและ Keypad	ระบบเปลี่ยนสถานะภารกิจ	ทำงานได้
13	การส่งอาหารไปโต๊ะเดียว	เลือกโต๊ะหนึ่งโต๊ะและเริ่มภารกิจ	หุ่นยนต์ไปถึงโต๊ะและกลับจุดเริ่มต้น	ทำงานได้
14	การส่งอาหารหลายโต๊ะ	กำหนดรายการส่งอาหารหลายรายการ	หุ่นยนต์ส่งตามลำดับและกลับจุดเริ่มต้น	ทำงานได้
15	การยกเลิกหรือรีเซ็ตภารกิจ	ยกเลิกภารกิจหรือกดรีเซ็ตเมื่อเกิดข้อผิดพลาด	หุ่นยนต์หยุดและกลับสู่สถานะพร้อมรับงาน	ทำงานได้
16	การทำงานซ้ำหลายรอบ	ทดสอบภารกิจซ้ำอย่างน้อย 3 รอบ	ระบบทำงานต่อเนื่องโดยไม่ค้าง	ทำงานได้

ตารางที่ 4.1 ผลการทดสอบหุ่นยนต์เสิร์ฟโดยรวม

4.7 ปัญหาและข้อจำกัดที่พบ

จากการทดสอบพบว่า หุ่นยนต์ยังเดินทางตาม Waypoint ที่กำหนดไว้ล่วงหน้า จึงยังปรับเส้นทางตามการเปลี่ยนแปลงของตำแหน่งโต๊ะหรือสิ่งกีดขวางได้จำกัด นอกจากนี้ การระบุตำแหน่งจาก Encoder และ Odometry อาจเกิดความคลาดเคลื่อนสะสมระหว่างการเคลื่อนที่ ส่วนการยืนยันรับอาหารยังต้องให้ผู้ใช้ดำเนินการผ่านจอสัมผัสหรือ Keypad ข้อจำกัดเหล่านี้ทำให้การใช้งานยังต้องมีผู้ดูแล และควรทดสอบเพิ่มเติมในสภาพแวดล้อมที่หลากหลายก่อนนำไปใช้งานจริง

บทที่ 5

สรุป และอภิปรายผล

5.1 สรุปผลการดำเนินงาน

โครงการนี้มีวัตถุประสงค์เพื่อประยุกต์ใช้ความรู้จากวิชา Embedded System Developer Module ในการออกแบบและพัฒนาต้นแบบหุ่นยนต์ส่งอาหารอัตโนมัติสำหรับร้านอาหารขนาดเล็ก โดยระบบสามารถรับรายการอาหาร กำหนดเส้นทางและโต๊ะปลายทาง ควบคุมการเคลื่อนที่ และส่งอาหารภายในพื้นที่จำลองได้ โครงการใช้ Raspberry Pi 4 เป็นหน่วยประมวลผลระดับสูงสำหรับการจัดการระบบ POS/Kiosk การจัดลำดับภารกิจ และการสื่อสารกับอุปกรณ์ต่าง ๆ ส่วน Arduino Uno R3 ทำหน้าที่ควบคุมมอเตอร์ อ่านค่า Encoder และรับข้อมูลจาก Keypad

อุปกรณ์หลักที่ใช้ในระบบประกอบด้วย Raspberry Pi 4, Arduino Uno R3, RPLiDAR A1M8, มอเตอร์กระแสตรงพร้อม Encoder, วงจรขับมอเตอร์ L298N, จอสัมผัส POS/Kiosk, Keypad 4x4 ร่วมกับ PCF8574, จอ MAX7219 Dot Matrix และแบตเตอรี่ขนาด 12 โวลต์ โดยระบบใช้การสื่อสารผ่าน USB Serial และ ROS 2 เพื่อแลกเปลี่ยนข้อมูลระหว่างส่วนควบคุมระดับสูงกับส่วนควบคุม Hardware

การทดสอบดำเนินการในพื้นที่จำลองภายในอาคาร โดยใช้เส้นทางและตำแหน่งโต๊ะที่กำหนดไว้ล่วงหน้าในรูปแบบ Waypoint ผลการทดสอบตามตารางที่ 4.1 มีรายการทดสอบทั้งหมด 16 รายการ ครอบคลุมการเปิดระบบ การเชื่อมต่ออุปกรณ์ การรับคำสั่งจากจอสัมผัสและ Keypad การเริ่มภารกิจ การสื่อสารระหว่าง Raspberry Pi กับ Arduino การเคลื่อนที่ การเลี้ยว การใช้ Encoder และ Odometry การเดินทางตาม Waypoint การตรวจจับสิ่งกีดขวางด้วย LiDAR การแสดงผลบน LED Matrix การยืนยันรับอาหาร การส่งอาหารไปยังโต๊ะเดียวและหลายโต๊ะ การยกเลิกหรือรีเซ็ตภารกิจ และการทำงานซ้ำหลายรอบ

จากผลการทดสอบพบว่า รายการทดสอบทั้ง 16 รายการสามารถทำงานได้ตามผลที่คาดหวัง คิดเป็นร้อยละ 100 ของรายการทดสอบที่กำหนด ระบบสามารถรับรายการอาหารและกำหนดโต๊ะปลายทางได้ หุ่นยนต์สามารถเดินทางตามเส้นทางที่กำหนด ตรวจพบสิ่งกีดขวางและหยุดการเคลื่อนที่ชั่วคราว เมื่อเส้นทางปลอดภัยสามารถเดินทางต่อได้ นอกจากนี้ยังสามารถรอการยืนยันรับอาหาร ส่งอาหารไปยังโต๊ะหลายโต๊ะตามลำดับ และกลับมายังจุดเริ่มต้นได้

ดังนั้น เมื่อพิจารณาตามวัตถุประสงค์ของโครงการ ต้นแบบหุ่นยนต์ส่งอาหารที่พัฒนาขึ้นสามารถทำงานตามกระบวนการหลักที่กำหนดไว้ได้ และบรรลุวัตถุประสงค์ของการพัฒนาต้นแบบสำหรับการทดสอบ

ในพื้นที่จำลอง อย่างไรก็ตาม ผลการทดสอบดังกล่าวเป็นผลจากการทดลองในขอบเขตที่กำหนดไว้ จึงยังไม่สามารถสรุปได้ว่าระบบพร้อมใช้งานเชิงพาณิชย์ในทุกสภาพแวดล้อม

5.2 อภิปรายผล

ผลการดำเนินงานที่ระบบสามารถทำงานได้ตามรายการทดสอบ เกิดจากการออกแบบสถาปัตยกรรมระบบให้แบ่งหน้าที่ระหว่าง Raspberry Pi และ Arduino Uno อย่างชัดเจน Raspberry Pi รับผิดชอบงานระดับสูง เช่น การแสดงผล POS/Kiosk การจัดการภารกิจ การสื่อสารกับผู้ใช้ และการตัดสินใจเกี่ยวกับเส้นทาง ส่วน Arduino Uno รับผิดชอบงานที่ต้องตอบสนองต่อฮาร์ดแวร์โดยตรง ได้แก่ การควบคุมมอเตอร์ การอ่านค่า Encoder การรับข้อมูลจาก Keypad และการควบคุมจอ LED Matrix การแบ่งหน้าที่ดังกล่าวช่วยลดภาระของแต่ละอุปกรณ์ และทำให้การควบคุมมอเตอร์กับการอ่านสัญญาณ Encoder สามารถทำงานได้อย่างต่อเนื่อง

การเคลื่อนที่ของหุ่นยนต์สอดคล้องกับหลักการ Differential Drive ซึ่งควบคุมความเร็วของล้อซ้ายและล้อขวาแยกจากกัน ทำให้หุ่นยนต์สามารถเดินหน้า ถอยหลัง เลี้ยว และหมุนอยู่กับที่ได้ การใช้ Encoder ช่วยให้ระบบสามารถวัดการหมุนของล้อและนำข้อมูลไปคำนวณระยะทางกับทิศทางการเคลื่อนที่ในรูปแบบ Odometry ข้อมูลดังกล่าวถูกนำไปใช้ในการควบคุมการเดินทางตามระยะและมุมที่กำหนด ผลการทดสอบที่หุ่นยนต์สามารถเดินทางตาม Waypoint และกลับมายังจุดเริ่มต้นได้ แสดงให้เห็นว่าหลักการดังกล่าวสามารถนำมาใช้กับต้นแบบในพื้นที่จำลองได้

นอกจากนี้ การใช้ตัวควบคุมแบบป้อนกลับและ PID ช่วยให้ระบบนำค่าความเร็วหรือการเคลื่อนที่ที่วัดได้จาก Encoder มาเปรียบเทียบกับค่าที่ต้องการ จากนั้นจึงปรับคำสั่ง PWM ให้เหมาะสมกับการเคลื่อนที่ของล้อแต่ละข้าง วิธีการนี้ช่วยลดความแตกต่างของความเร็วระหว่างล้อ และช่วยให้หุ่นยนต์สามารถเคลื่อนที่เป็นเส้นทางตรงหรือหมุนไปยังมุมเป้าหมายได้ดีขึ้น อย่างไรก็ตาม ความคลาดเคลื่อนของ Odometry ยังสามารถสะสมได้จากการสั่นไหวของล้อ พื้นผิวที่ไม่เรียบ น้ำหนักของอาหาร และความคลาดเคลื่อนของการติดตั้งอุปกรณ์

การจัดการภารกิจด้วย Finite State Machine หรือ FSM เป็นอีกปัจจัยหนึ่งที่ทำให้ระบบทำงานเป็นลำดับได้อย่างชัดเจน ระบบเริ่มจากการรอรับรายการอาหาร จากนั้นเข้าสู่สถานะเตรียมการ เดินทางไปยังโต๊ะรอการยืนยันรับอาหาร ตรวจสอบว่ามีโต๊ะถัดไปหรือไม่ และกลับมายังจุดเริ่มต้นเมื่อดำเนินการครบทุกภารกิจ การใช้ FSM ช่วยป้องกันการทำงานซ้ำซ้อน เช่น การเริ่มภารกิจโดยไม่มีโต๊ะปลายทาง หรือการยืนยันรับ

อาหารผิตรายการ นอกจากนี้ยังช่วยให้ระบบสามารถจัดการการยกเลิกภารกิจและการรีเซ็ตกลับเข้าสู่สถานะพร้อมรับงานได้

ในส่วนของระบบตรวจจับสิ่งกีดขวาง LiDAR ถูกนำมาใช้ตรวจสอบพื้นที่ด้านหน้าของหุ่นยนต์ตามระยะและมุมที่กำหนด เมื่อพบสิ่งกีดขวาง ระบบจะหยุดการเคลื่อนที่และรอจนกว่าพื้นที่จะปลอดภัย จึงกลับมาเดินทางต่อจากเส้นทางเดิม การทำงานนี้สอดคล้องกับแนวคิดการใช้ LiDAR เป็น Virtual Bumper เพื่อเพิ่มความปลอดภัย อย่างไรก็ตาม LiDAR ในโครงงานนี้ยังทำหน้าที่เพียงตรวจจับและสั่งหยุด ไม่ได้จำแนกประเภทของวัตถุ ไม่ได้วางแผนเส้นทางหลบหลีก และไม่ได้คำนวณระยะเบรกตามความเร็วของหุ่นยนต์

เมื่อเปรียบเทียบกับแนวคิดและหลักการที่ศึกษาไว้ในบทที่ 2 ระบบที่พัฒนาขึ้นมีความสอดคล้องกับการประยุกต์ใช้ระบบฝังตัวในงานหุ่นยนต์บริการ ทั้งในด้านการแบ่งงานระหว่างไมโครคอนโทรลเลอร์กับคอมพิวเตอร์บอร์ดเดียว การควบคุมมอเตอร์แบบป้อนกลับ การใช้ Encoder การสื่อสารแบบ Serial และการจัดการภารกิจด้วย FSM แต่ระบบยังมีขอบเขตแตกต่างจากระบบนำทางอัตโนมัติเต็มรูปแบบ เนื่องจากการเดินทางยังอาศัย Waypoint และตำแหน่งโตะที่กำหนดไว้ล่วงหน้า แม้ว่าจะมีการใช้ ROS 2 และ LiDAR แต่ยังไม่ได้ใช้ SLAM หรือ Nav2 เพื่อวางแผนเส้นทางไปยังเป้าหมายโดยอัตโนมัติ

ดังนั้น ผลการดำเนินงานจึงแสดงให้เห็นว่าระบบสามารถทำงานได้ตามหลักการที่ออกแบบไว้ในระดับต้นแบบ แต่ความสามารถในการใช้งานจริงยังขึ้นอยู่กับสภาพแวดล้อม ความแม่นยำของการระบุตำแหน่ง ความเสถียรของการสื่อสาร แหล่งจ่ายไฟ และการมีผู้ใช้งานคอยยืนยันการรับอาหาร

5.3 ข้อเสนอแนะ

5.3.1 แนวทางการนำไปใช้

ต้นแบบหุ่นยนต์นี้สามารถนำไปประยุกต์ใช้ในร้านอาหารขนาดเล็ก พื้นที่บริการภายในอาคาร ห้องปฏิบัติการ หรือพื้นที่ที่มีเส้นทางและตำแหน่งโตะค่อนข้างคงที่ โดยสามารถใช้เป็นระบบช่วยลำเลียงอาหารจากจุดเตรียมอาหารไปยังโตะ ลดภาระงานที่ต้องเดินซ้ำของพนักงาน และช่วยแสดงให้เห็นแนวทางการประยุกต์ใช้ Embedded System กับระบบบริการอัตโนมัติ

นอกจากนี้ ระบบสามารถใช้เป็นพื้นฐานสำหรับพัฒนาหุ่นยนต์บริการประเภทอื่น เช่น หุ่นยนต์ขนส่งสิ่งของภายในอาคาร หุ่นยนต์ส่งเอกสาร หรือหุ่นยนต์นำทางในพื้นที่จำกัด โดยสามารถต่อยอดจากโครงสร้างการรับคำสั่ง การจัดการสถานะ การควบคุมมอเตอร์ และระบบตรวจจับสิ่งกีดขวางที่มีอยู่เดิม

5.3.2 ข้อจำกัดของโครงการ

โครงการนี้มีข้อจำกัดหลายประการ

5.3.2.1 ระบบได้รับการทดสอบในพื้นที่จำลองภายในอาคารและใช้เส้นทางแบบ Waypoint ที่กำหนดไว้ล่วงหน้า จึงยังไม่สามารถปรับเส้นทางตามการเปลี่ยนแปลงของตำแหน่งโต๊ะหรือสิ่งกีดขวางได้อย่างอิสระ

5.3.2.2 การระบุตำแหน่งจาก Encoder และ Odometry อาจมีความคลาดเคลื่อนสะสมระหว่างการเคลื่อนที่ โดยเฉพาะเมื่อพื้นผิวไม่เรียบ ล้อเกิดการลื่นไถล หรือมีน้ำหนักบรรทุกแตกต่างจากการทดสอบ

5.3.2.3 ระบบยังต้องพึ่งพาผู้ใช้ในการยืนยันการรับอาหารผ่านจอสัมผัสหรือ Keypad และยังไม่มีเซนเซอร์ตรวจสอบโดยตรงว่าอาหารถูกนำออกจากชั้นวางแล้ว ประการที่สี่ ระบบตรวจจับสิ่งกีดขวางด้วย LiDAR ทำหน้าที่หยุดรถเมื่อพบวัตถุในเขตที่กำหนด แต่ยังไม่สามารถวิเคราะห์ประเภทของวัตถุหรือเลือกเส้นทางใหม่เพื่อหลบหลีกได้

5.3.2.4 การทดสอบยังไม่ได้ประเมินสมรรถนะเชิงปริมาณอย่างละเอียด เช่น เวลาเดินทาง ระยะหยุด ความคลาดเคลื่อนของตำแหน่ง ระยะเวลาการทำงานของแบตเตอรี่ และความสามารถในการรับน้ำหนักที่แตกต่างกัน

5.3.3 แนวทางการปรับปรุงและพัฒนาต่อ

ควรพัฒนาระบบนำทางให้สามารถสร้างและใช้แผนที่ของพื้นที่ รวมถึงรองรับการวางแผนเส้นทางและการหลบหลีกสิ่งกีดขวางแบบพลวัต โดยอาจต่อยอดจาก ROS 2, SLAM และ Nav2 เพื่อให้หุ่นยนต์สามารถปรับเส้นทางได้เมื่อสภาพแวดล้อมเปลี่ยนแปลง

ควรปรับปรุงความแม่นยำของ Encoder และ Odometry ด้วยการสอบเทียบเส้นผ่านศูนย์กลางล้อ ระยะห่างระหว่างล้อ ค่าความละเอียดของ Encoder และผลจากการลื่นไถล รวมทั้งควรทดสอบระบบบนพื้นผิวหลายประเภท เพื่อประเมินความคลาดเคลื่อนของตำแหน่งในสภาพการใช้งานที่แตกต่างกัน

ควรเพิ่มระบบตรวจสอบการรับอาหาร เช่น เซนเซอร์น้ำหนัก เซนเซอร์ตรวจจับ QR Code หรือ RFID เพื่อช่วยยืนยันว่าอาหารถูกนำออกจากชั้นวางแล้ว และลดการพึ่งพาการกดปุ่มจากผู้ใช้เพียงอย่างเดียว นอกจากนี้ควรเพิ่มระบบบันทึกเหตุการณ์และข้อมูลการทำงาน เช่น เวลาเริ่มและสิ้นสุดภารกิจ ระยะทางที่เดินทาง ระยะห่างจากสิ่งกีดขวาง ระยะหยุด อัตราความสำเร็จ และสถานะแบตเตอรี่

สำหรับการทดสอบในอนาคต ควรทดสอบในสภาพแวดล้อมจริงที่มีพื้นผิวแตกต่างกัน น้ำหนักอาหารหลายระดับ แสงและสิ่งกีดขวางหลายรูปแบบ รวมถึงการทำงานต่อเนื่องหลายรอบ ควรบันทึกผลเป็นข้อมูลตัวเลขเพื่อใช้ประเมินประสิทธิภาพและเปรียบเทียบก่อนและหลังการปรับปรุงอย่างเป็นระบบ ทั้งนี้ การพัฒนาต่อควรให้ความสำคัญกับความปลอดภัย ความเสถียรของระบบ การจัดการพลังงาน และความสะดวกในการดูแลรักษา ก่อนนำไปใช้งานในสถานที่จริง.

ภาคผนวก

ภาคผนวก ก การวิเคราะห์ต้นทุน

การวิเคราะห์ต้นทุน (NRE – Non-Recurring Engineering)

ในการพัฒนาระบบหุ่นยนต์ส่งอาหารอัตโนมัติ (Food Delivery Robot) มีการประเมินต้นทุนการออกแบบและพัฒนาที่ไม่เกิดซ้ำ หรือ Non-Recurring Engineering (NRE) รวมประมาณ 13,750 บาท ต้นทุนดังกล่าวเกิดจากงานพัฒนาต้นแบบหลายขั้นตอน เริ่มตั้งแต่การออกแบบวงจรและกำหนดการเชื่อมต่อระหว่าง Raspberry Pi, Arduino Uno, วงจรขับเคลื่อนมอเตอร์ และอุปกรณ์ต่อพ่วง ตามด้วยการพัฒนาโปรแกรมบน Arduino สำหรับควบคุมมอเตอร์ อ่านค่า Encoder และ Keypad รวมถึงแสดงสถานะผ่านจอ Dot Matrix ส่วนโปรแกรมบน Raspberry Pi ครอบคลุมระบบ POS การจัดลำดับภารกิจส่งอาหาร การสื่อสารกับ Arduino และการควบคุมการเดินทางไปยังโต๊ะที่กำหนด

นอกจากนี้ ยังมีงานออกแบบและประกอบโครงสร้างหุ่นยนต์ต้นแบบ เพื่อใช้ติดตั้งชิ้นวาง มอเตอร์ และอุปกรณ์ควบคุม เมื่อประกอบระบบแล้วต้องทดสอบการเคลื่อนที่ การส่งอาหารตามลำดับ และการหยุดรถเมื่อ LiDAR พบสิ่งกีดขวาง รวมถึงปรับแก้ปัญหาที่พบระหว่างทดสอบ การจัดทำรายงานและบันทึกผลการทดสอบเป็นอีกส่วนหนึ่งของงานพัฒนาต้นแบบที่นำมาคิดรวมในต้นทุน NRE

ดังนั้น ต้นทุน NRE ประมาณ 13,750 บาทจึงเป็นต้นทุนที่ประเมินสำหรับการออกแบบและพัฒนาต้นแบบครั้งแรก หากนำแบบและโปรแกรมเดิมไปสร้างหุ่นยนต์เพิ่มเติม ต้นทุนส่วนนี้สามารถเฉลี่ยไปตามจำนวนเครื่องที่ผลิต ทำให้ต้นทุนการพัฒนาเฉลี่ยต่อเครื่องลดลง ทั้งนี้ ตัวเลขดังกล่าวเป็นการประเมินมูลค่างานพัฒนา ไม่ใช่ยอดเงินที่จ่ายจริง และยังไม่รวมราคาวัสดุอุปกรณ์ที่ใช้ประกอบหุ่นยนต์แต่ละเครื่อง

รายละเอียดค่าใช้จ่ายในการพัฒนาครั้งเดียว (Non-Recurring Engineering: NRE) คิดเป็นเงินประมาณ 25,850 บาท

1. ออกแบบวงจรไฟฟ้าและระบบ Embedded System

งาน: กำหนดโครงสร้างระบบ เลือกอุปกรณ์ และออกแบบการเชื่อมต่อระหว่าง Raspberry Pi, Arduino Uno, วงจรขับเคลื่อนมอเตอร์ และอุปกรณ์ต่อพ่วง

เวลา: 12 ชั่วโมง × 125 บาท/ชั่วโมง

รวม = 1,500 บาท

2. พัฒนาโปรแกรมควบคุม Arduino Uno

งาน: เขียนและปรับโปรแกรมควบคุมมอเตอร์ อ่านค่า Encoder และ Keypad รวมถึงแสดงสถานะผ่านจอ Dot Matrix

เวลา: 20 ชั่วโมง × 125 บาท/ชั่วโมง

รวม = 2,500 บาท

3. พัฒนาระบบ POS และโปรแกรมควบคุมภารกิจ

งาน: พัฒนาหน้าจอรับรายการ ระบบจัดลำดับการส่งอาหารแบบ FSM และการสื่อสารระหว่าง Raspberry Pi กับ Arduino Uno

เวลา: 25 ชั่วโมง × 125 บาท/ชั่วโมง

รวม = 3,125 บาท

4. พัฒนาและปรับระบบนำทาง

งาน: ตั้งค่า ROS 2 และ Waypoint เชื่อมต่อ LiDAR เพื่อตรวจสอบสิ่งกีดขวาง และปรับการเคลื่อนที่ของหุ่นยนต์ในพื้นที่ทดสอบ

เวลา: 20 ชั่วโมง × 125 บาท/ชั่วโมง

รวม = 2,500 บาท

5. ออกแบบและประกอบโครงสร้างหุ่นยนต์ต้นแบบ

งาน: จัดวางอุปกรณ์ ติดตั้งมอเตอร์ ชั้่นวางอาหาร และโครงสร้างสำหรับทดสอบการใช้งาน

เวลา: 10 ชั่วโมง × 125 บาท/ชั่วโมง

รวม = 1,250 บาท

6. ประกอบวงจรและทดสอบระบบร่วมกัน

งาน: เดินสาย ติดตั้งอุปกรณ์ โหลดโปรแกรม ทดสอบการทำงานร่วมกัน และแก้ไขปัญหาที่พบระหว่างทดสอบ

เวลา: 15 ชั่วโมง × 125 บาท/ชั่วโมง

รวม = 1,875 บาท

7. จัดทำเอกสารและแผนการทดสอบ

งาน: จัดทำรายงาน บันทึกผลการทดสอบ และตรวจสอบขั้นตอนการทำงานของระบบ

เวลา: 8 ชั่วโมง × 125 บาท/ชั่วโมง

รวม = 1,000 บาท

รวมเวลา 110 ชั่วโมง คิดเป็นค่า NRE ประมาณ 13,750 บาท

หมายเหตุ: การประมาณค่าใช้จ่าย NRE (Non-Recurring Engineering) ในรายงานฉบับนี้ เป็นการประเมินเบื้องต้นเพื่อประกอบการศึกษาและวิเคราะห์เท่านั้น ตัวเลขที่แสดงมิได้มีผลผูกพันเชิงพาณิชย์หรือทำบัญชีโดยตรง

รายละเอียด unit cost metrics คิดเป็นราคา 13,008 บาท

วัสดุ และอุปกรณ์	จำนวน	ราคาต่อหน่วย	ราคารวม (บาท)
Step Down DC-DC 10A Buck Step-down	1	118	118
แกนต่อมอเตอร์หกลีบ Extended motor shaft 30mm [6mm]	4	33	132
UNO R3 แบบถอดชิปได้ พร้อมสาย USB	1	159	159
ABS Case พร้อมพัดลม สำหรับ Raspberry Pi 4 (OEM)	1	200	200
Kingston microSD Card 64GB Canvas Select Plus	1	590	590
ZENOVA แบตเตอรี่ลิเธียม 12V ความจุ 15Ah + เครื่องชาร์จ	1	383	383
RPLIDAR A1M8-R6 - 360 Degree Laser Scanner	1	3,585	3,585
Raspberry Pi 4 Model B Single Board Computer 4GB	1	4,450	4,450
สวิตช์ DC 12V	1	29	29
L298N Motor Driver Module	1	80	80
PCF8574	1	50	50
Keypad Matrix 4x4	1	20	20
LED Matrix 32x8	1	148	148
Arduino UNO Terminal Shield	1	82	82

Bus Bar ขนาด 14 ช่อง	2	45	90
Bus Bar ขนาด 5 ช่อง	1	25	25
มอเตอร์ DC Encoder JGB37-520	2	182	364
ล้อยางขนาด 130 mm	2	168	335
มอดูลอ่านความจุแบตเตอรี่ Rideon DC 8V-72V	1	81	81
หน้าจอสัมผัส Aprotii LCD DHMI 7 นิ้ว	1	1,000	1,000
สาย Micro HDMI to HDMI ยาว 1.5 m	1	137	137
สาย Micro USB to USB A ยาว 2 m	1	86	86
สาย USB B to USB A ยาว 2 m	1	55	55
อลูมิเนียมโปรไฟล์ 20x20 ยาว 1 m	1	205	205
ข้อต่ออลูมิเนียมโปรไฟล์	8	13	104
แผ่นอะคริลิกหนา 5 mm ขนาด 60 x 60 cm	1	200	200
ชั้นวางอเนกประสงค์	1	250	250
สายไฟจัมเปอร์			200
น็อต สกรู			100
รวมทั้งสิ้น			13,258

ราคา unit cost metrics รวมเป็นเงิน 13,258 บาท

ดังนั้น เมื่อได้ NRE cost และ unit cost แล้วจะสามารถนำมาคำนวณต้นทุน เพื่อหาราคาขายได้ โดยต้นทุน NRE เป็นต้นทุนคงที่ (Fixed Cost) ที่เกิดขึ้นครั้งเดียว เมื่อผลิตจำนวนมากขึ้น ต้นทุน NRE ต่อเครื่องจะถูกเฉลี่ยลดลง

$$\text{ต้นทุนต่อเครื่อง} = \text{Unit Cost} + \frac{\text{NRE}}{\text{จำนวนเครื่องที่วางแผนผลิต (Q)}}$$

- กรณีผลิตเครื่องเดียว (ทำเป็นต้นแบบ/Custom Made 1 เครื่อง):

- ต้นทุน = $13,258 + 13,750 = 27,008$ บาท

- กรณีผลิตล็อตเล็ก (Batch เล็ก เช่น 5 เครื่อง):

- ค่า NRE ต่อเครื่อง = $\frac{13,750}{5} = 2,750$ บาท

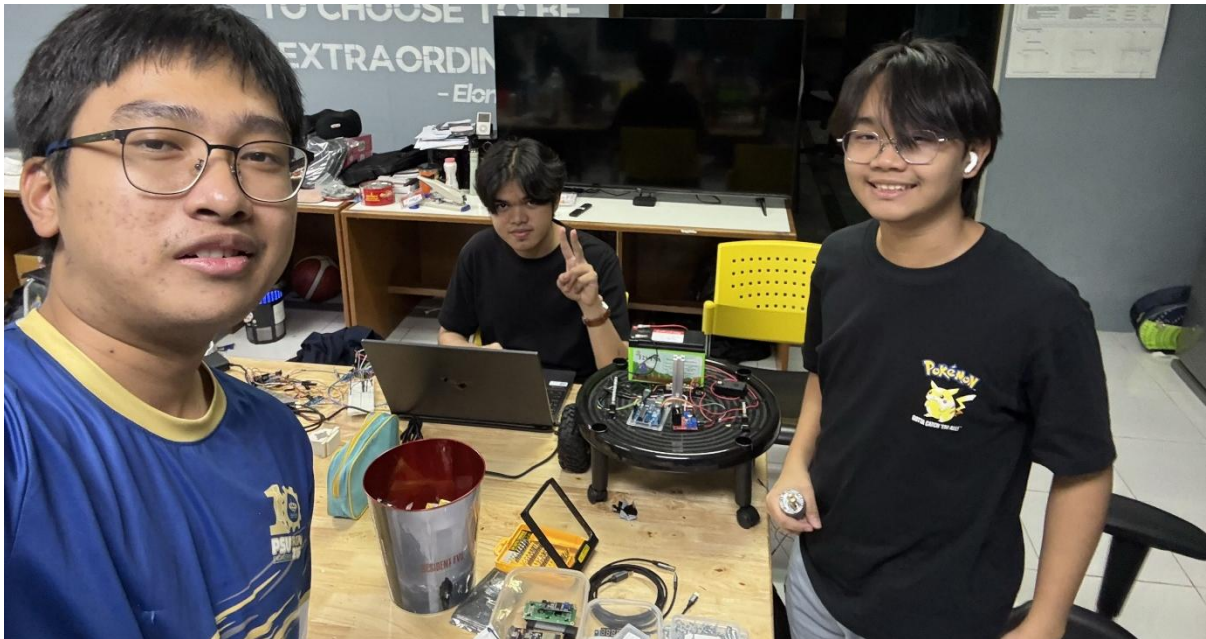
- ต้นทุน = $13,258 + 2,750 = 16,008$ บาท/เครื่อง
- กรณีผลิต 10 เครื่อง:
 - ค่า NRE ต่อเครื่อง = $\frac{13,750}{10} = 1,375$ บาท
 - ต้นทุน = $13,258 + 1,375 = 14,633$ บาท/เครื่อง
- กรณีผลิตจำนวนมาก (เช่น 50 เครื่องขึ้นไป):
 - ค่า NRE ต่อเครื่องจะเหลือเพียง $\frac{13,750}{50} \approx 275$ บาท ต้นทุนจะใกล้เคียง $13,500$ บาท/เครื่อง

หากต้องการตั้งราคาขายแบบขายเครื่องเมื่อต้องการ margin กำไร ที่ประมาณ 70 % จะคำนวณราคาขายได้ดังนี้

- กรณีผลิตเครื่องเดียว
 - ราคาต้นทุน $27,008$ บาท
 - กำไรที่ต้องการ 70% จะได้ $\frac{170}{100} \times 27,008 = 45,913$ บาท
- กรณีผลิต 50 เครื่องขึ้นไป
 - ราคาต้นทุน $13,500$ บาท
 - กำไรที่ต้องการ 70% จะได้ $\frac{170}{100} \times 13,500 = 22,950$ บาท

เห็นได้ว่าจำนวนการผลิตนั้นส่งผลต่อราคาขายเป็นอย่างมาก ดังนั้นหากต้องการนำไปขายจริงในตลาด หุ่นยนต์โดยใช้โมเดลธุรกิจแบบซื้อขาด การลดต้นทุน (unit cost) และการเพิ่มจำนวนการผลิต จะช่วยให้ราคาขายลดลง และดึงดูดผู้ซื้อได้มากขึ้น

ภาคผนวก ข ภาพการทำงาน



รูปที่ x ภาพบรรยากาศการทำงาน การประกอบชิ้นงาน หุ่นยนต์ในชั้นที่ 1



รูปที่ x ภาพบรรยากาศการทำงาน ประกอบชิ้นวางของของหุ่นยนต์ครบทุกชิ้น



รูปที่ x ภาพบรรยากาศการทดสอบหุ่นยนต์ในสถานที่จำลอง บริเวณอาคารหุ่นยนต์ ใต้สาขาวิศวกรรม
คอมพิวเตอร์